Computational Fluid Dynamics applied to urban cases

Modelling wind and dispersion in urban environments

Text book references: Computational Fluid Dynamics, a practical approach, J.Tu
Computational Methods for Fluid Dynamics, Ferziger and Peric,

TU Delft

Learning Objectives:

After this class, you will be able to:

- Understand the fundamentals of theory behind CFD
- Understand the best practice guidelines of urban flows

Activities during the class:

During this class, you will perform the following activities:

1. Hands-on comparisons with OpenFOAM laboratories

NO summative assignments

Activities during the class:

During this class, you will perform the following activities:

1. Hands-on comparisons with OpenFOAM laboratories

NO summative assignments

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Outline of the class:

1. **Introduction**

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

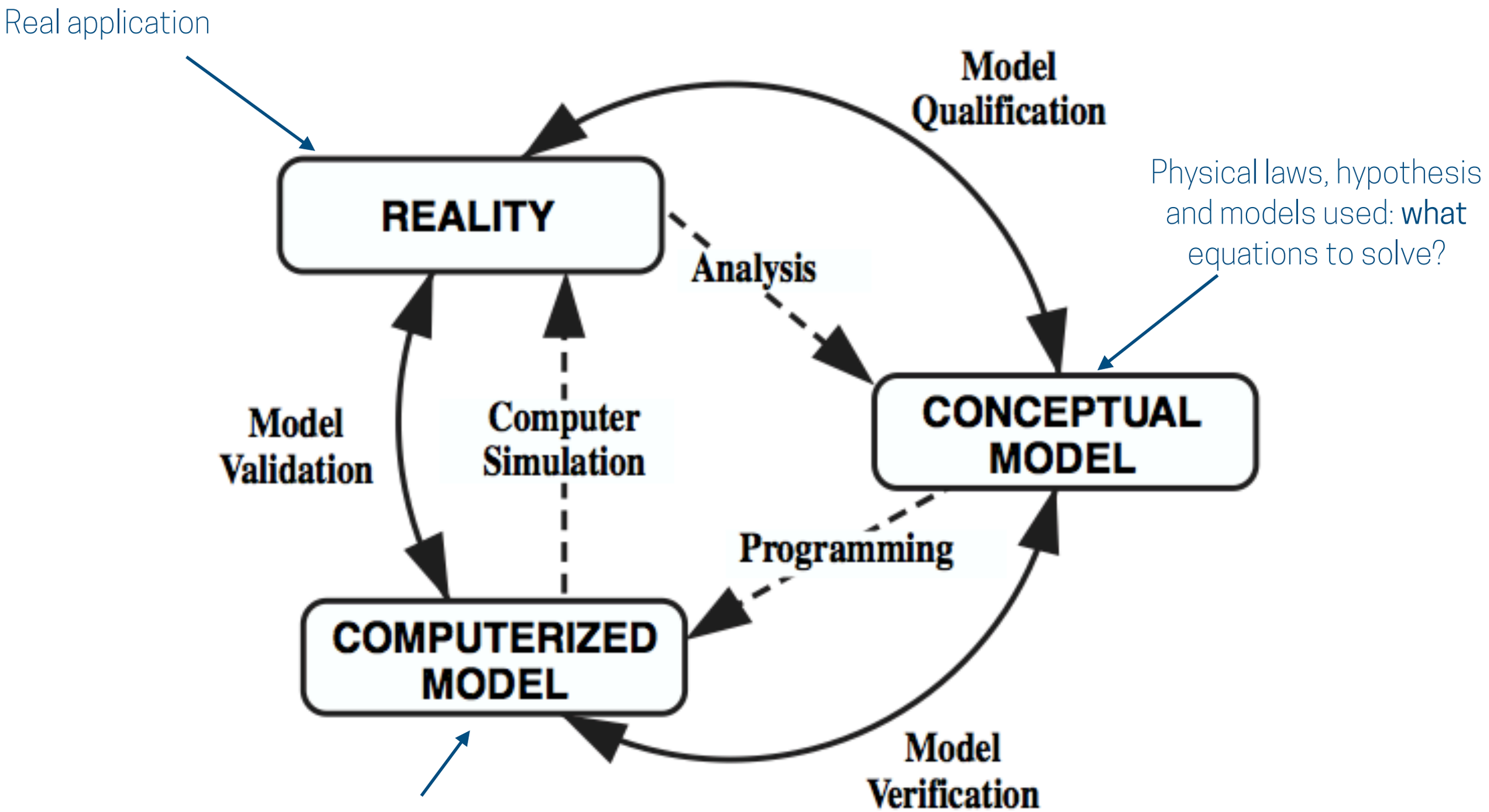
For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

↓

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

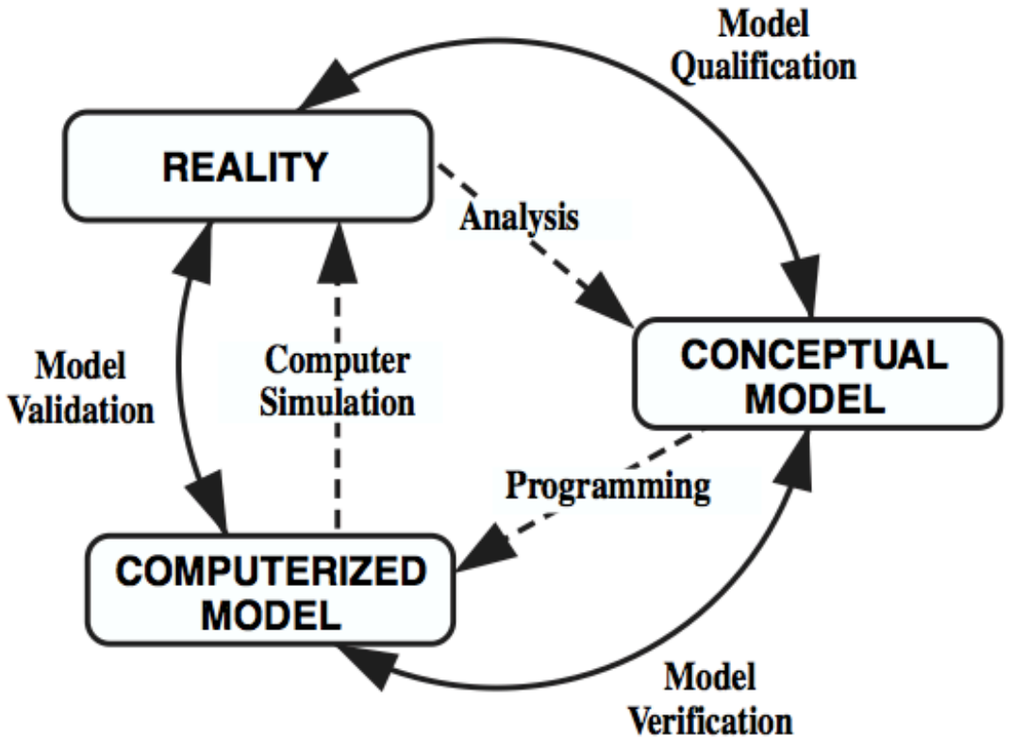
Computational fluid dynamics represents an alternative -or at least complementary- method.

Introduction:



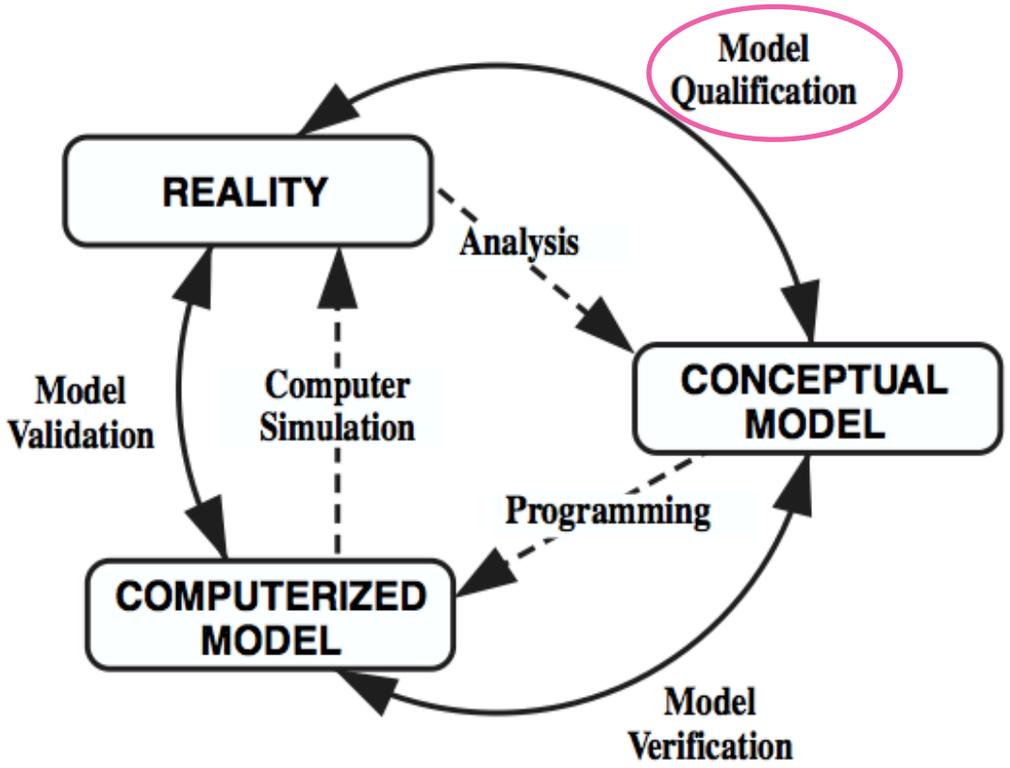
[Schlesinger S., Simulation, 1979]

Introduction:



Introduction:

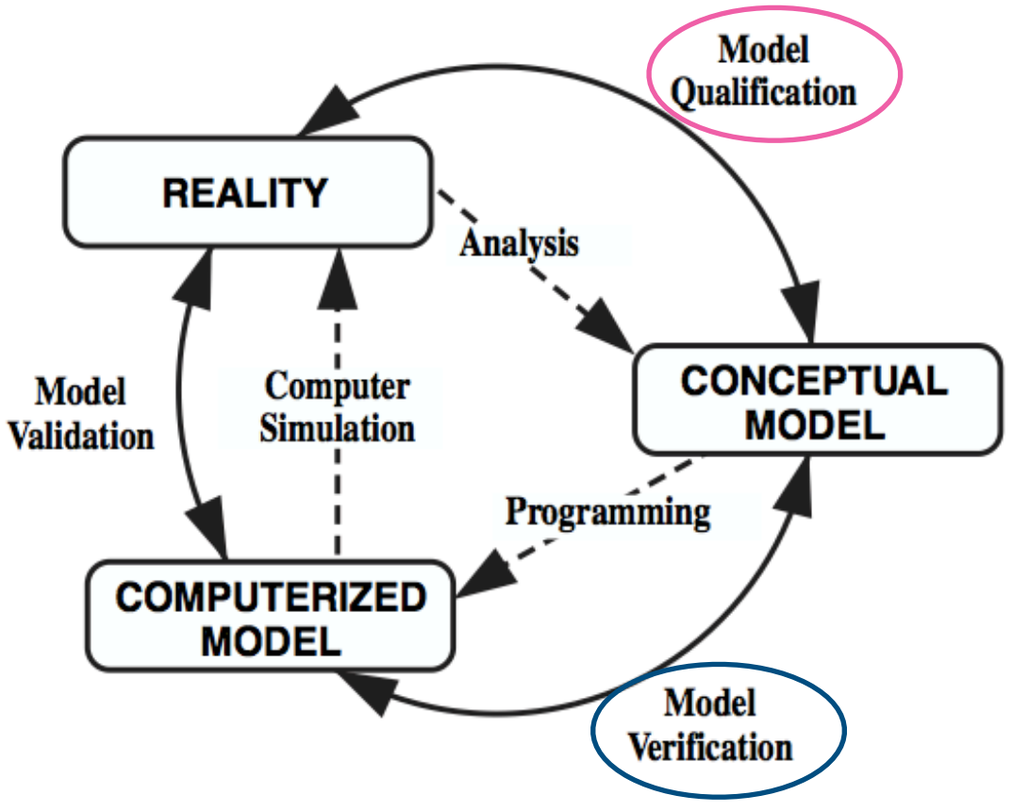
Qualification: is the conceptual model appropriate to provide acceptable level of agreement with the application? Are we solving the correct equations?



Introduction:

Qualification: is the conceptual model appropriate to provide acceptable level of agreement with the application? Are we solving the correct equations?

Verification: 1) model implementation accurately represents the conceptual model; 2) the solution of the model is accurate. Are we solving the equations correctly?

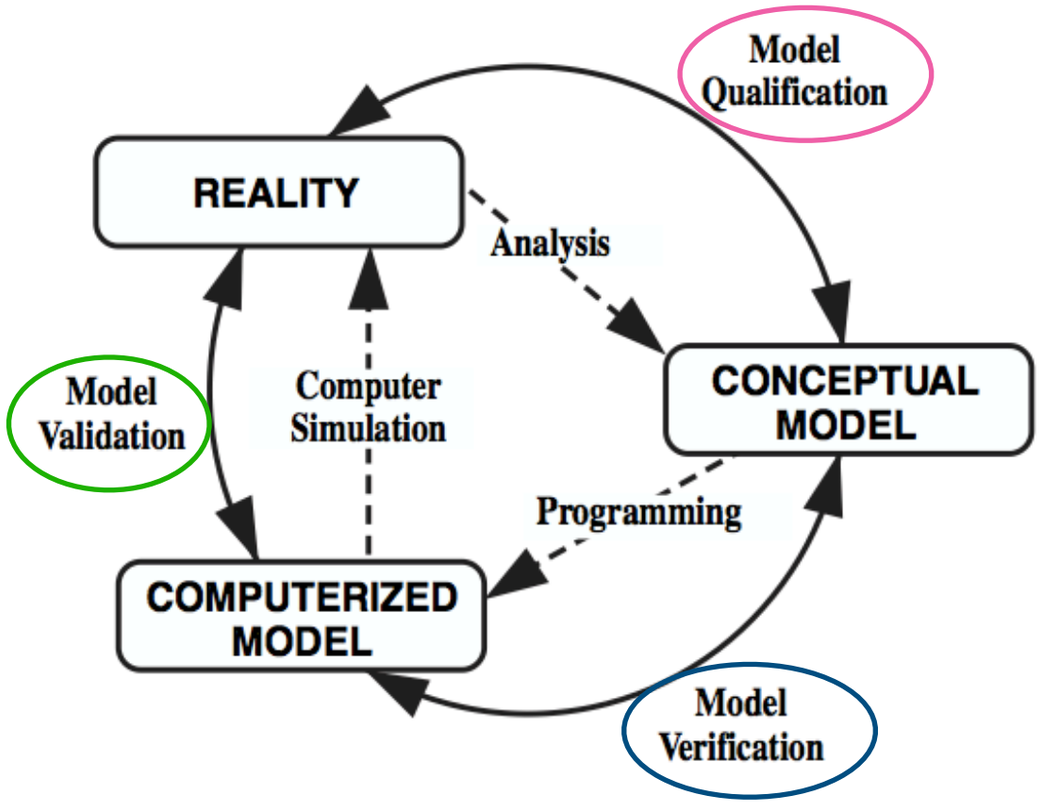


Introduction:

Qualification: is the conceptual model appropriate to provide acceptable level of agreement with the application? Are we solving the correct equations?

Verification: 1) model implementation accurately represents the conceptual model; 2) the solution of the model is accurate. Are we solving the equations correctly?

Validation: determine to which degree the model is an accurate representation of reality from the perspective of the intended application

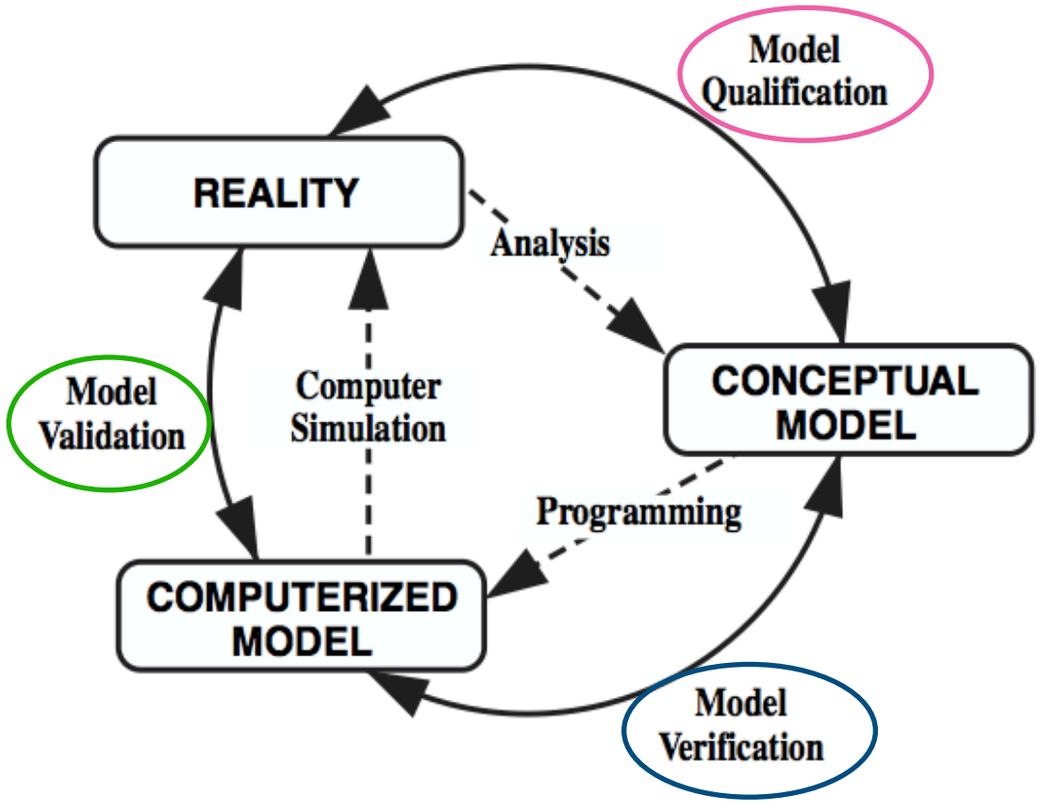


Introduction:

Qualification: is the conceptual model appropriate to provide acceptable level of agreement with the application? Are we solving the correct equations?

Verification: 1) model implementation accurately represents the conceptual model; 2) the solution of the model is accurate. Are we solving the equations correctly?

Validation: determine to which degree the model is an accurate representation of reality from the perspective of the intended application



Once the verification of the code is one, qualification == validation

Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.

Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.

Flows can be described by PDEs

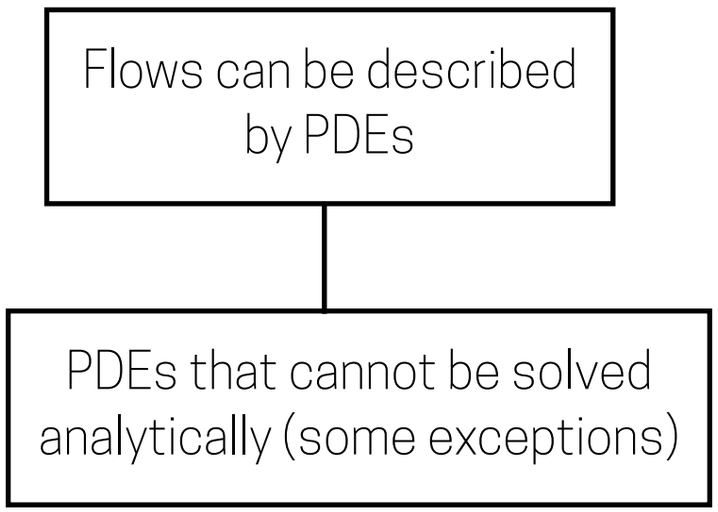
Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.



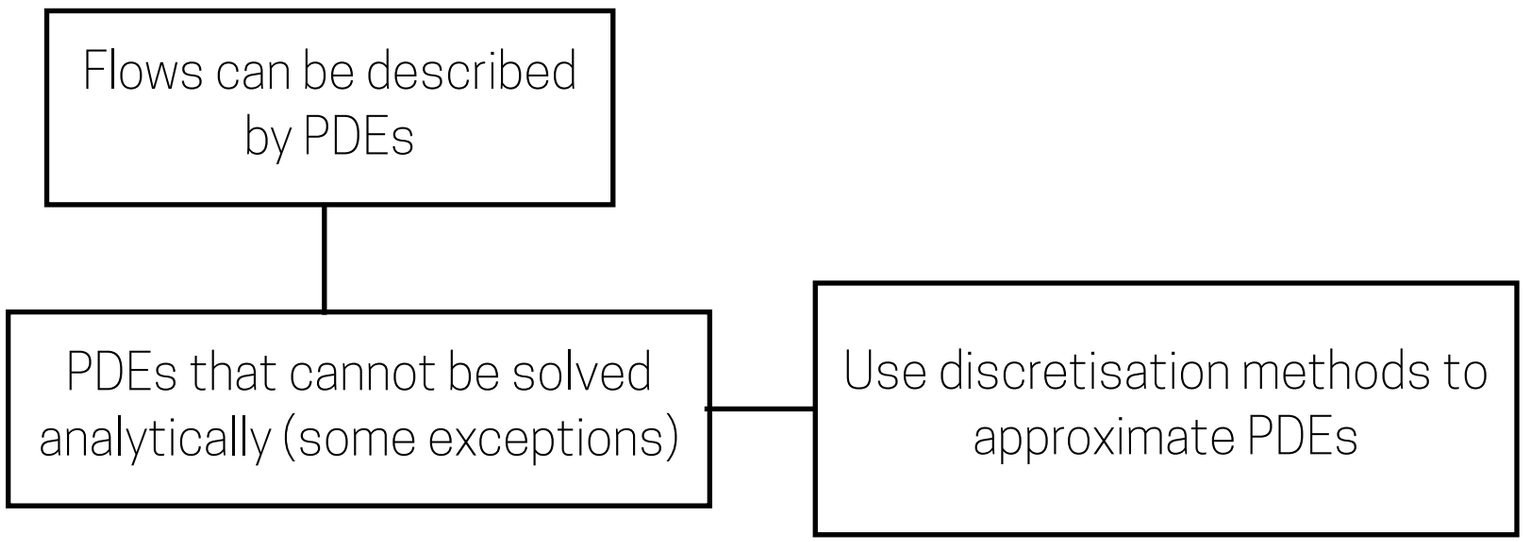
Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.



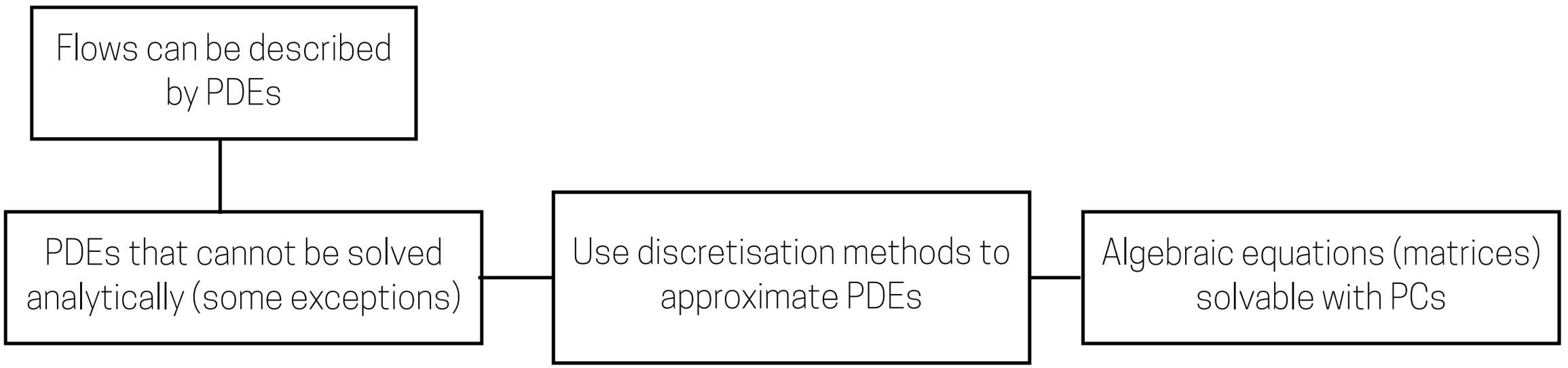
Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.



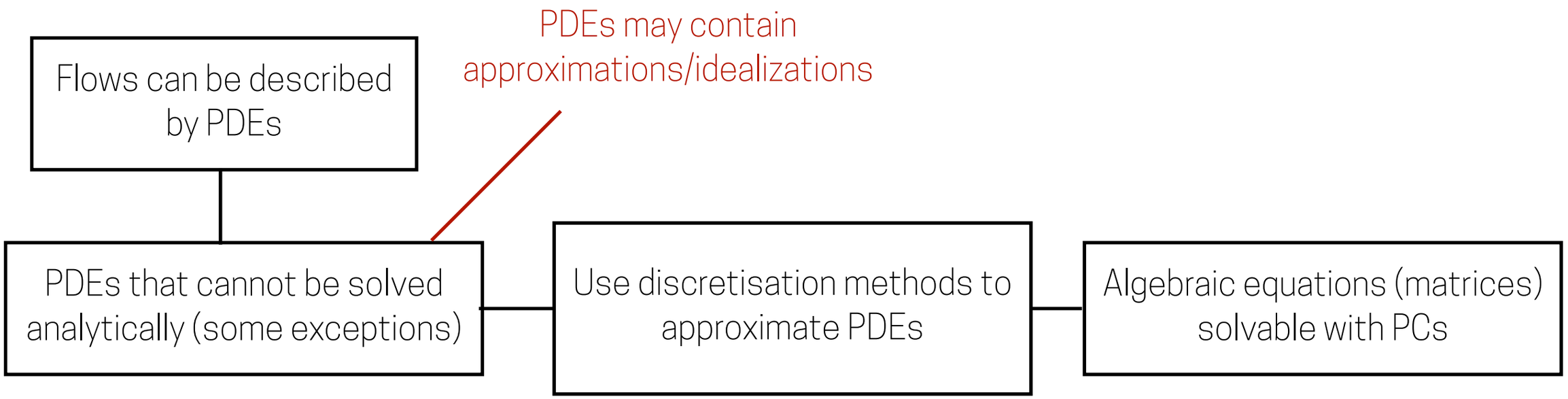
Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.



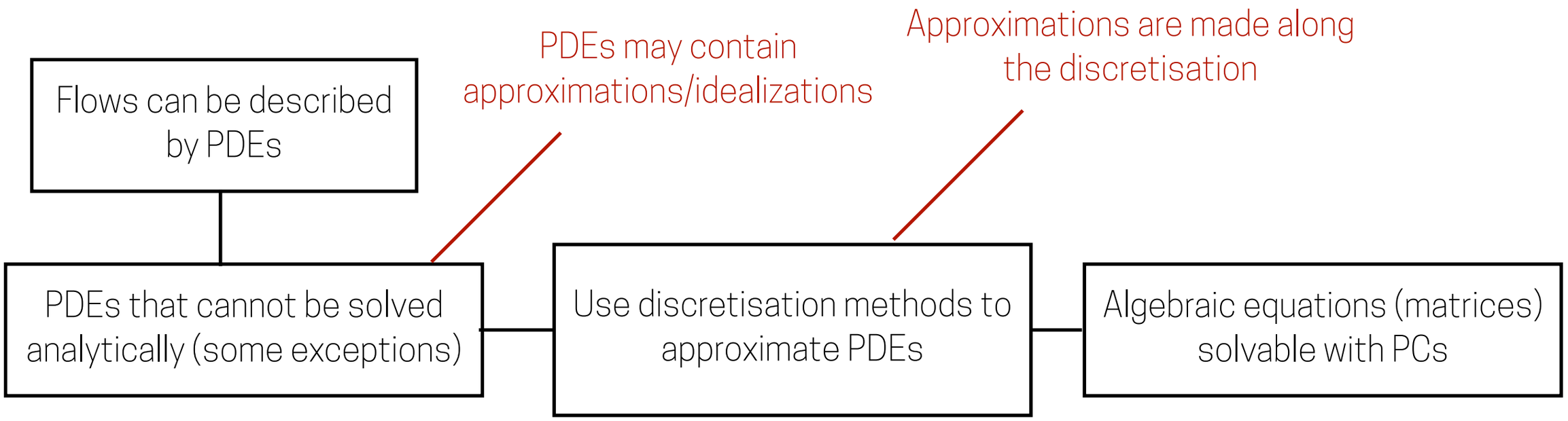
Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.



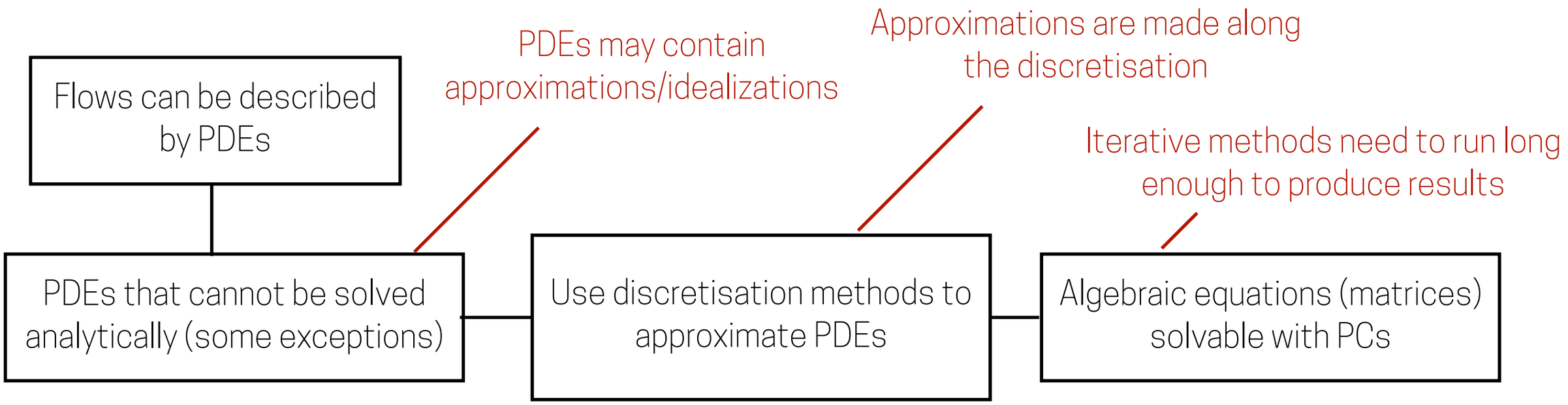
Introduction:

The equations of fluid mechanics are solvable for only a limited number of flows.

For other type of flows they can be reproduced with experiments if the Reynolds number that characterises the phenomenon can be matched.

Which implies that the speed of the flow needs to increase, in order to achieve the same Reynolds number for some cases such as the flow around aircrafts or cities.

Computational fluid dynamics represents an alternative -or at least complementary- method.



Introduction:

Mathematical formulation

PDE

IC/BC

Discretization Step

Numerical formulation

ADE

Solution Step

Numerical Solution

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Mentimeter

Mathematical Model

Which equations do we need to model to know…:

1. …the wind speed in a city environment if the stratification of the atmospheric boundary layer is neutral?
2. …the ventilation inside a building?
3. …the positions of air filters in a climbing gym?

Questions so far?

Outline of the class:

1. Introduction

2. **Mathematical Model**

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Mathematical Model

$$\frac{\partial \overline{U_j}}{\partial x_j} = 0$$

$$\frac{\partial \overline{U_i}}{\partial t} + \overline{U_j}\frac{\partial \overline{U_i}}{\partial x_j} = -\delta_{i3}g + f_c\varepsilon_{ij3}\overline{U_j} - \frac{1}{\overline{\rho}}\frac{\partial \overline{P}}{\partial x_i} + \frac{\partial}{\partial x_j}\left[\frac{(\mu+\mu_t)}{\rho}\frac{\partial \overline{U_i}}{\partial x_j}\right]$$

$$\overline{U_j}\frac{\partial k}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\frac{\mu+\mu_t}{\rho\sigma_k}\right)\frac{\partial k}{\partial x_j}\right] + P_k - \varepsilon$$

$$\overline{U_j}\frac{\partial \varepsilon}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\frac{\mu+\mu_t}{\rho\sigma_\varepsilon}\right)\frac{\partial \varepsilon}{\partial x_j}\right] + C_{\varepsilon 1}\frac{\varepsilon}{k}P_k - C_{\varepsilon 2}\frac{\varepsilon^2}{k}$$

$$\mu_t = \rho C_\mu \frac{k^2}{\varepsilon}$$

Mathematical Model

$$\frac{\partial \overline{U_j}}{\partial x_j} = 0$$

$$\frac{\partial \overline{U_i}}{\partial t} + \overline{U_j}\frac{\partial \overline{U_i}}{\partial x_j} = -\delta_{i3}g + f_c\varepsilon_{ij3}\overline{U_j} - \frac{1}{\overline{\rho}}\frac{\partial \overline{P}}{\partial x_i} + \frac{\partial}{\partial x_j}\left[\frac{(\mu+\mu_t)}{\rho}\frac{\partial \overline{U_i}}{\partial x_j}\right]$$

$$\overline{U_j}\frac{\partial k}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\frac{\mu+\mu_t}{\rho\sigma_k}\right)\frac{\partial k}{\partial x_j}\right] + P_k - \varepsilon$$

$$\overline{U_j}\frac{\partial \varepsilon}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\frac{\mu+\mu_t}{\rho\sigma_\varepsilon}\right)\frac{\partial \varepsilon}{\partial x_j}\right] + C_{\varepsilon 1}\frac{\varepsilon}{k}P_k - C_{\varepsilon 2}\frac{\varepsilon^2}{k}$$

$$\mu_t = \rho C_\mu \frac{k^2}{\varepsilon}$$

It also includes the initial and boundary conditions, values that we set-up to start a simulation —> more on this will be explained in the next class when we see governing equations

Questions so far?

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

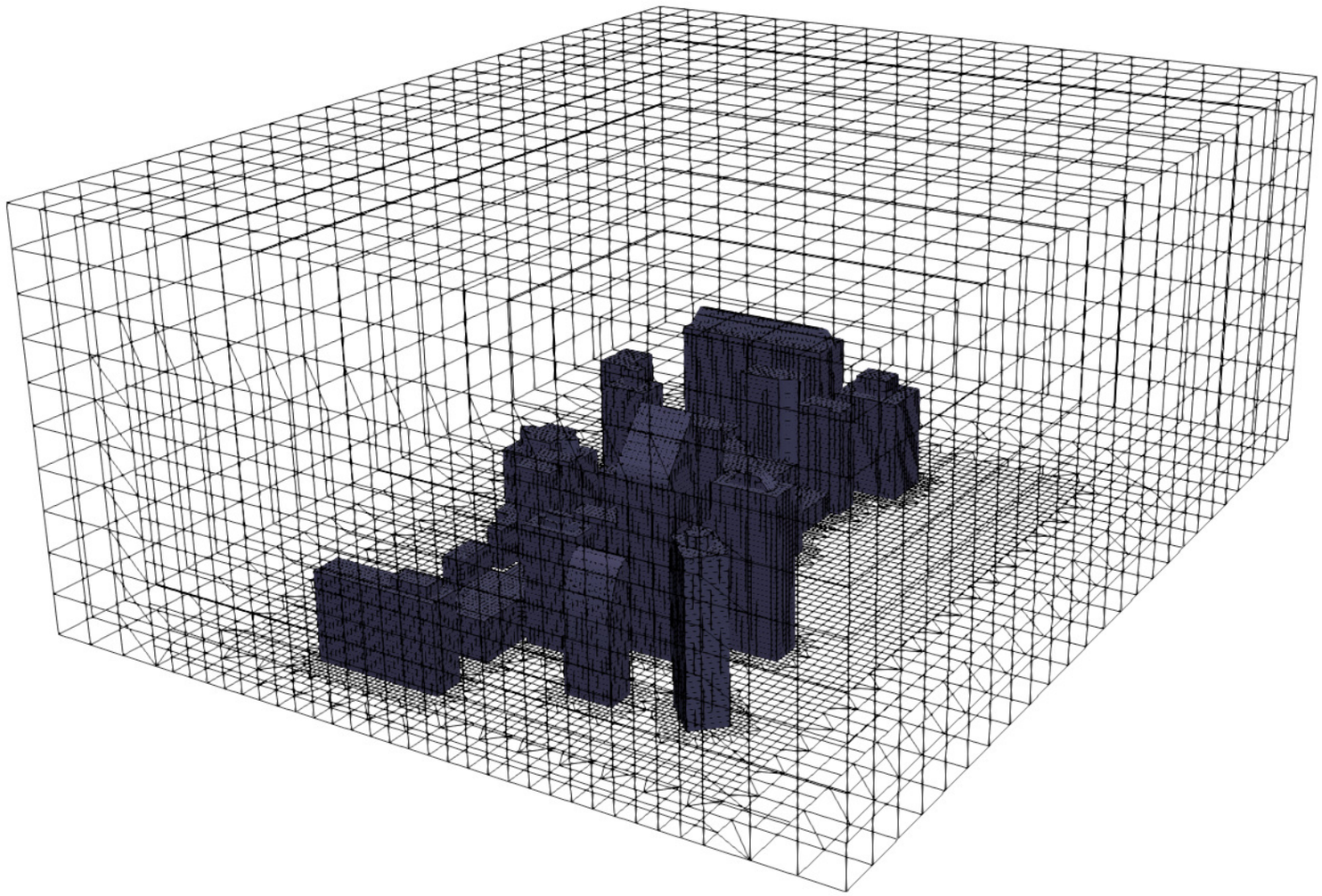
6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

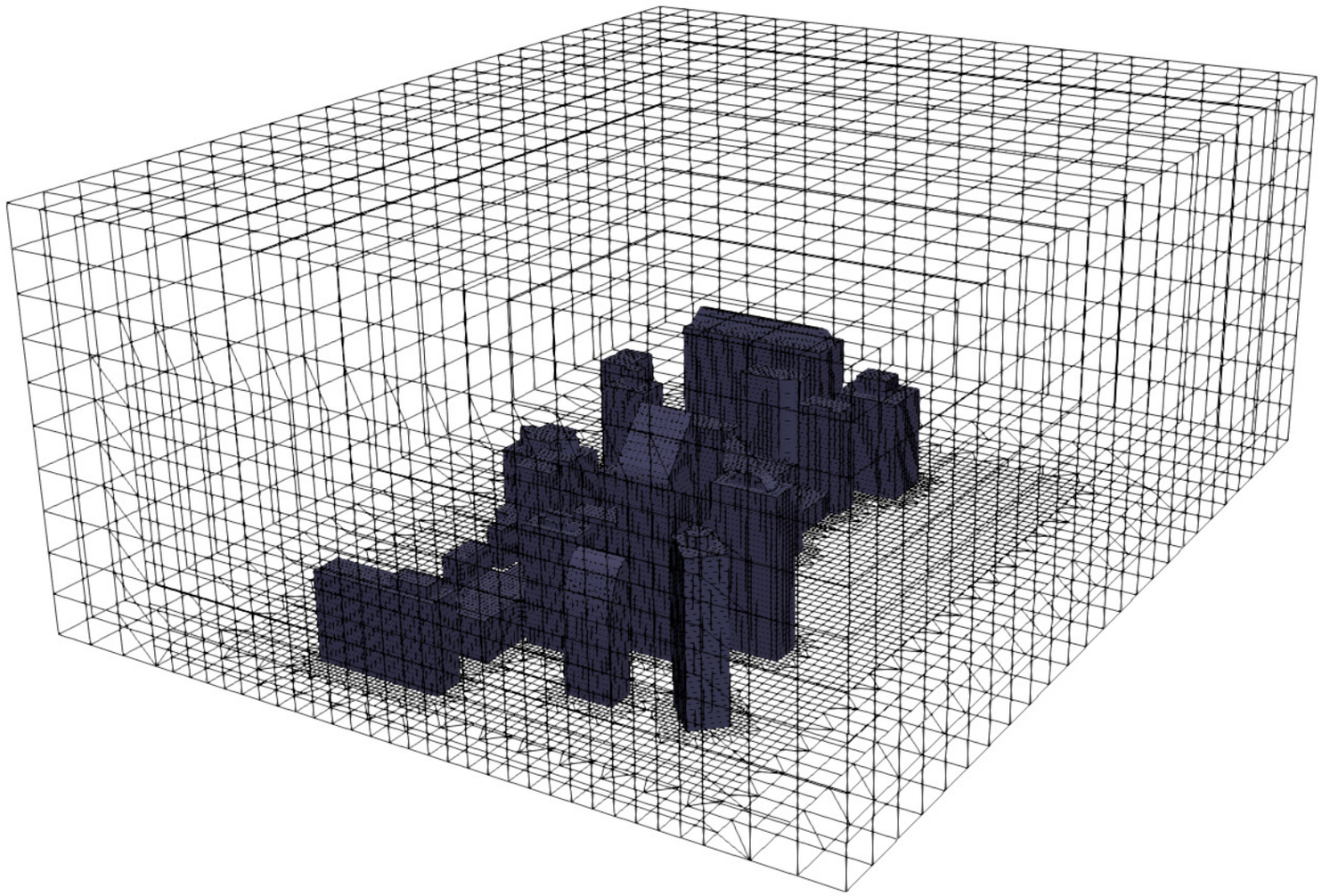
10. Wrap-up

Mesh design and generation



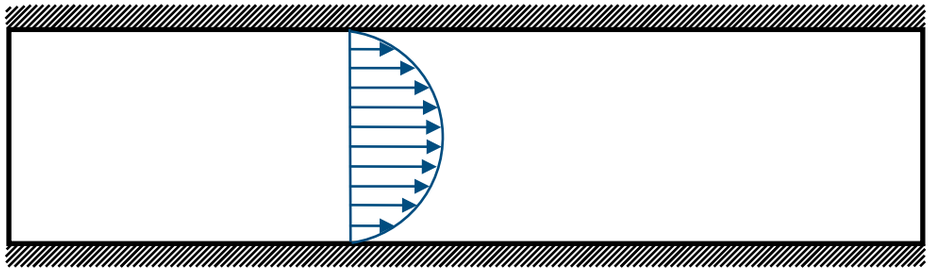
Mesh design and generation

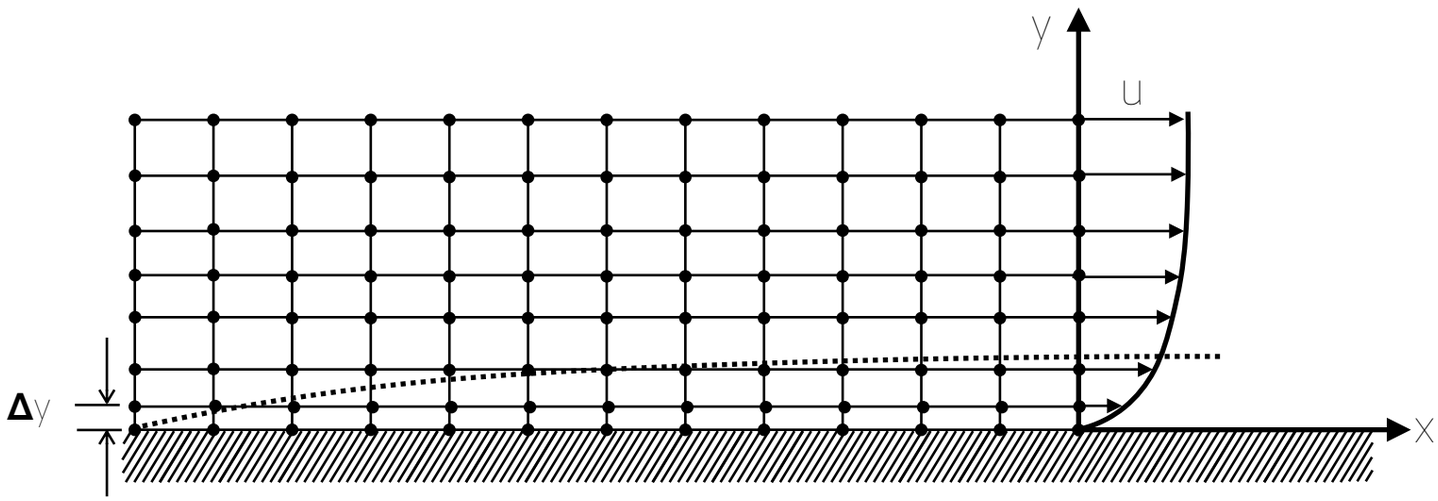
The numerical grid defines the discrete locations at which the variables need to be calculated, this is essentially a discrete representation of the geometric domain where the fluid is confined



Mesh design and generation

Importance of mesh spacing

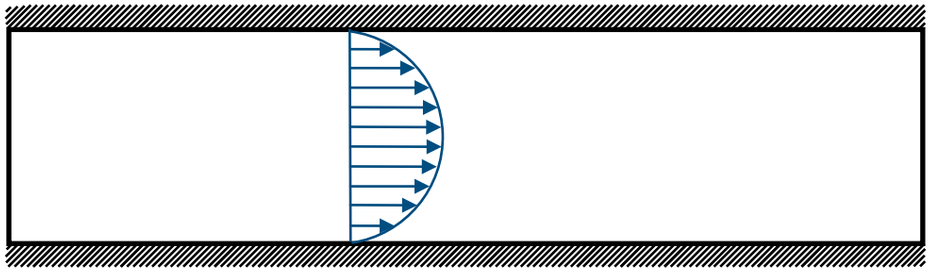


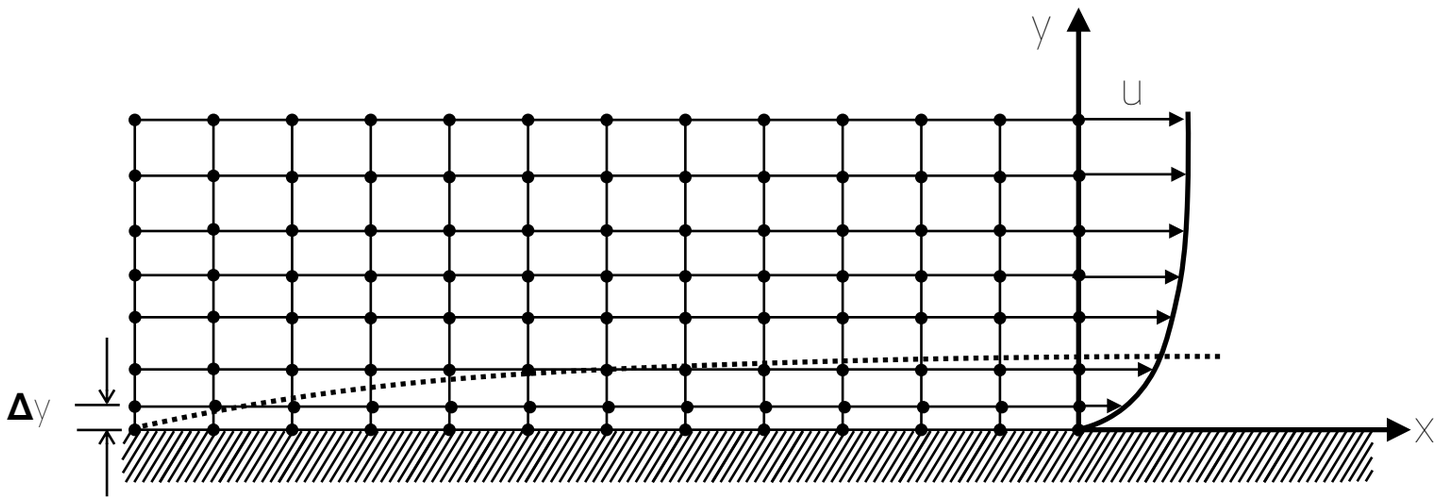


Mesh design and generation

Importance of mesh spacing

Mesh spacing is fundamental to capture flow features:

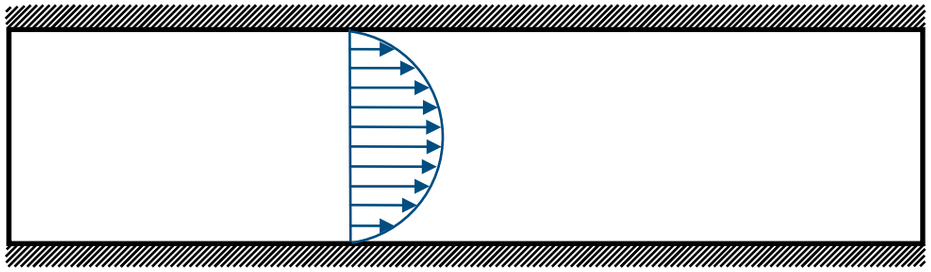




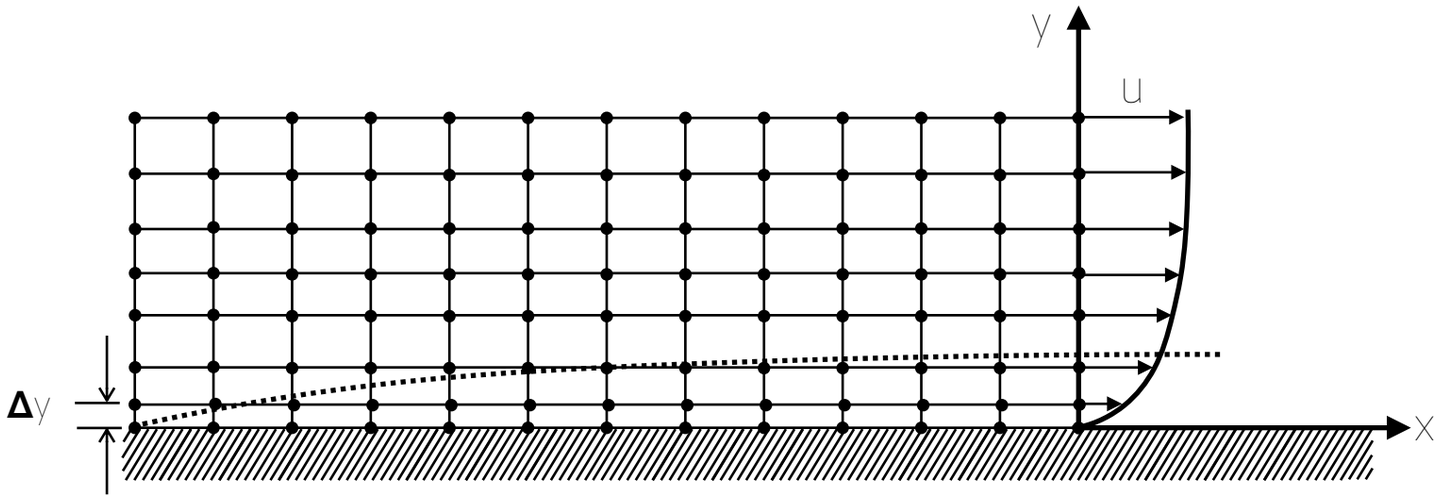
Mesh design and generation

Importance of mesh spacing

Mesh spacing is fundamental to capture flow features:



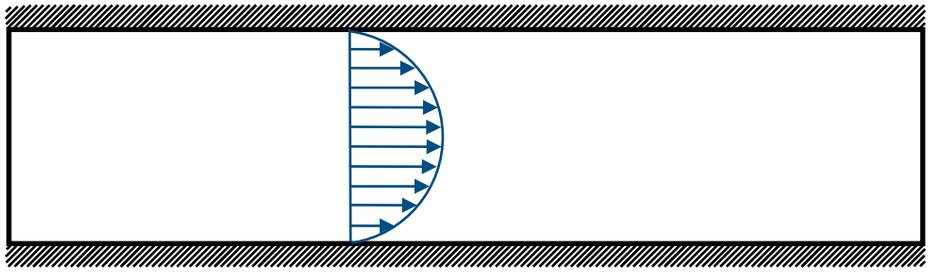
Uniform mesh spacing



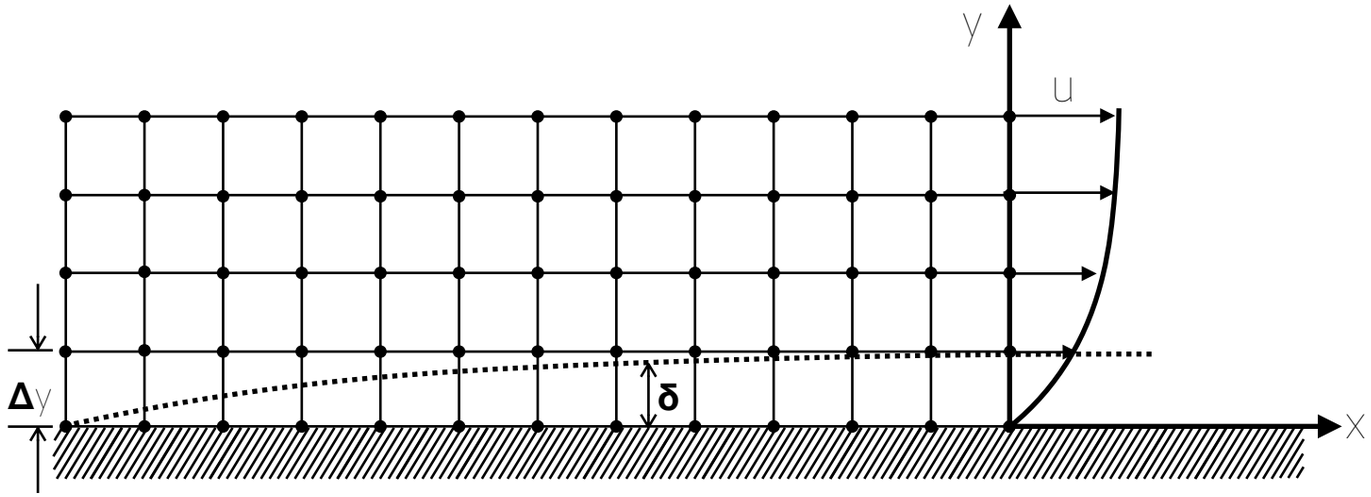
Mesh design and generation

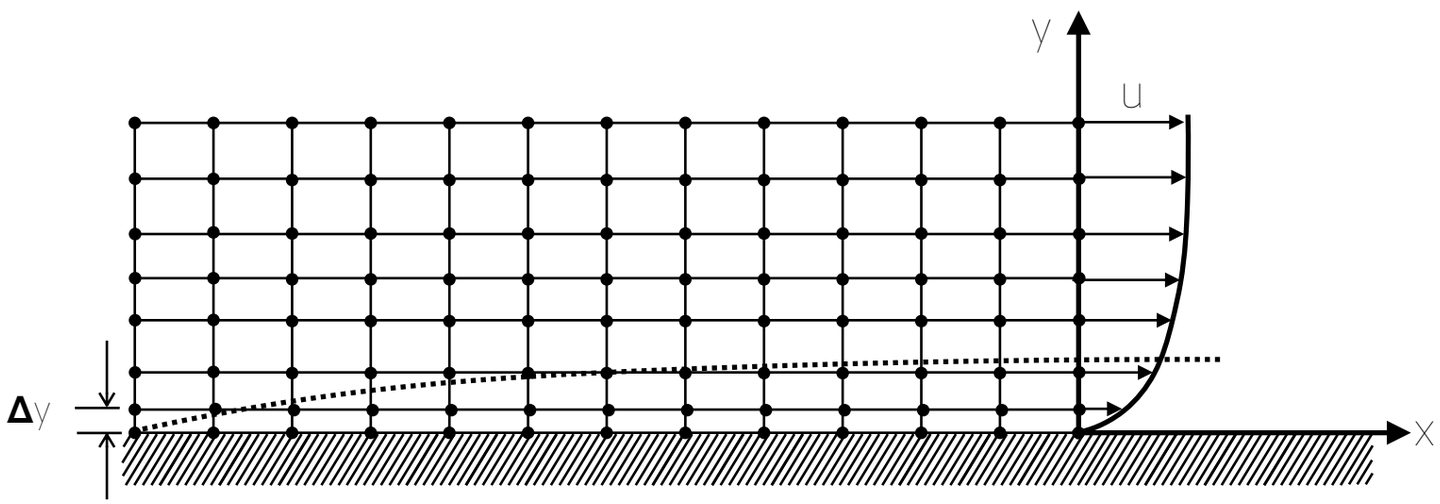
Importance of mesh spacing

Mesh spacing is fundamental to capture flow features:



Uniform mesh spacing

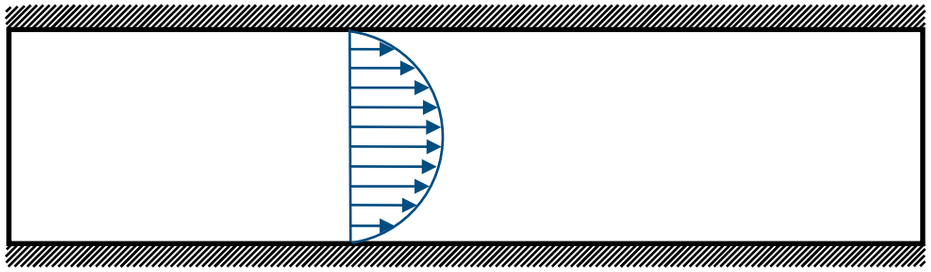




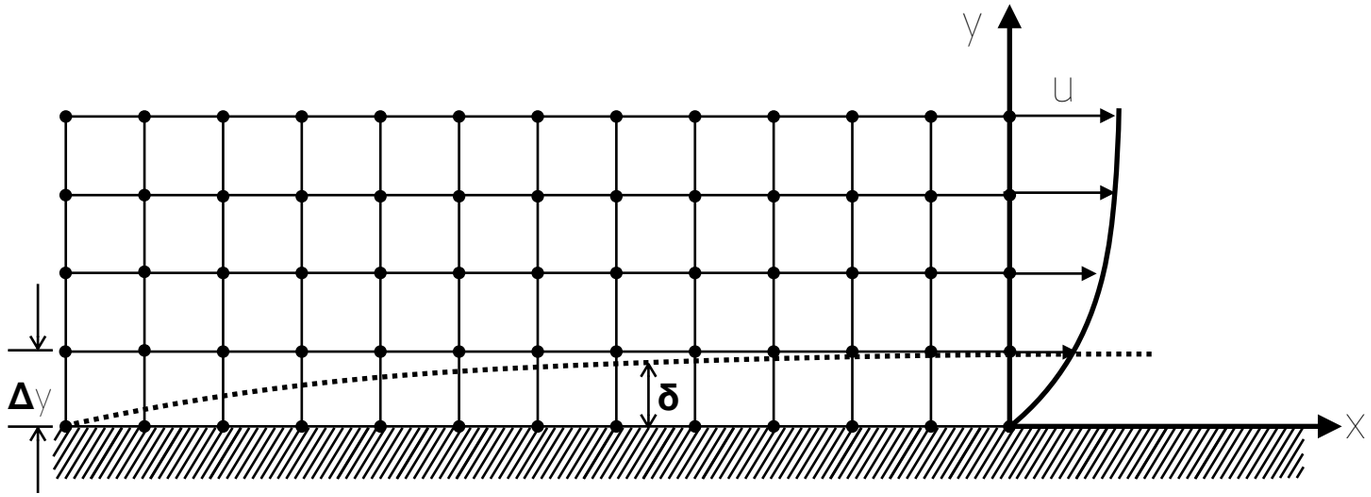
Mesh design and generation

Importance of mesh spacing

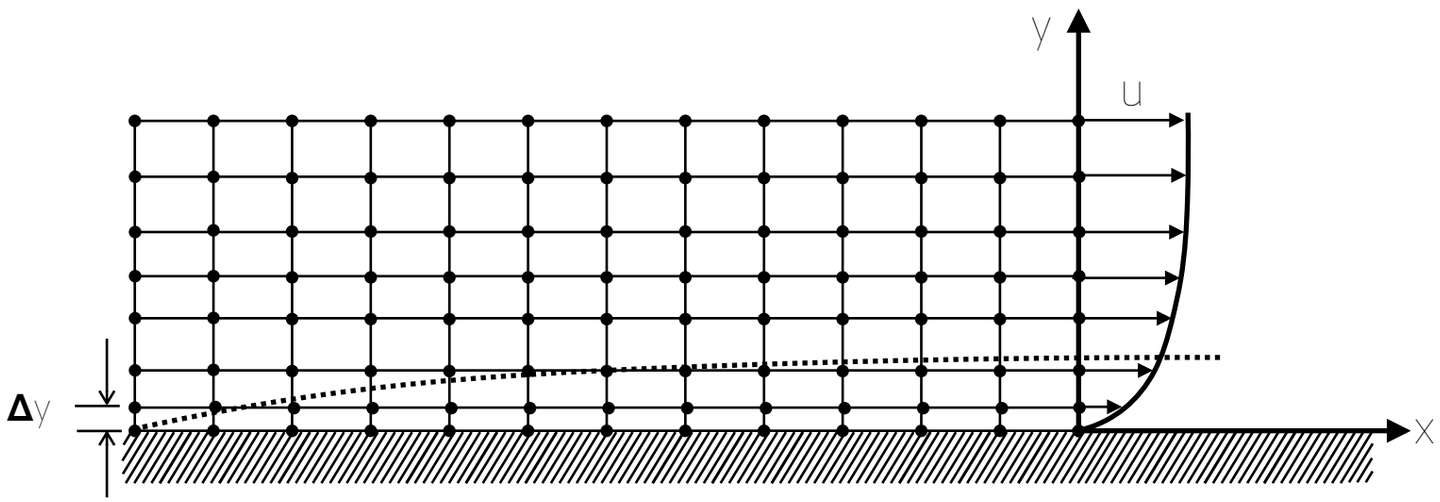
Mesh spacing is fundamental to capture flow features:



Uniform mesh spacing

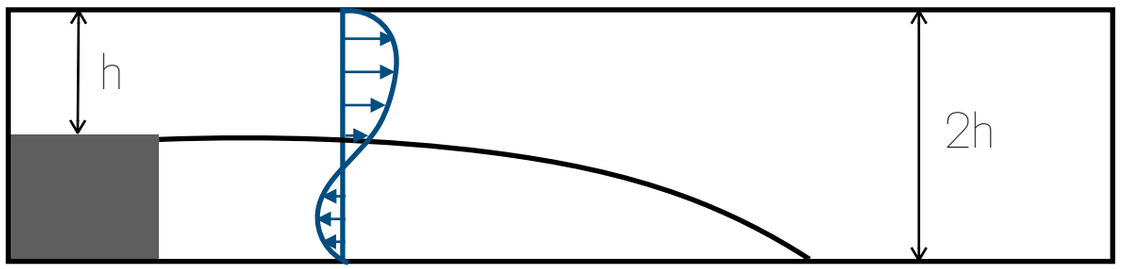


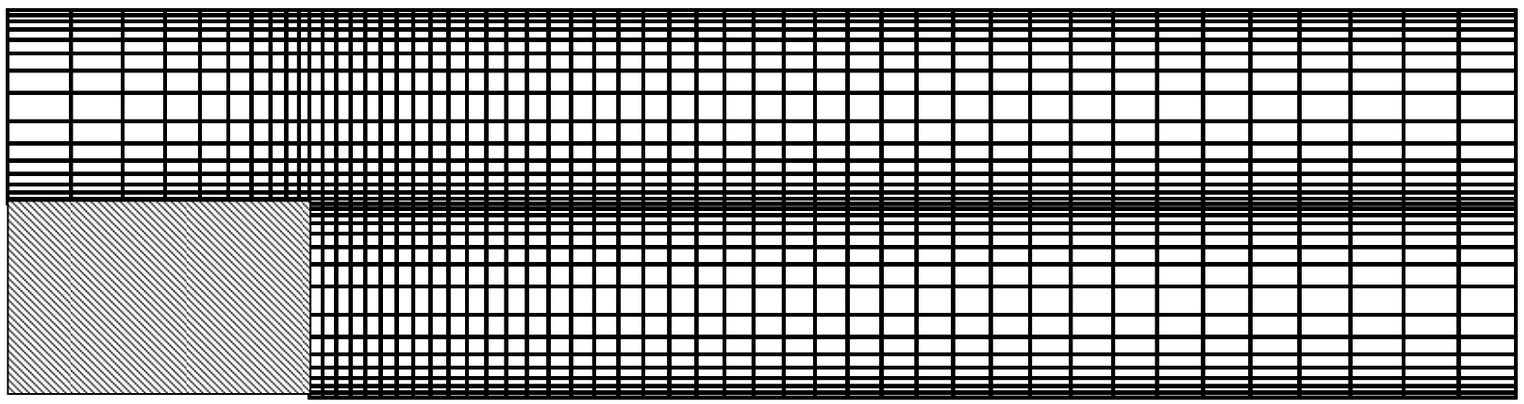
Non-uniform mesh spacing



Mesh design and generation

Importance of mesh spacing

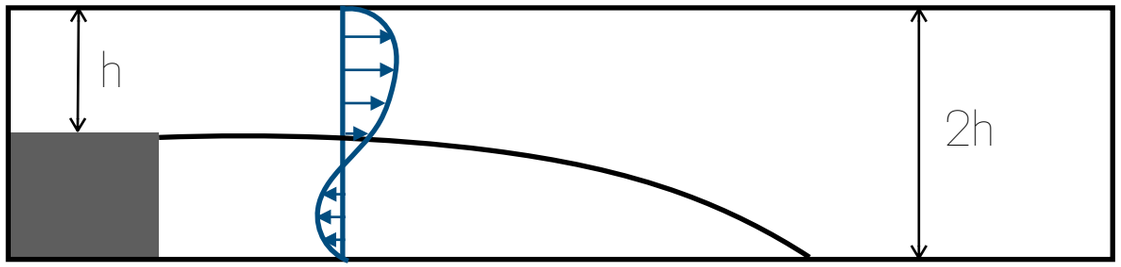


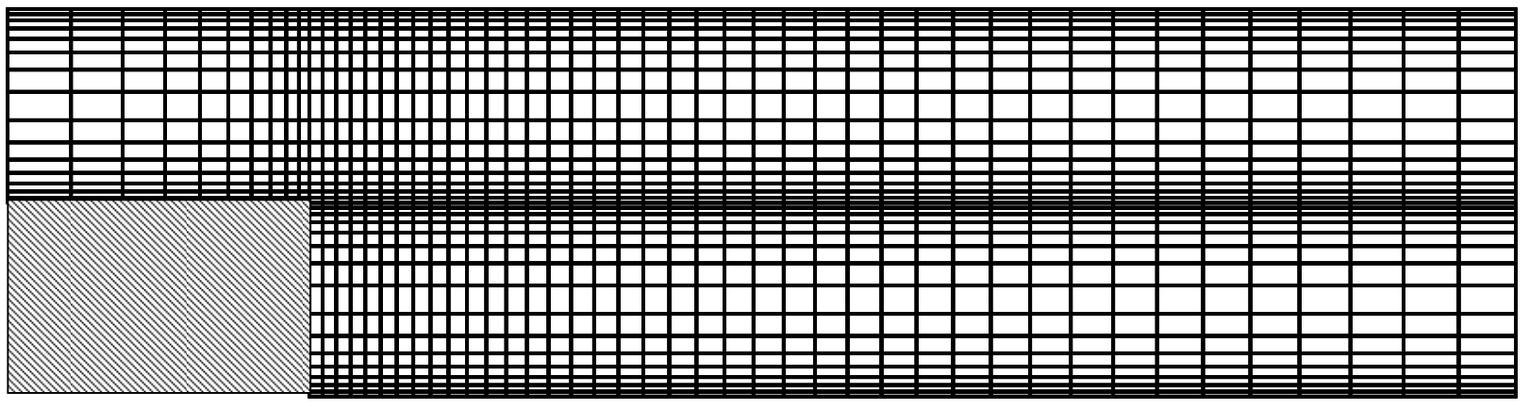


Mesh design and generation

Importance of mesh spacing

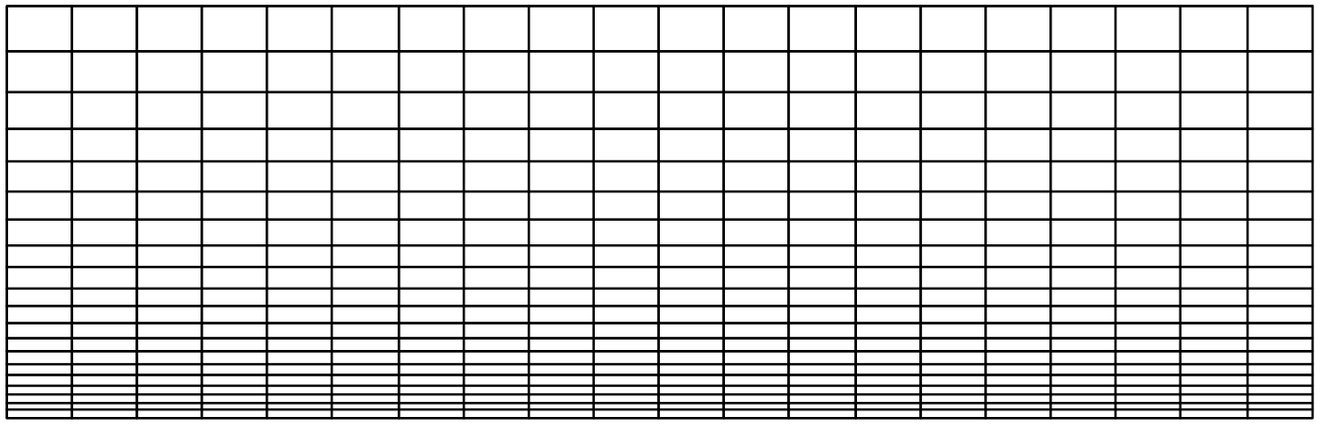
Backwards facing step:

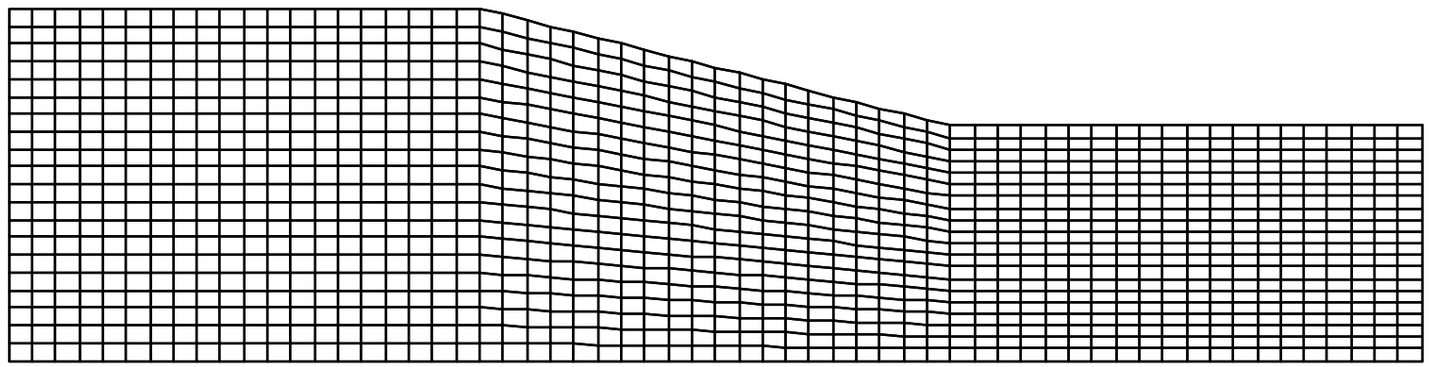




Mesh design and generation

Types of Mesh

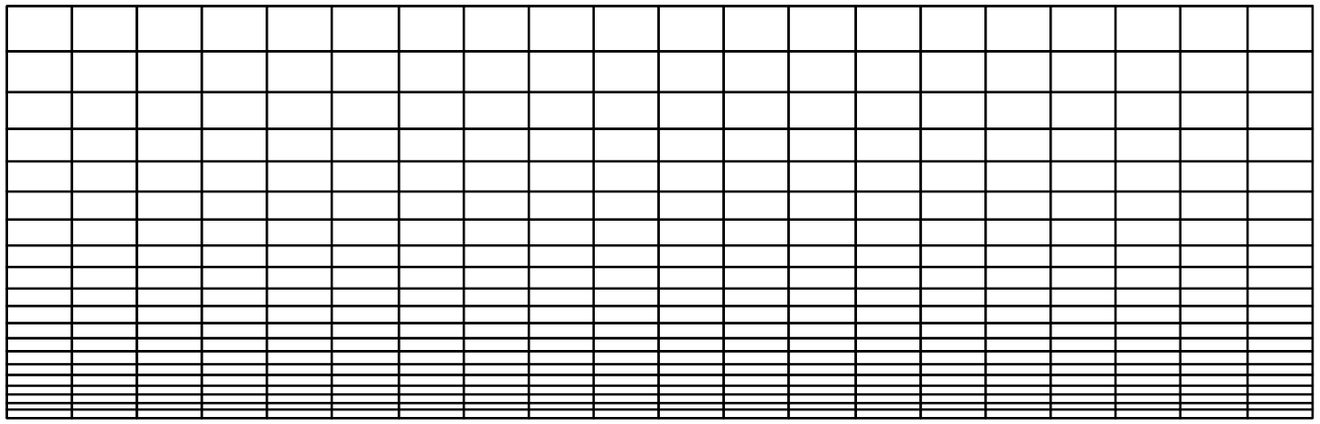


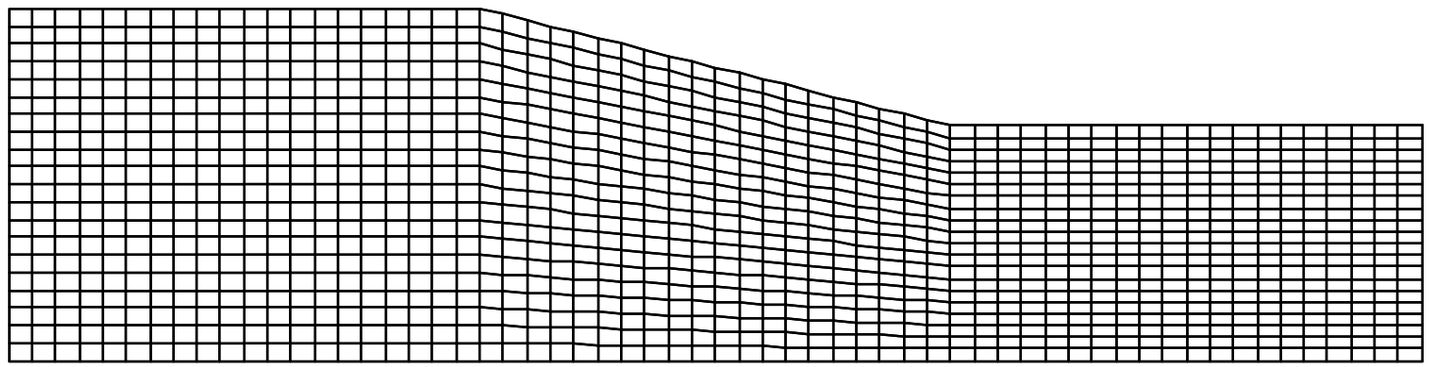


Mesh design and generation

Types of Mesh

Structured Cartesian Mesh

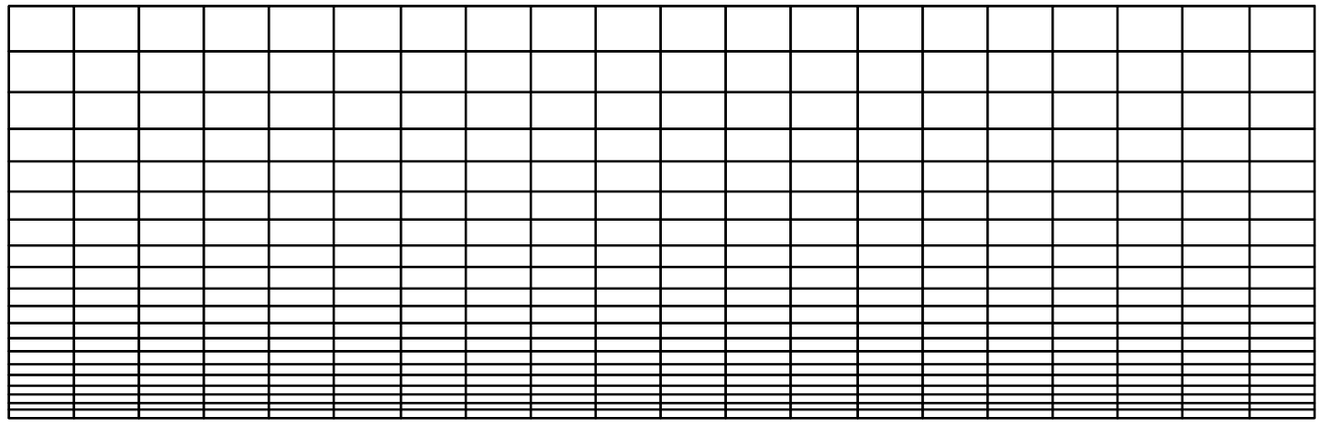




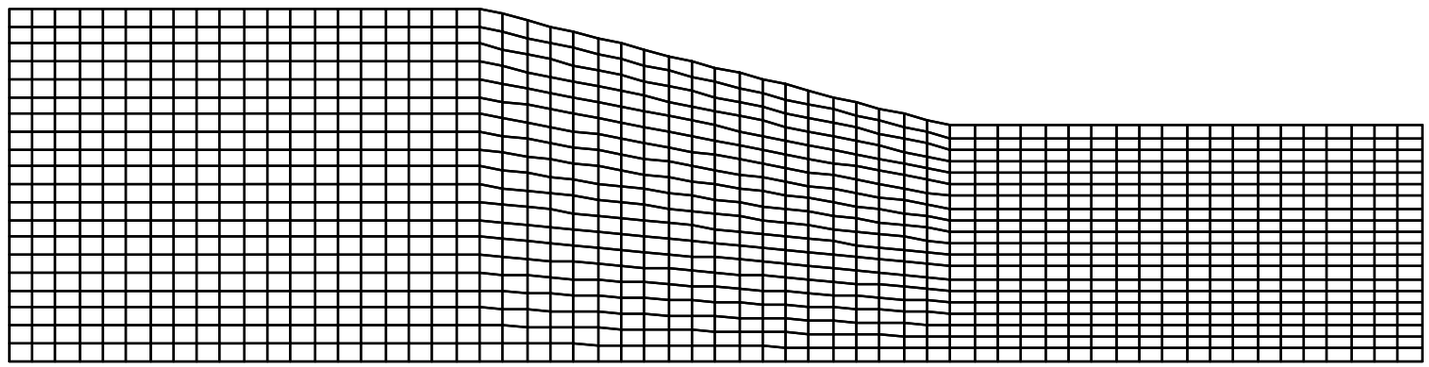
Mesh design and generation

Types of Mesh

Structured Cartesian Mesh

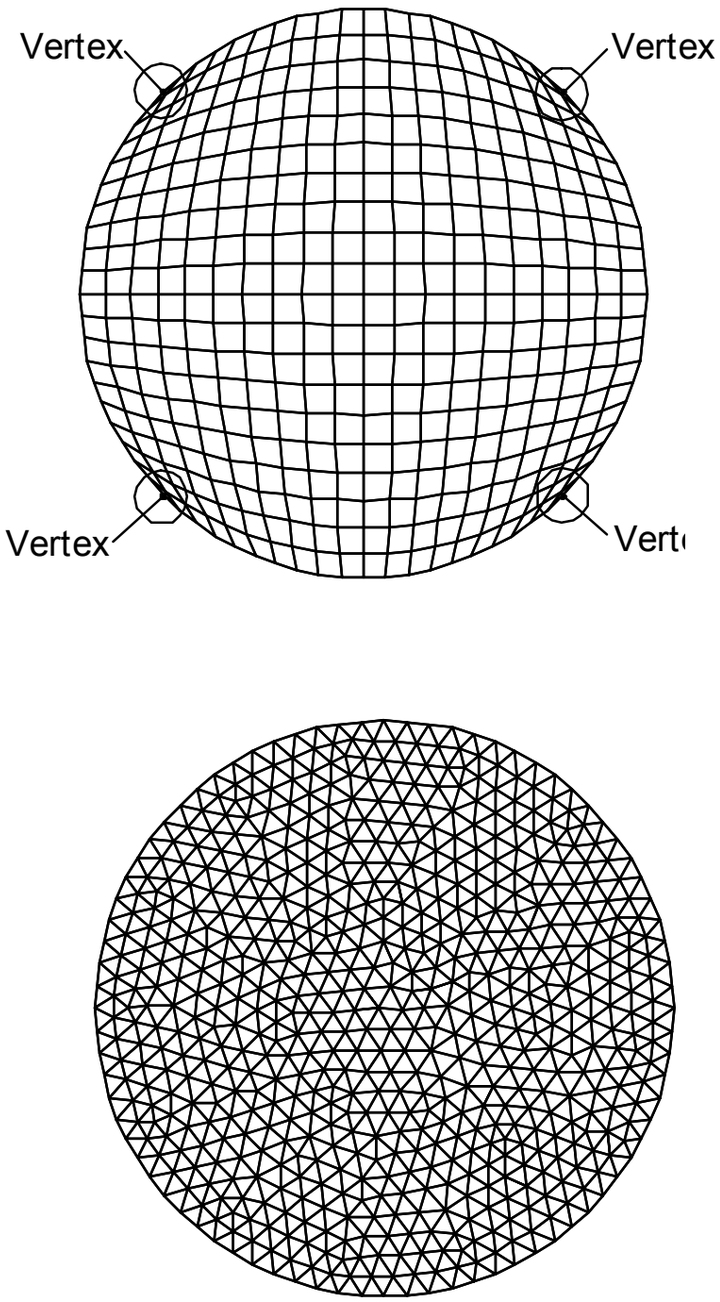


Structured body-fitted Mesh



Mesh design and generation

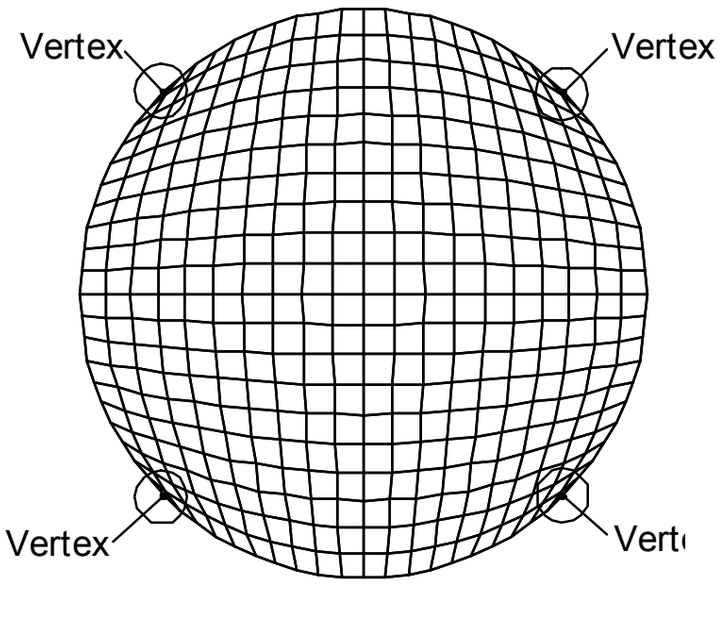
Types of Mesh

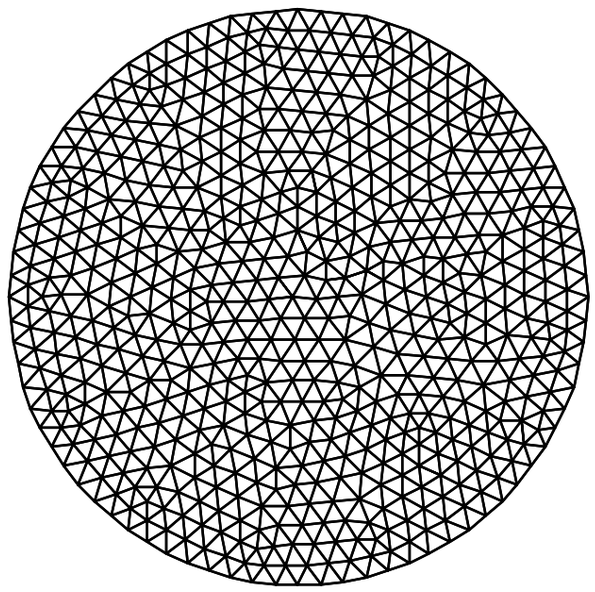


Mesh design and generation

Types of Mesh

Structured Mesh

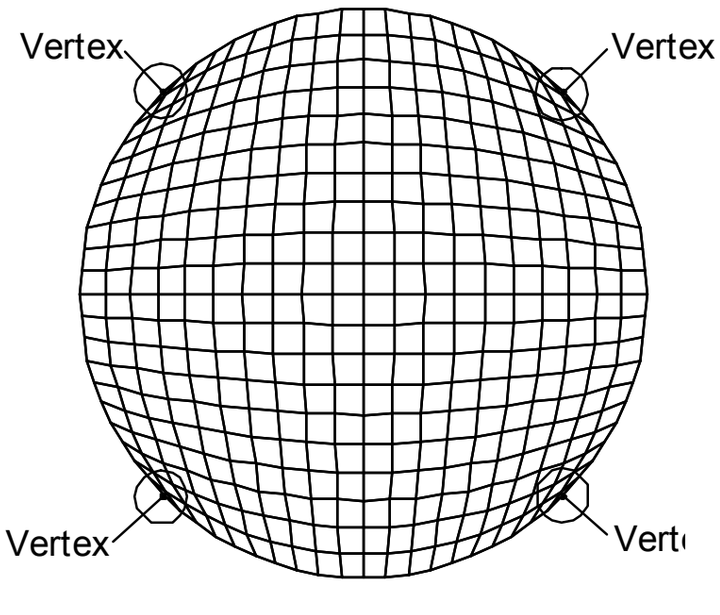




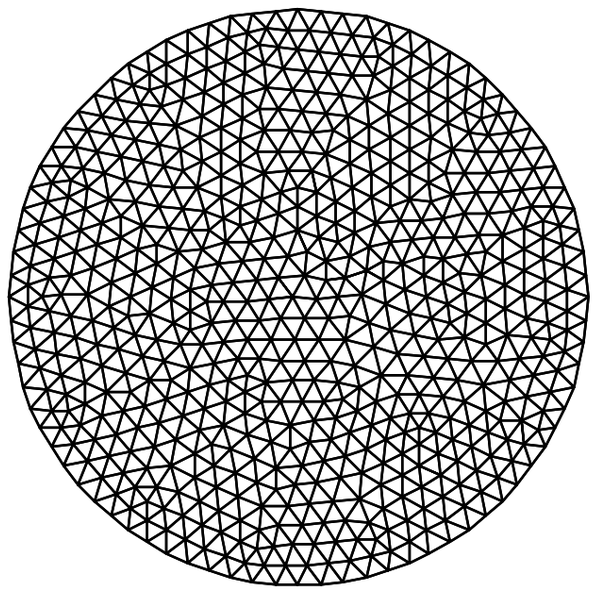
Mesh design and generation

Types of Mesh

Structured Mesh

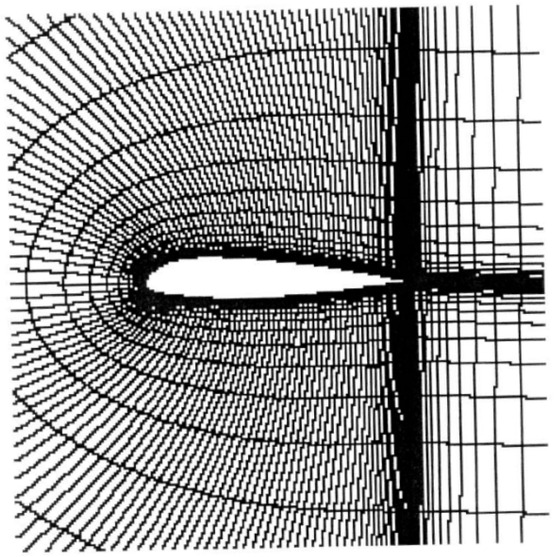


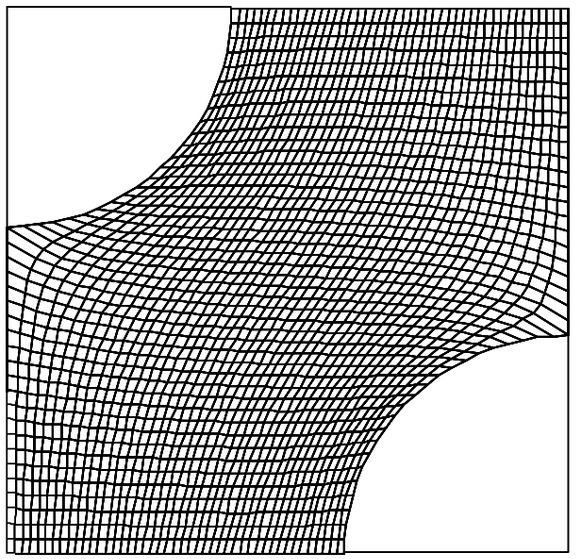
Unstructured Mesh

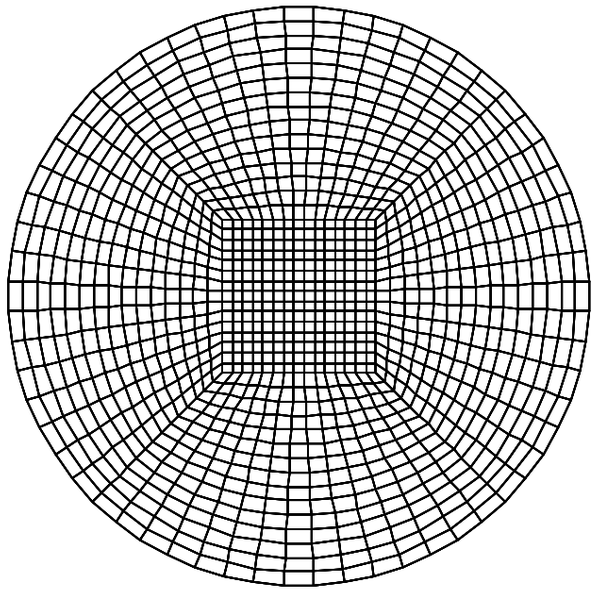


Mesh design and generation

Types of Mesh



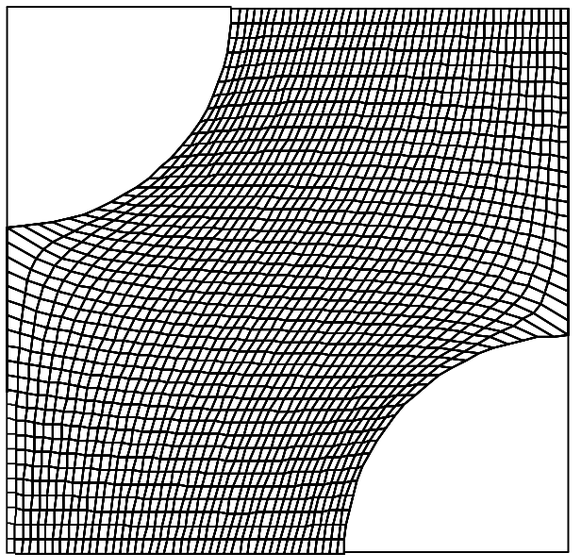


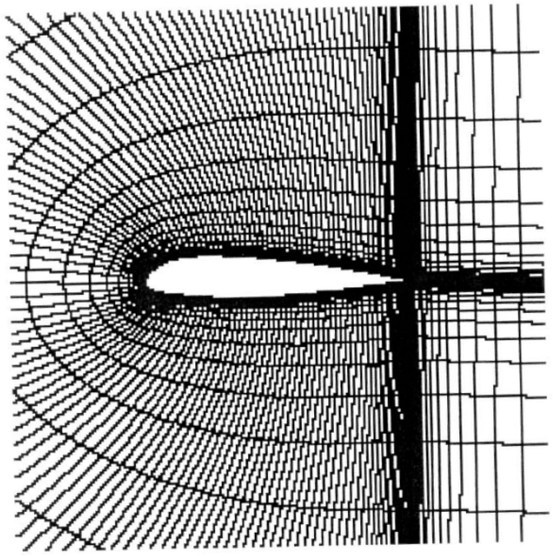


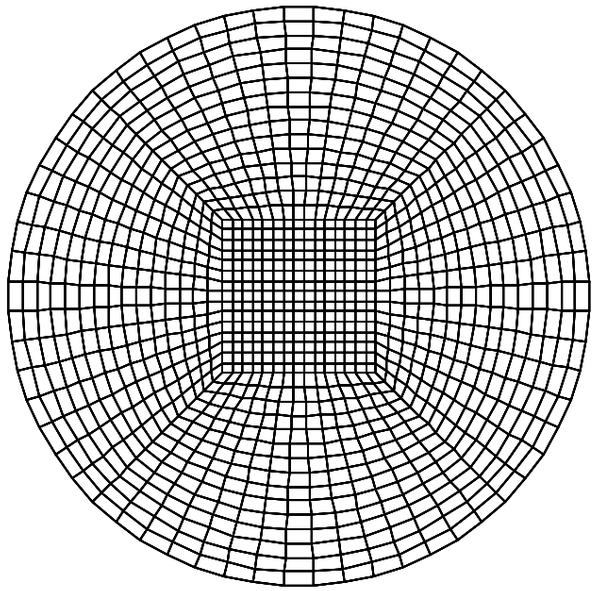
Mesh design and generation

Types of Mesh

C-type Mesh



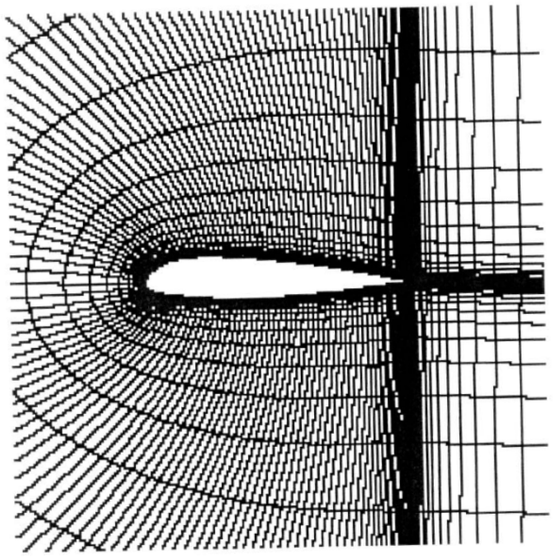




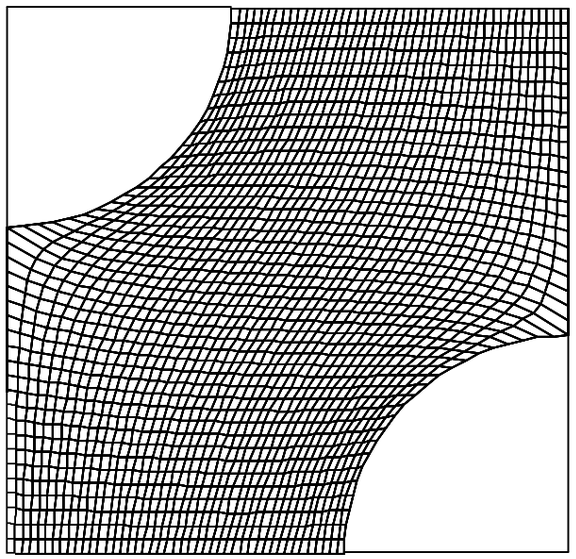
Mesh design and generation

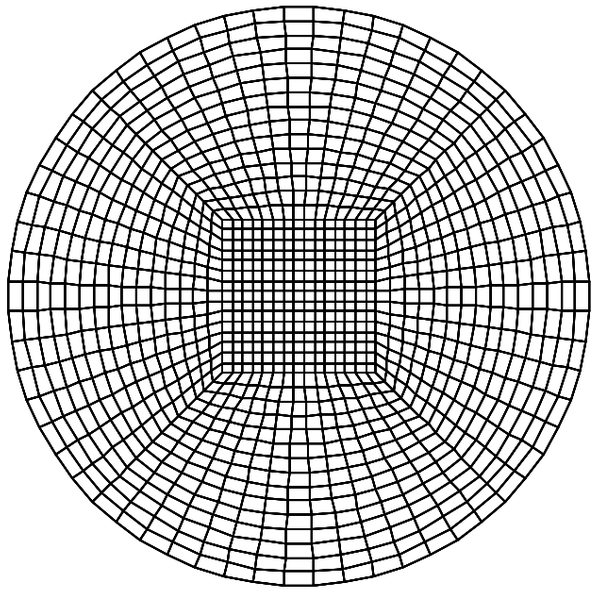
Types of Mesh

C-type Mesh



H-type Mesh

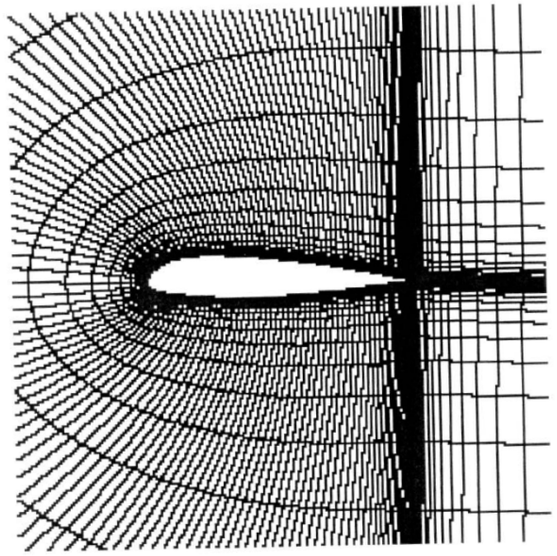




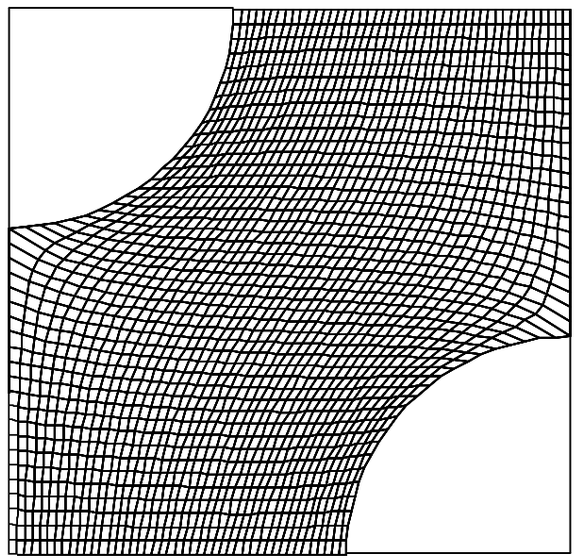
Mesh design and generation

Types of Mesh

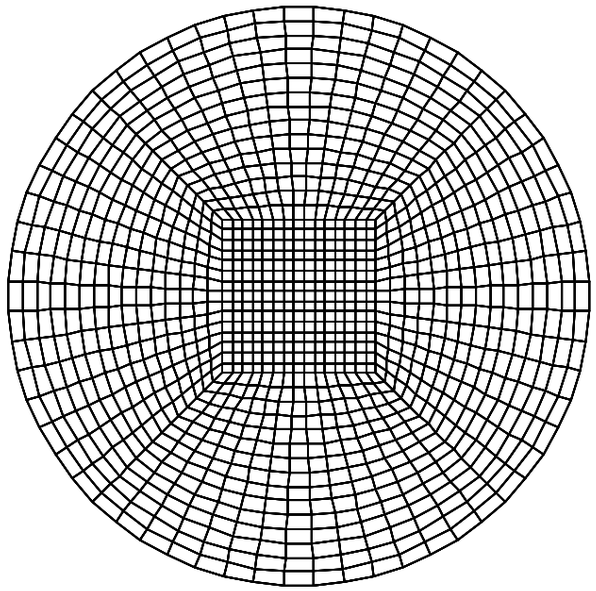
C-type Mesh



H-type Mesh



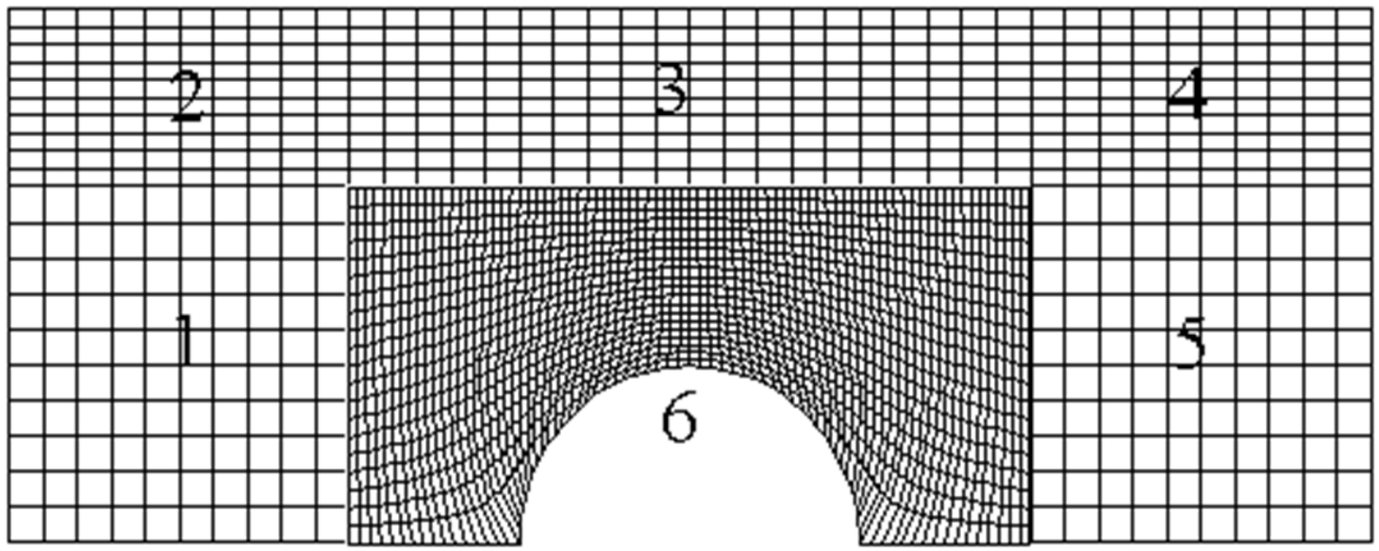
O-type Mesh



Mesh design and generation

Multi-block mesh

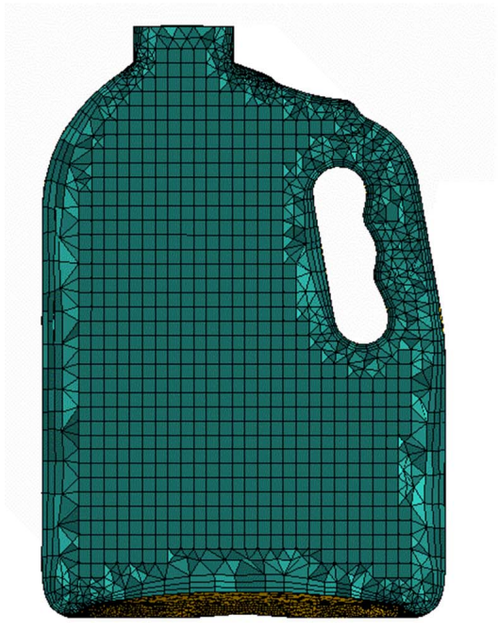
Matching cell faces



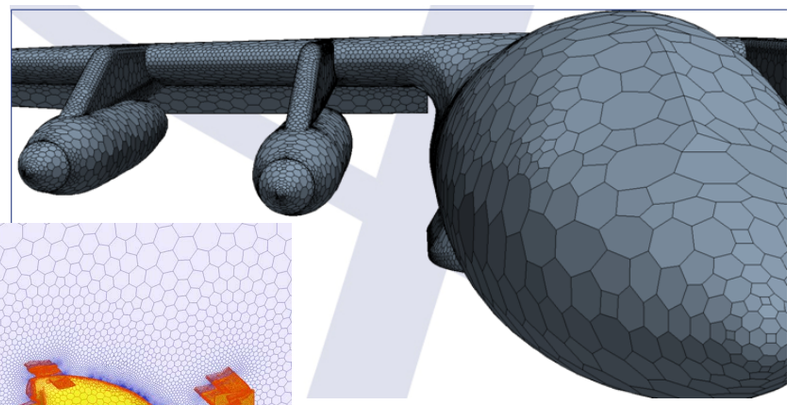
Non-Matching cell faces

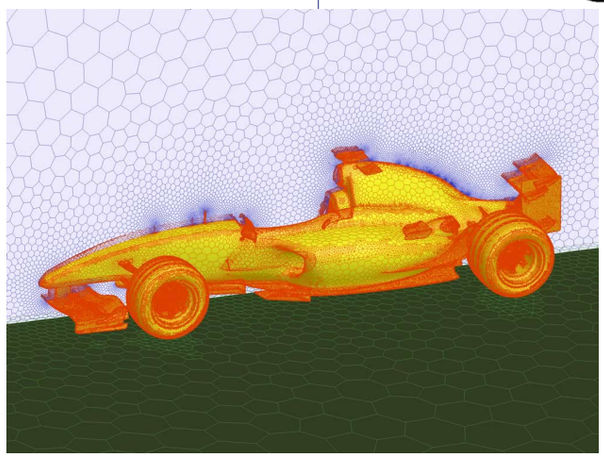
Mesh design and generation

Hybrid (Hexa-Tetra) mesh



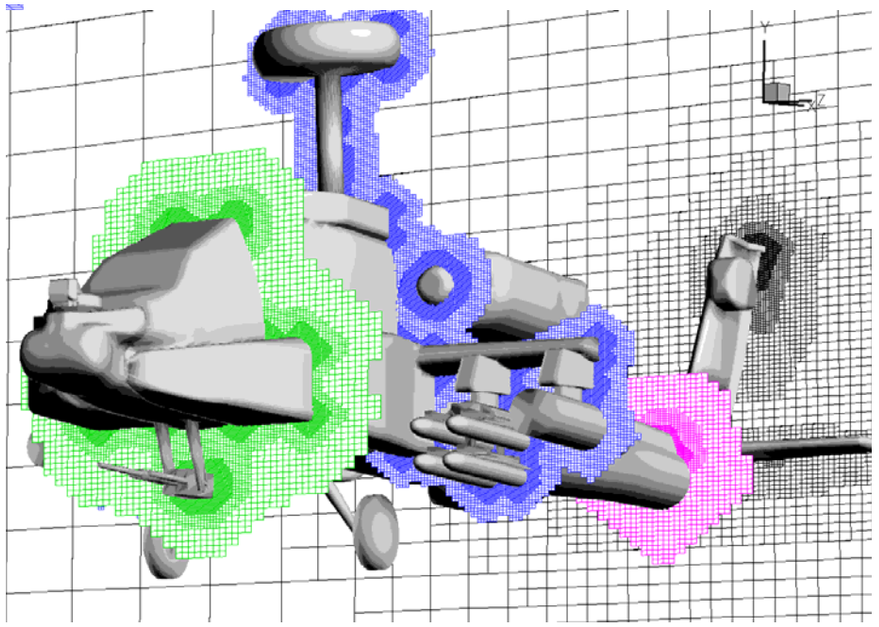
Polyhedral mesh

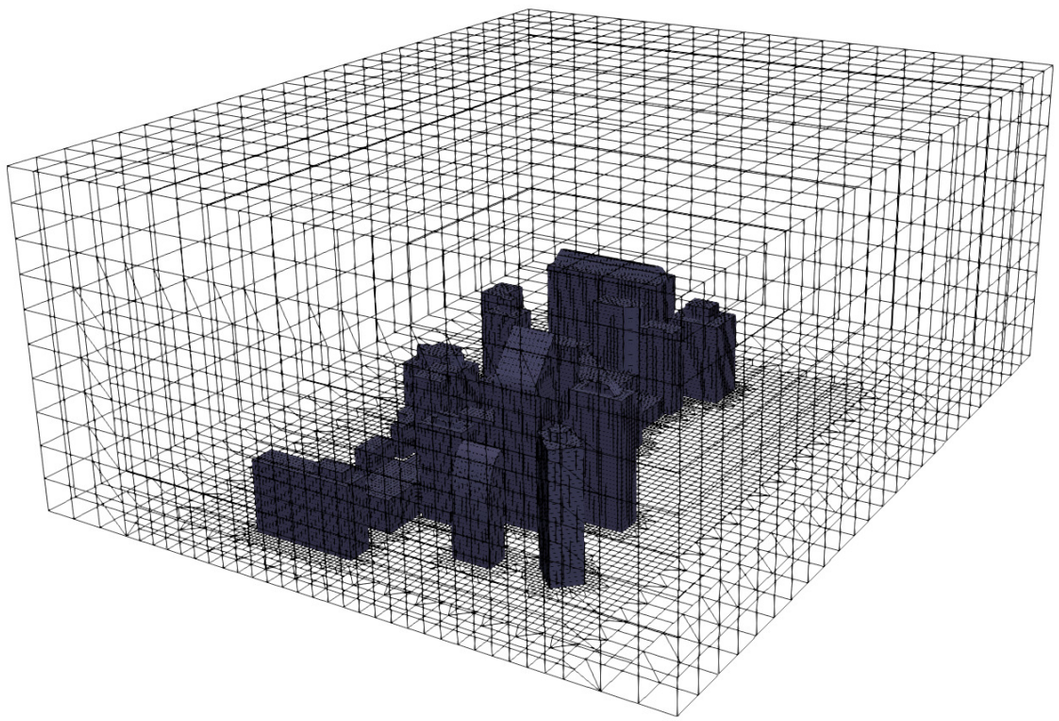




Mesh design and generation

Local Refinement





Questions so far?

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

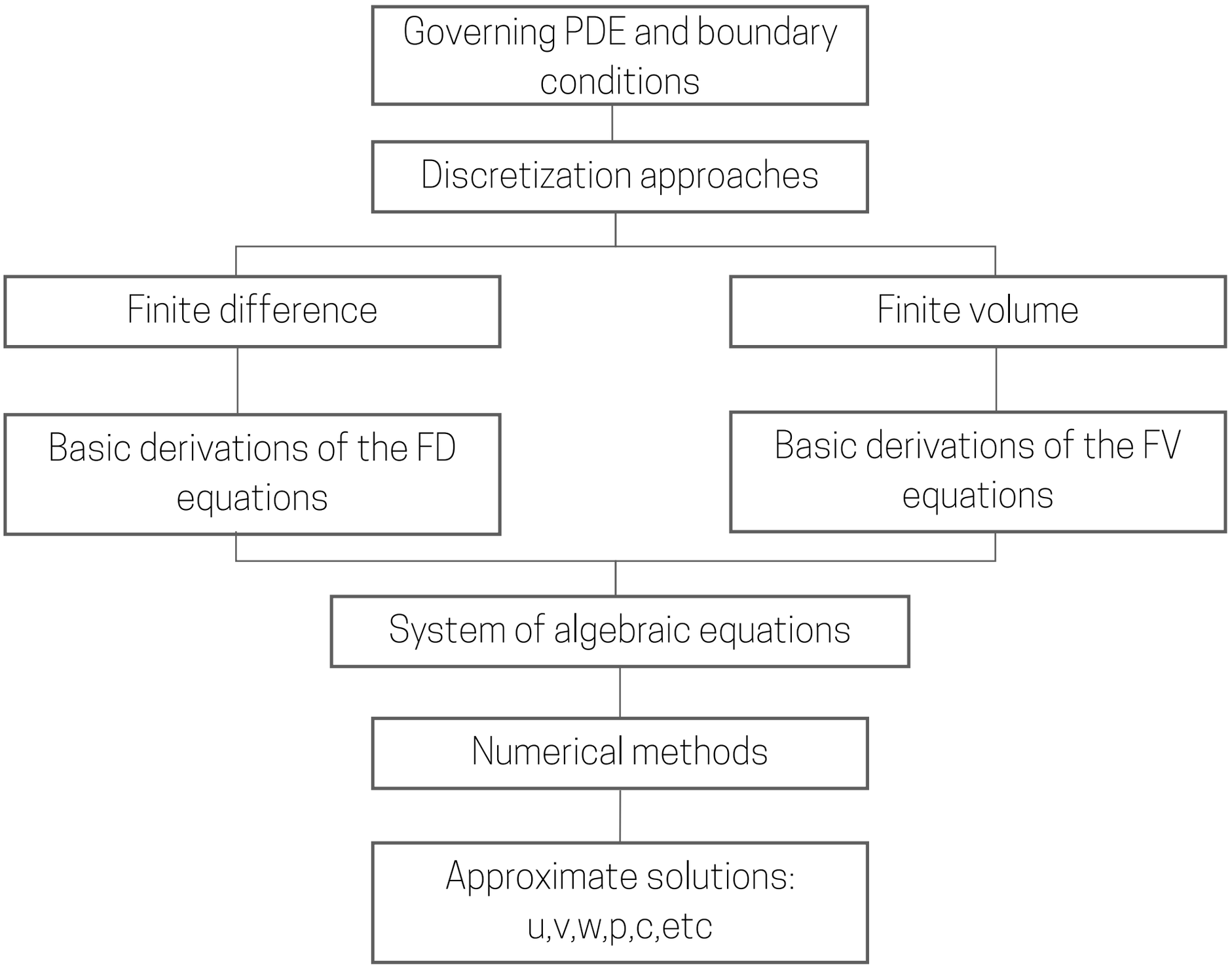
8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Discretization

Overall computing process



Discretization

Finite Differences

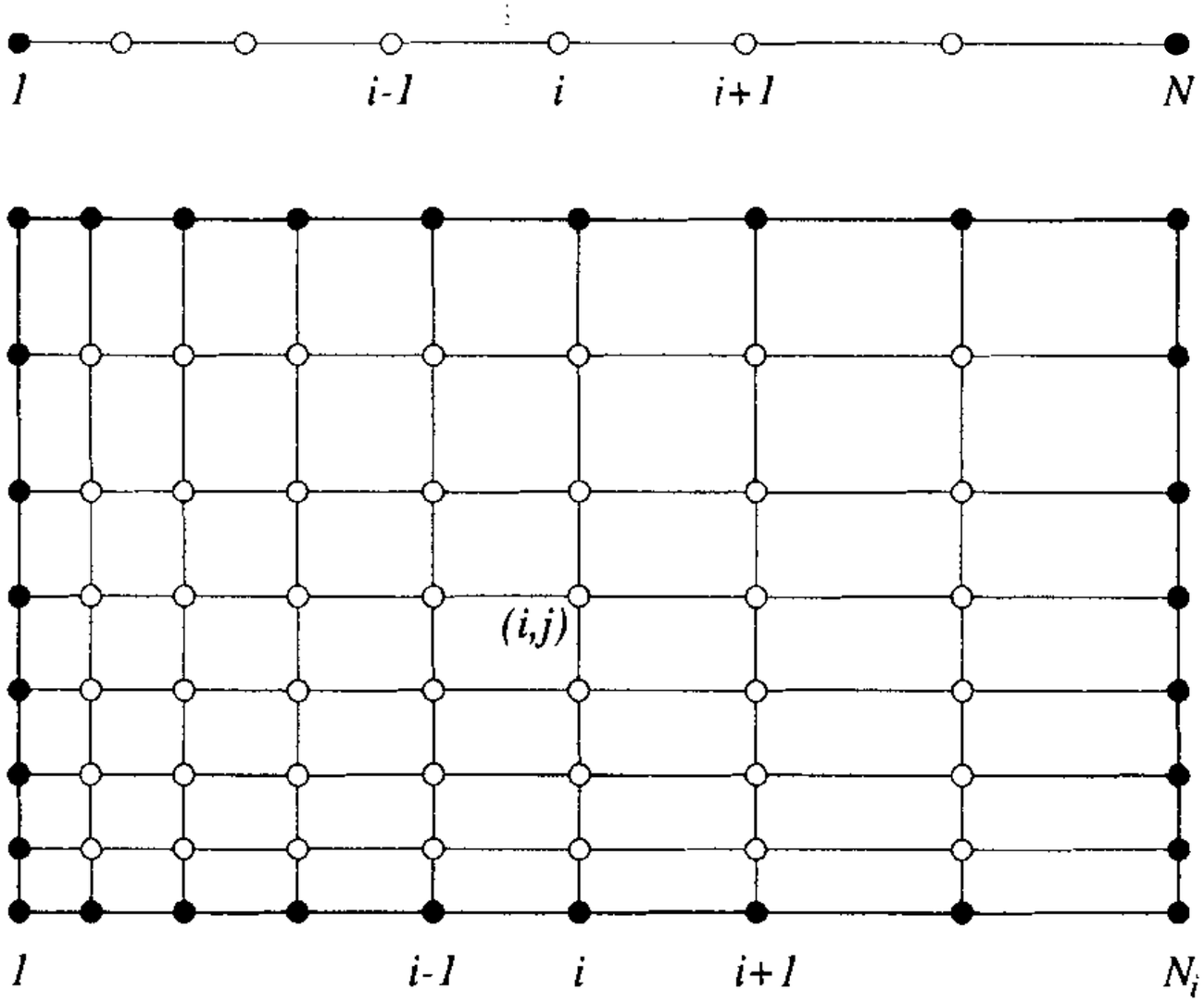
Discretization

Finite Differences

- The region of interest (where all the fluid we want to model is) is transformed in a grid
- The solution of the original equations is only computed at specific locations of the grid (normally center/faces)
- Commonly used for regular meshes
- Uses the differential form of the equations
- At each nodal point, Taylor series expansions are used to generate finite difference approximations to the partial derivatives of the governing equations
- Conservation property is not enforced

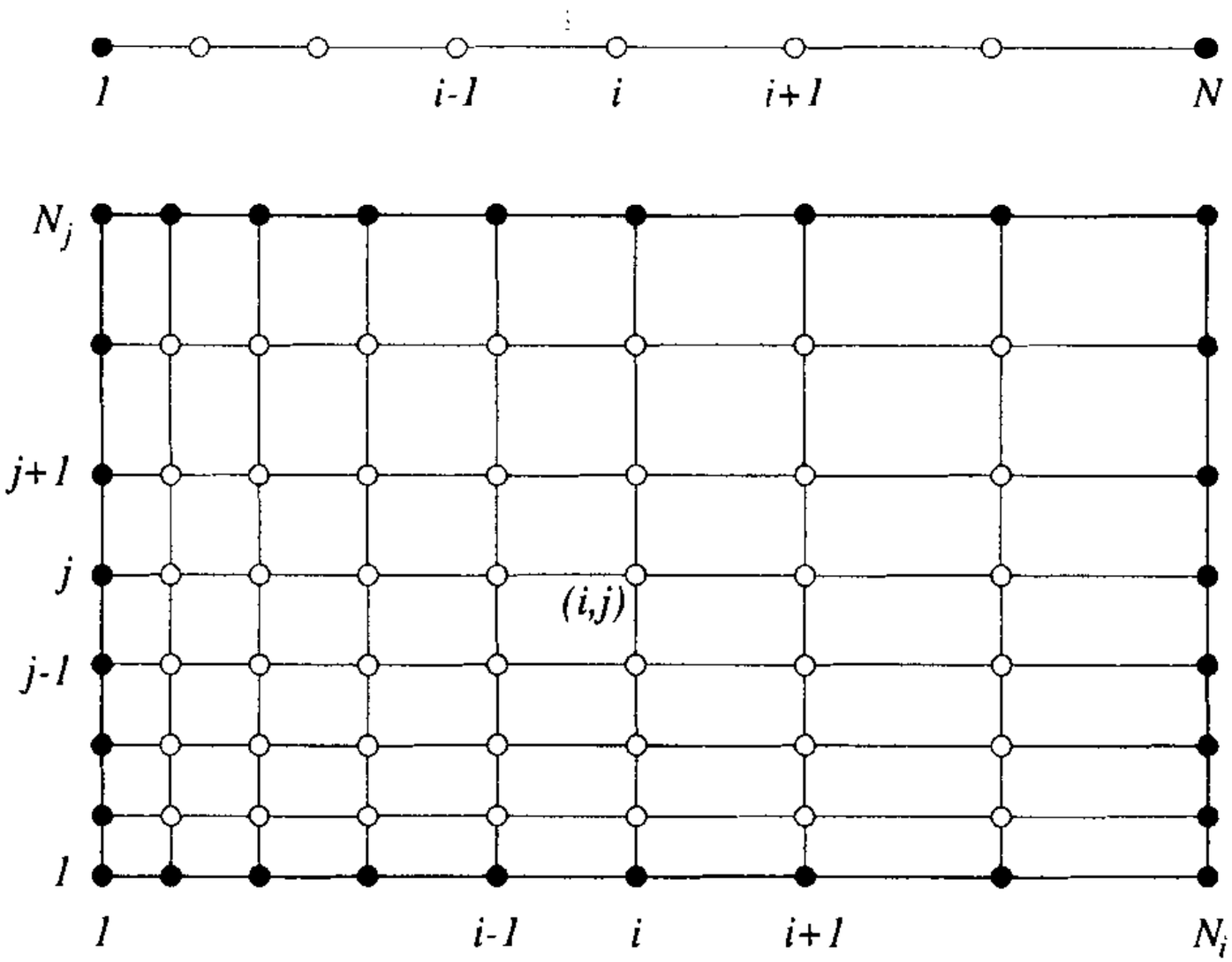
Discretization

Finite Differences



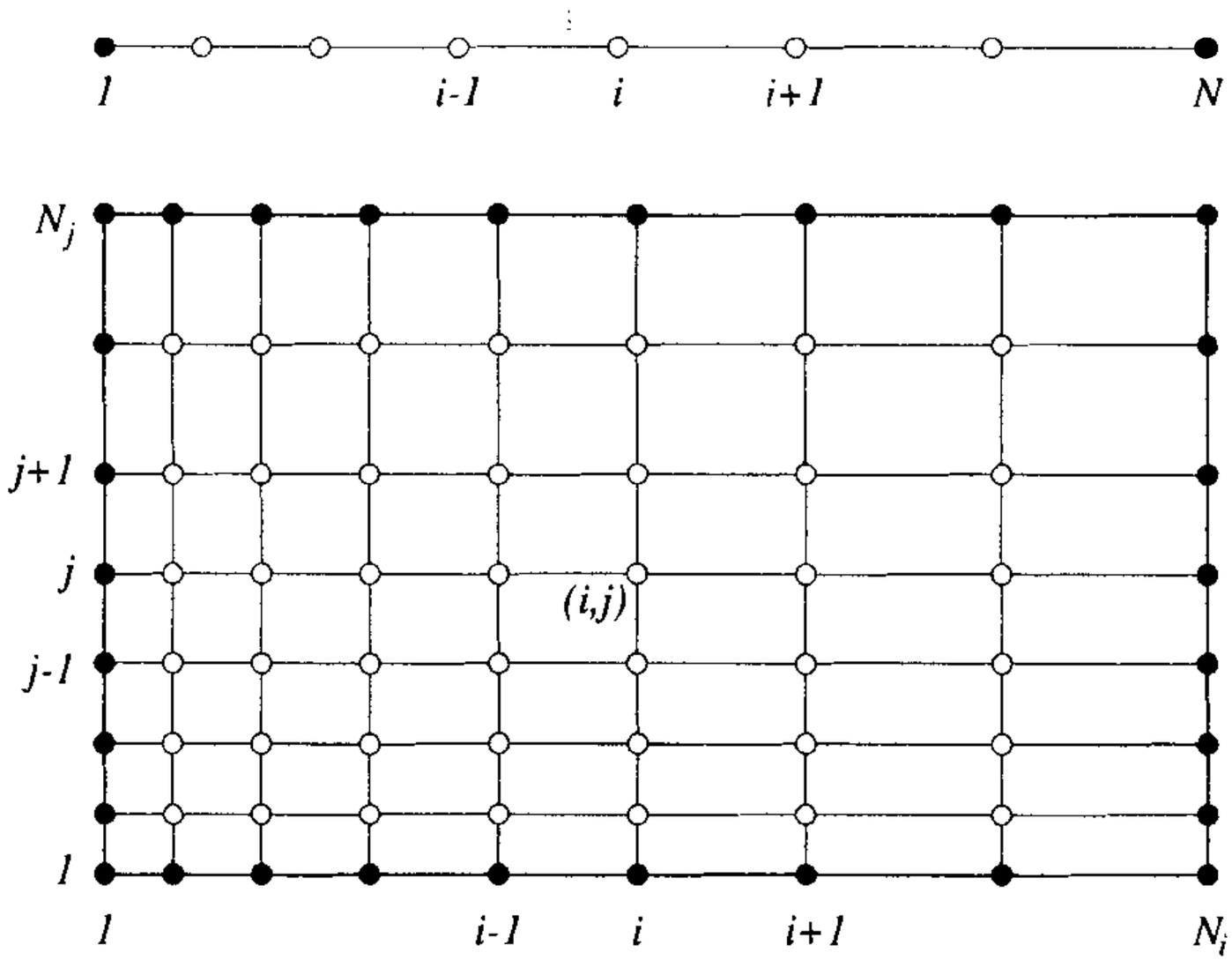
Discretization

Finite Differences



Discretization

Finite Differences

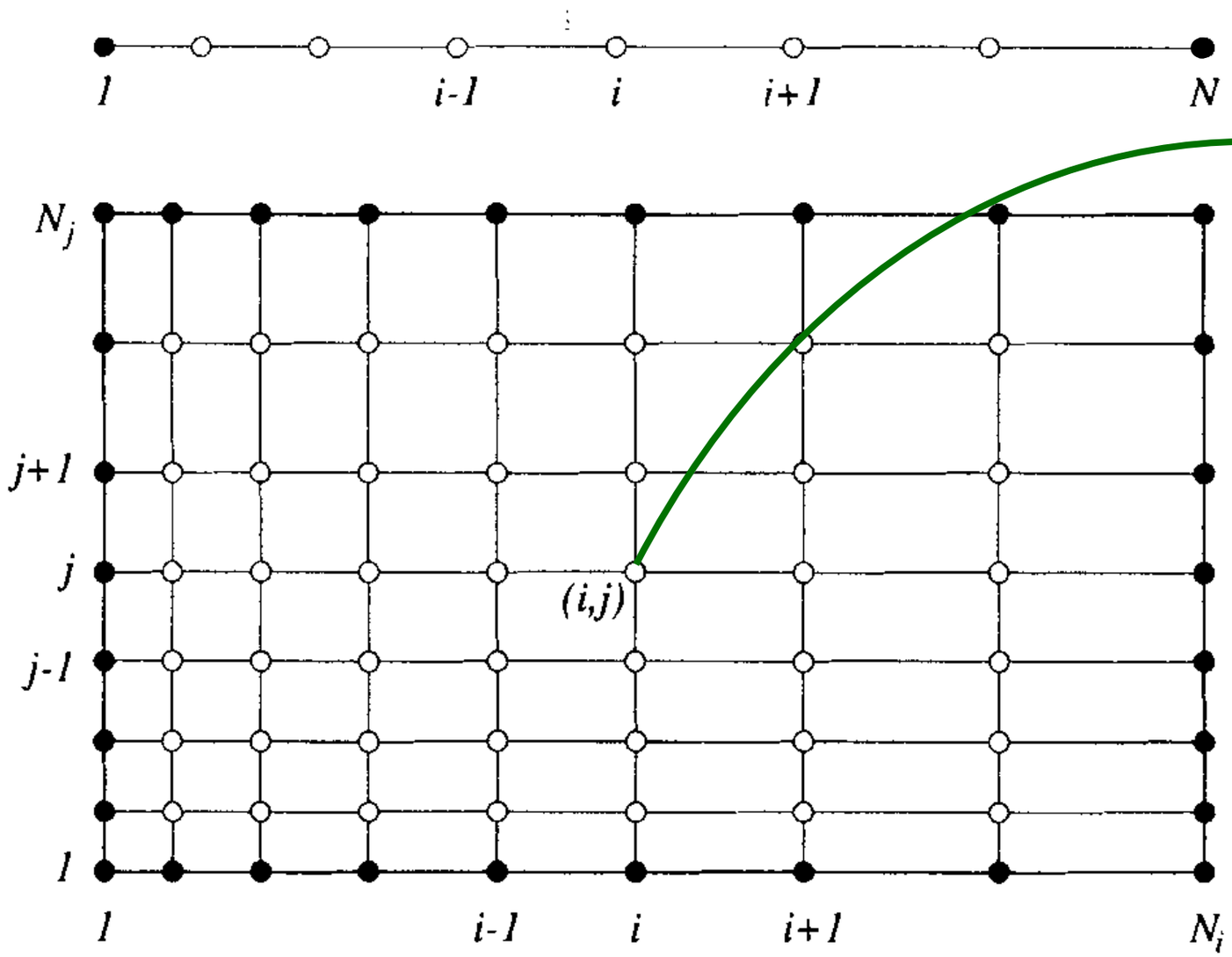


Simplest case since it is uniform spacing

If you are resolving in time, you will also discretise with respect to time

Discretization

Finite Differences



An unknown value of a field variable that depends on the neighbouring nodes, providing and algebraic equation

Simplest case since it is uniform spacing

If you are resolving in time, you will also discretise with respect to time

Discretization

$$\frac{\partial \phi(x)}{\partial x} = \lim_{\Delta x \to 0} \frac{\phi(x + \Delta x) - \phi(x)}{\Delta x}$$

Discretization

- The continuous derivatives are transformed in discrete algebraic relationships between the grid point values (Finite Difference Methods)

$$\frac{\partial \phi(x)}{\partial x} = \lim_{\Delta x \to 0} \frac{\phi(x + \Delta x) - \phi(x)}{\Delta x}$$

Discretization

- The continuous derivatives are transformed in discrete algebraic relationships between the grid point values (Finite Difference Methods)

- Using the derivative definition:

$$\frac{\partial \phi(x)}{\partial x} = \lim_{\Delta x \to 0} \frac{\phi(x + \Delta x) - \phi(x)}{\Delta x}$$

Discretization

- The continuous derivatives are transformed in discrete algebraic relationships between the grid point values (Finite Difference Methods)

- Using the derivative definition:

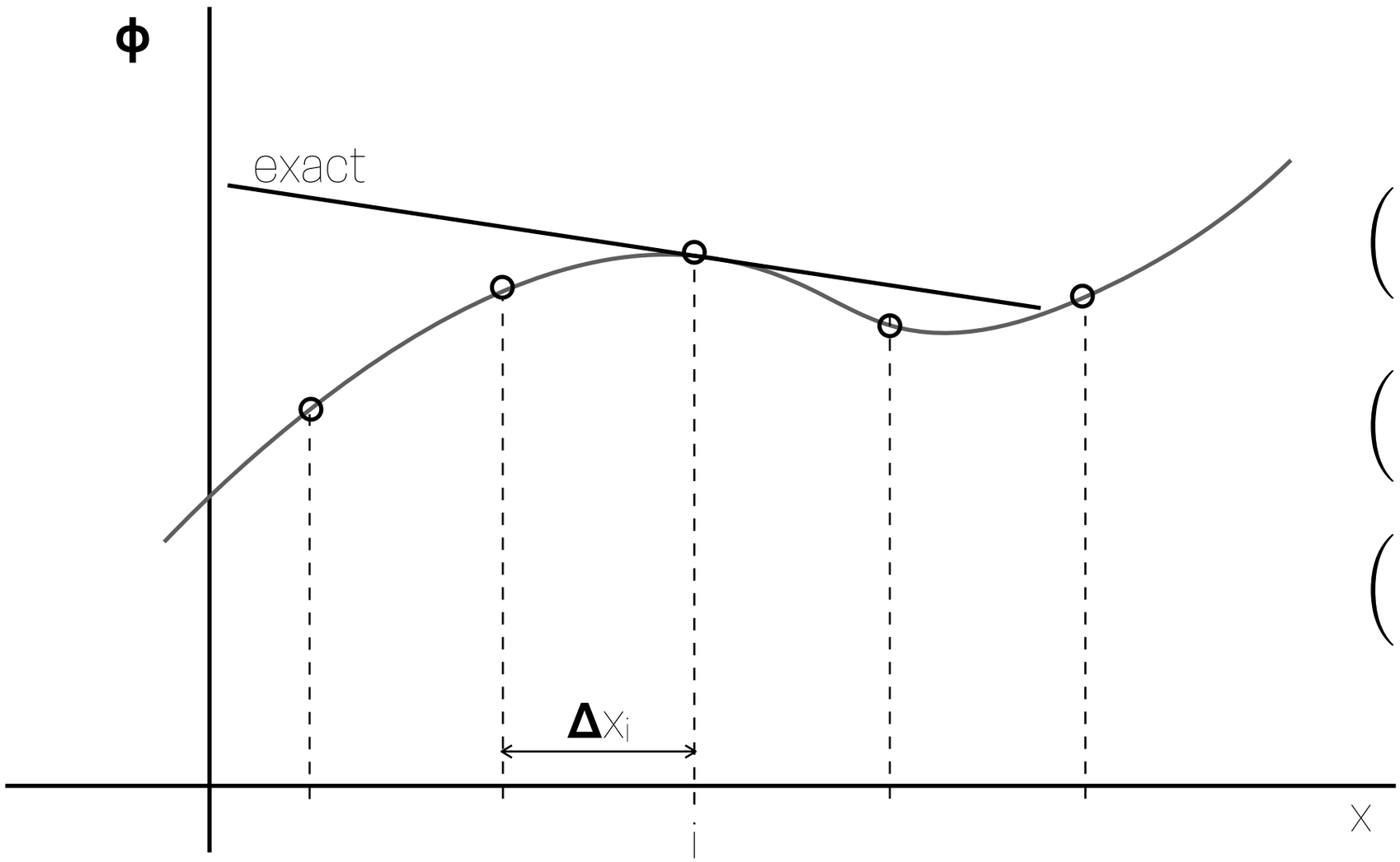
$$\frac{\partial \phi(x)}{\partial x} = \lim_{\Delta x \to 0} \frac{\phi(x + \Delta x) - \phi(x)}{\Delta x}$$

- But without evaluating the limit:

$$\frac{\partial \phi(x)}{\partial x} \approx \frac{\phi(x + \Delta x) - \phi(x)}{\Delta x} = \frac{\phi_{i+1} - \phi_i}{\Delta x}$$

Discretization errors

Discretization



$$\left(\frac{\partial \phi}{\partial x}\right)_i \approx$$

$$\left(\frac{\partial \phi}{\partial x}\right)_i \approx$$

$$\left(\frac{\partial \phi}{\partial x}\right)_i \approx$$

The choice influences the accuracy of the approximation

Discretization

The choice influences the accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i)\frac{\partial \phi}{\partial x}|_i + \frac{(x - x_i)^2}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + \ldots + \frac{(x - x_i)^n}{n!}\frac{\partial^n \phi}{\partial x^n}|_i$$

Discretization

The choice influences the accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i)\frac{\partial \phi}{\partial x}|_i + \frac{(x - x_i)^2}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + \ldots + \frac{(x - x_i)^n}{n!}\frac{\partial^n \phi}{\partial x^n}|_i$$

Using a **forward** discretisation:

$$\frac{\partial \phi}{\partial x}|_i =$$

Discretization

The choice influences the accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i)\frac{\partial \phi}{\partial x}|_i + \frac{(x - x_i)^2}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + \ldots + \frac{(x - x_i)^n}{n!}\frac{\partial^n \phi}{\partial x^n}|_i$$

Using a **forward** discretisation:

$$\frac{\partial \phi}{\partial x}|_i =$$

Using a **central** discretisation:

$$\frac{\partial \phi}{\partial x}|_i =$$

Discretization

Finite Volume

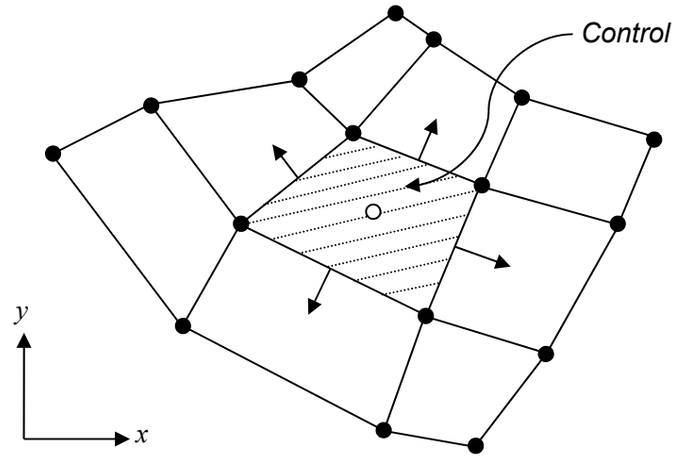
Discretization

Finite Volume

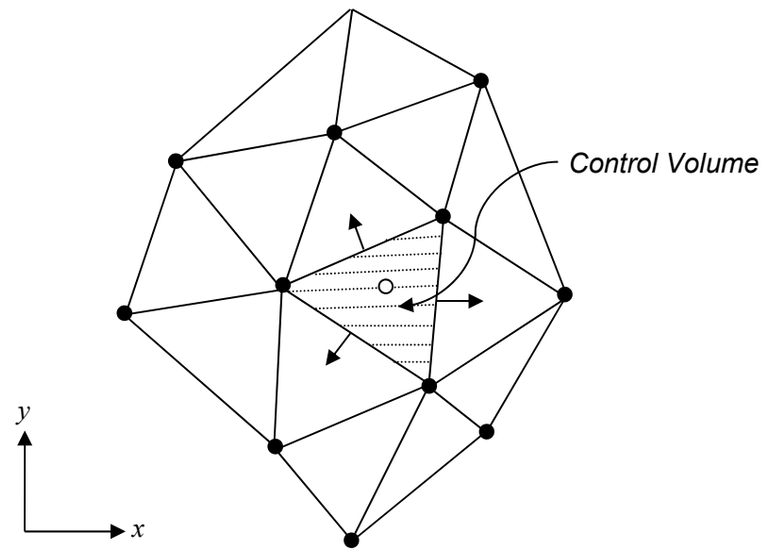
- The region of interest (where all the fluid we want to model is) is transformed in a set of contiguous control volumes
- The variables are computed at the centroids of each control volume
- Normally used for complex meshes
- Uses the integral form of the equations
- The grid defines the control volumes boundaries, the method is conservative so long as the surface integrals that area applied at these boundaries are the same as the control values sharing the boundary.

Discretization

Finite Volume



Structured Grid



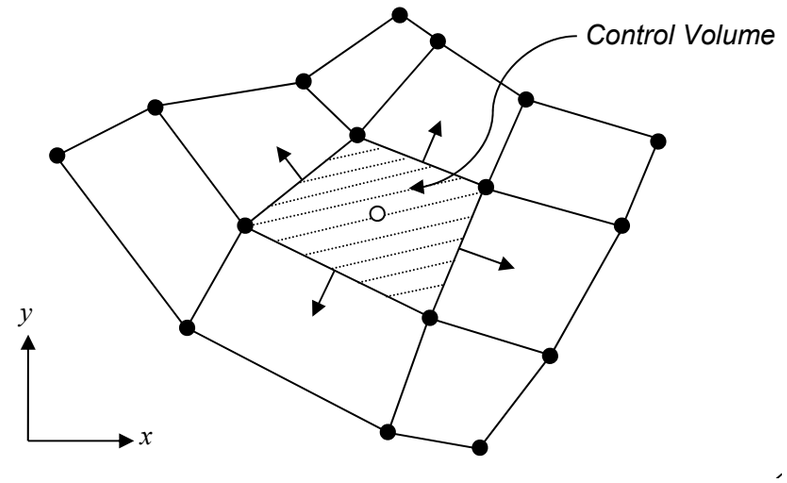
Unstructured Grid

Discretization

Finite Volume

By using Gauss's divergence theorem in the x direction:

$$\int\int\int_V (\nabla F) dV = \int\int_A (F\vec{n}) dA$$



$$\frac{\partial \phi}{\partial x} = \frac{1}{\Delta V}\int_V \frac{\partial \phi}{\partial x} dV =$$

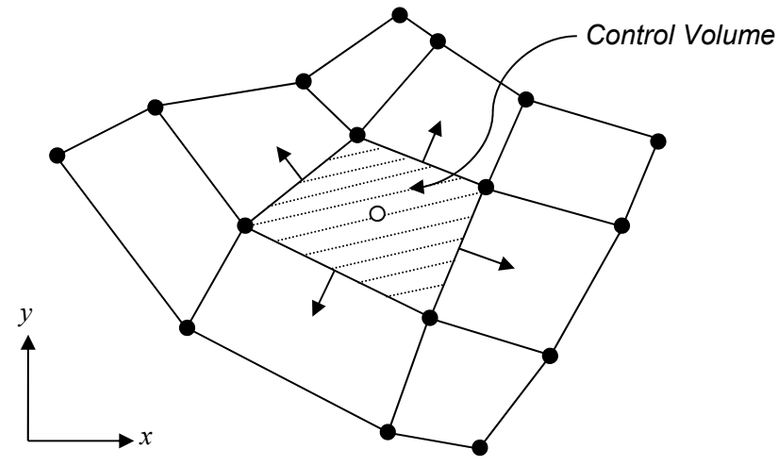
$$\frac{\partial \phi}{\partial y} = \frac{1}{\Delta V}\int_V \frac{\partial \phi}{\partial y} dV =$$

Discretization

Finite Volume

By using Gauss's divergence theorem in the x direction:

$$\int\int\int_V (\nabla F) dV = \int\int_A (F\vec{n}) dA$$



$$\frac{\partial \phi}{\partial x} = \frac{1}{\Delta V}\int_V \frac{\partial \phi}{\partial x} dV = \frac{1}{\Delta V}\int_A \phi dA^x \approx \frac{1}{\Delta V}\sum_{i=1}^{N} \phi A_i^x$$

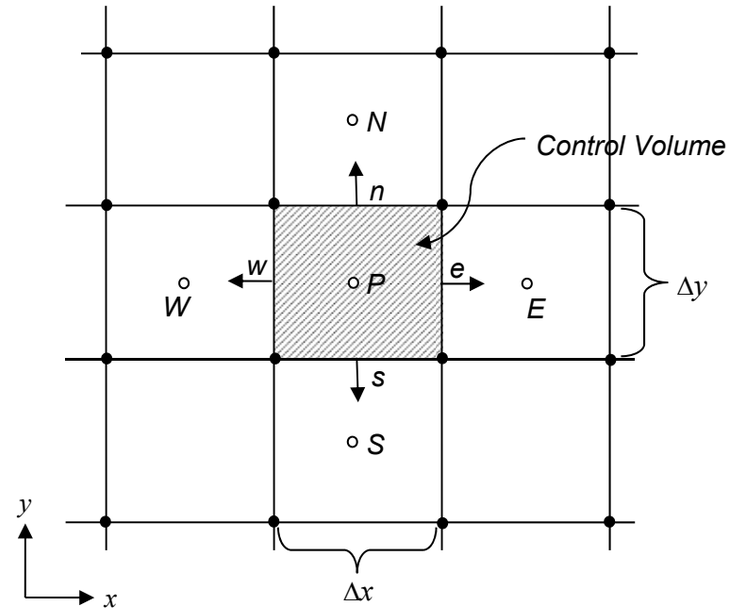
$$\frac{\partial \phi}{\partial y} = \frac{1}{\Delta V}\int_V \frac{\partial \phi}{\partial y} dV = \frac{1}{\Delta V}\int_A \phi dA^y \approx \frac{1}{\Delta V}\sum_{i=1}^{N} \phi A_i^y$$

Discretization

Finite Volume

Determine the discretised form of the 2D continuity equation using the FV method in a structured uniform grid arrangement:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

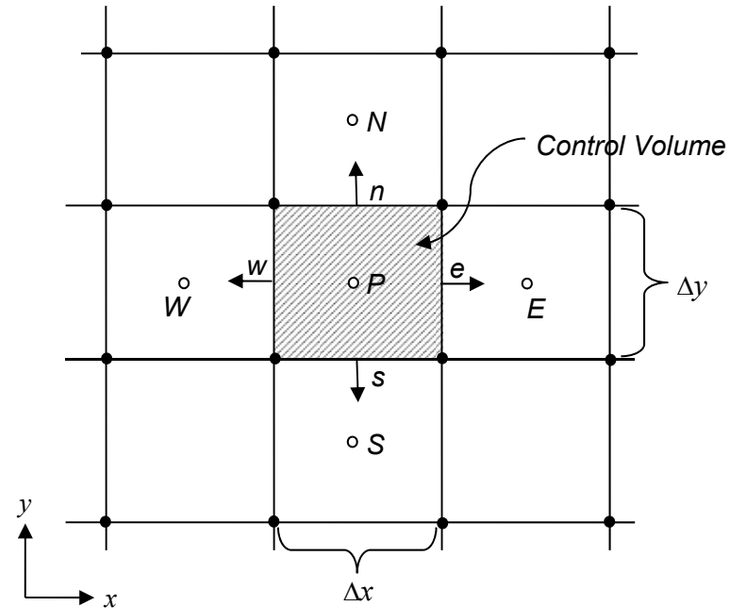


Discretization

Finite Volume

Determine the discretised form of the 2D continuity equation using the FV method in a structured uniform grid arrangement:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$



$$\frac{\partial u}{\partial x} = \frac{1}{\Delta V}\int_A u dA^x \approx \frac{1}{\Delta V}\sum_{i=1}^{N} u_i A_i^x = \frac{1}{\Delta V}(u_e A_e^x - u_w A_w^x + \cancelto{0}{v_n A_n^x} - \cancelto{0}{v_s A_s^x}) = \frac{1}{\Delta V}(u_e A_e^x - u_w A_w^x)$$

$$\frac{\partial v}{\partial y} = \frac{1}{\Delta V}\int_A v dA^y \approx \frac{1}{\Delta V}\sum_{i=1}^{N} v_i A_i^y = \frac{1}{\Delta V}(\cancelto{0}{v_e A_e^y} - \cancelto{0}{v_w A_w^y} + v_n A_n^y - v_s A_s^y) = \frac{1}{\Delta V}(v_n A_n^y - v_s A_s^y)$$

Questions so far?

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Algebraic form of equations

Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

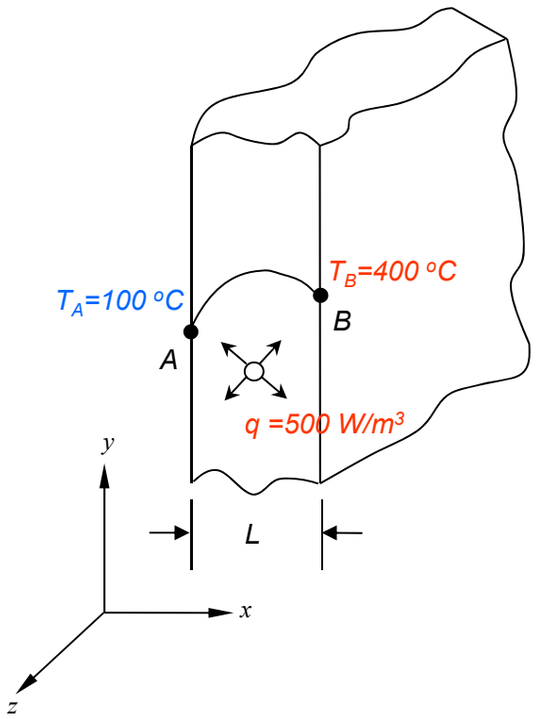
How do we do that? Let’s see this with a quick example:

Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

How do we do that? Let's see this with a quick example:

Imagine you have a steady heat conduction problem in a large brick plate with a uniform heat generation

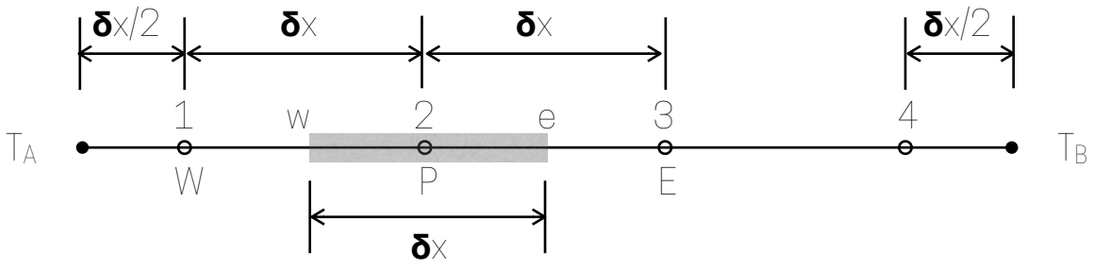


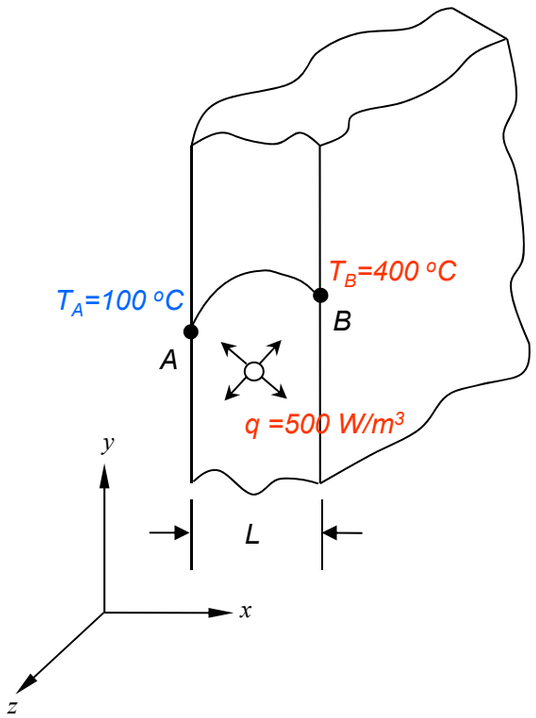
Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

How do we do that? Let's see this with a quick example:

Imagine you have a steady heat conduction problem in a large brick plate with a uniform heat generation



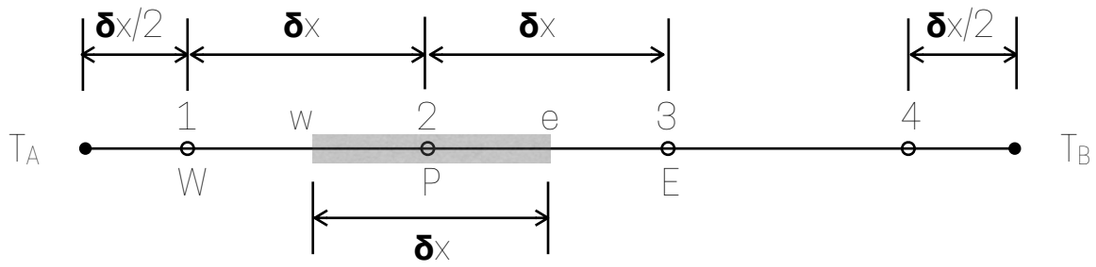


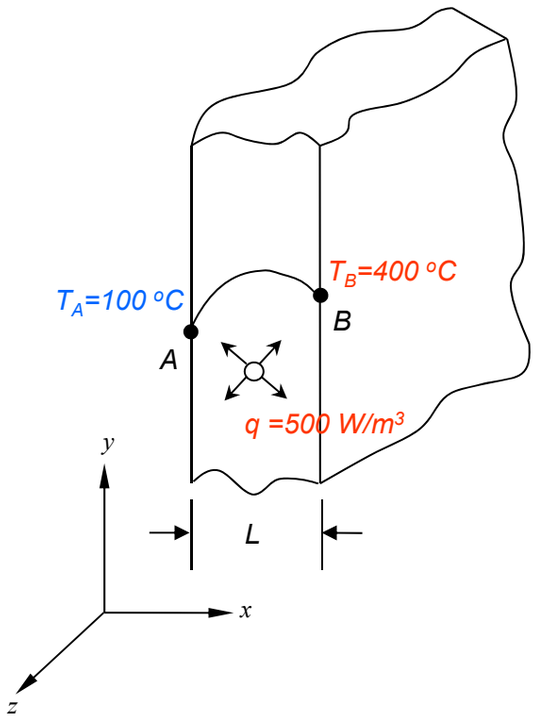
Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

How do we do that? Let's see this with a quick example:

Imagine you have a steady heat conduction problem in a large brick plate with a uniform heat generation





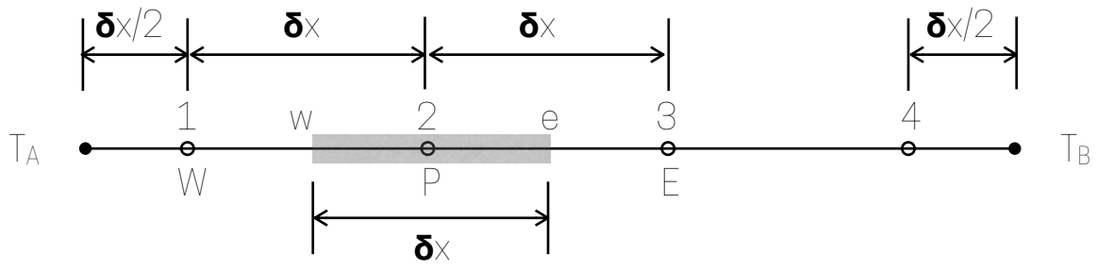
What is the governing equation for a constant thermal conductivity and uniform heat generations

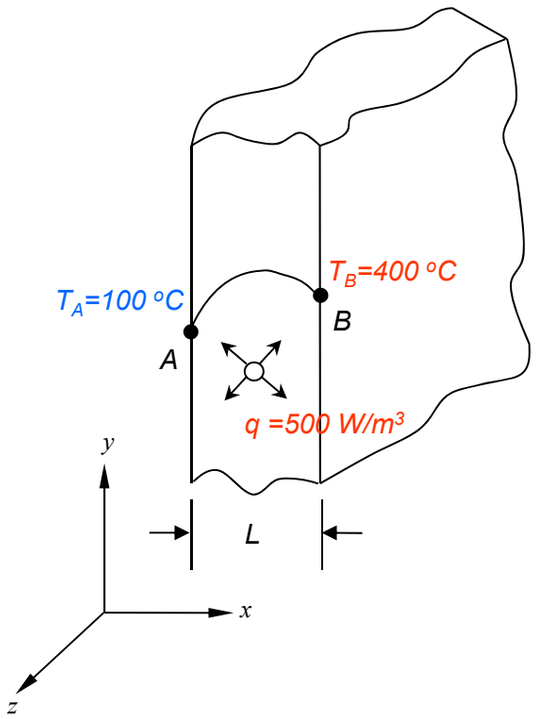
Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

How do we do that? Let's see this with a quick example:

Imagine you have a steady heat conduction problem in a large brick plate with a uniform heat generation





What is the governing equation for a constant thermal conductivity and uniform heat generations

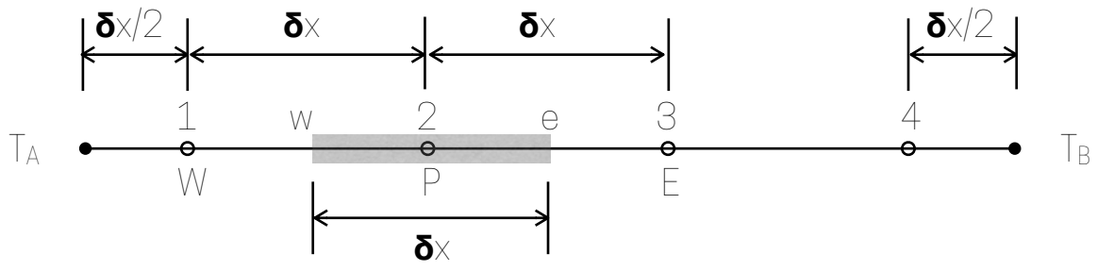
$$k\left(\frac{\partial T}{\partial x}\right)_e - k\left(\frac{\partial T}{\partial x}\right)_w + q\delta x = 0$$

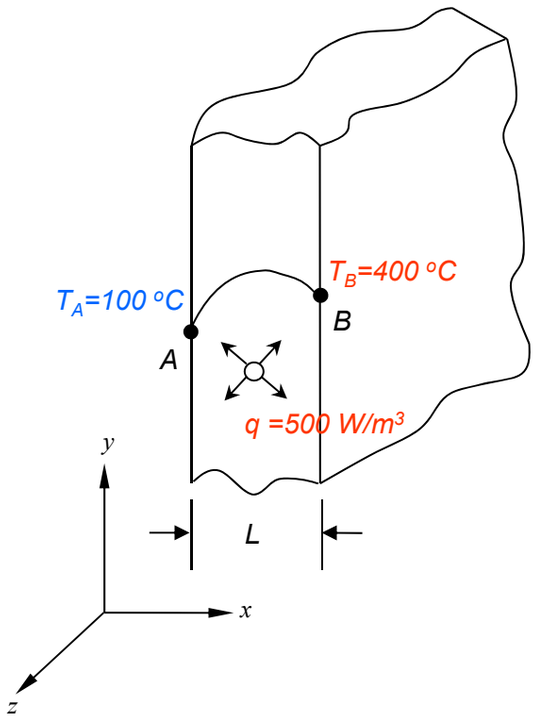
Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

How do we do that? Let's see this with a quick example:

Imagine you have a steady heat conduction problem in a large brick plate with a uniform heat generation





What is the governing equation for a constant thermal conductivity and uniform heat generations

$$k\left(\frac{\partial T}{\partial x}\right)_e - k\left(\frac{\partial T}{\partial x}\right)_w + q\delta x = 0$$

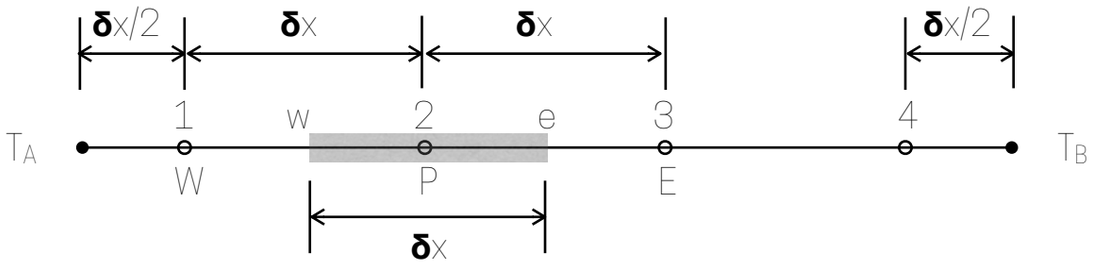
Linear approximation in a uniform grid gives:

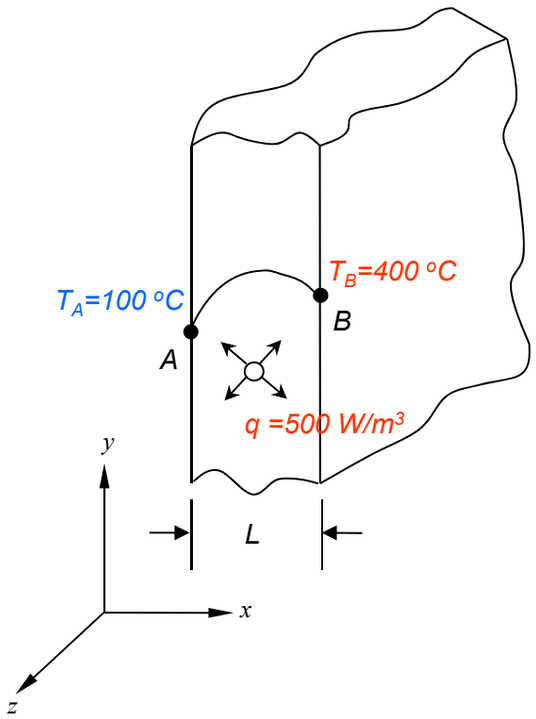
Algebraic form of equations

Once you have discretised the equations, the last step before solving numerically the system is to put all of them in algebraic form (simply matrix form).

How do we do that? Let's see this with a quick example:

Imagine you have a steady heat conduction problem in a large brick plate with a uniform heat generation





What is the governing equation for a constant thermal conductivity and uniform heat generations

$$k\left(\frac{\partial T}{\partial x}\right)_e - k\left(\frac{\partial T}{\partial x}\right)_w + q\delta x = 0$$

Linear approximation in a uniform grid gives:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

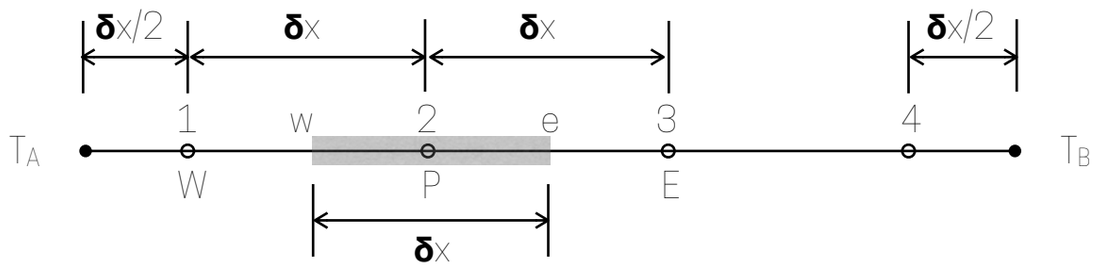
Specifically for each node:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:

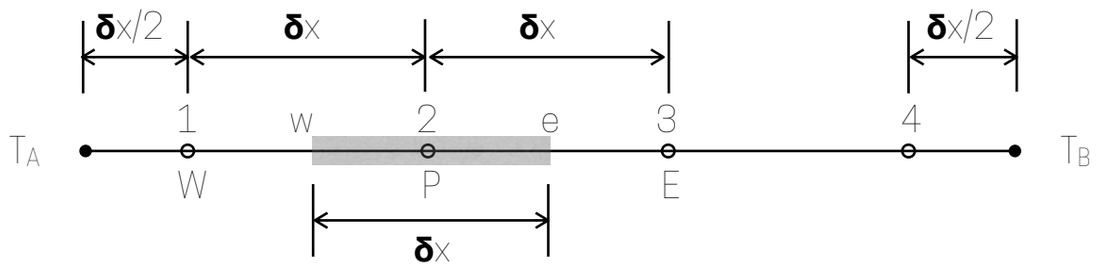


Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3:

- Node 1:

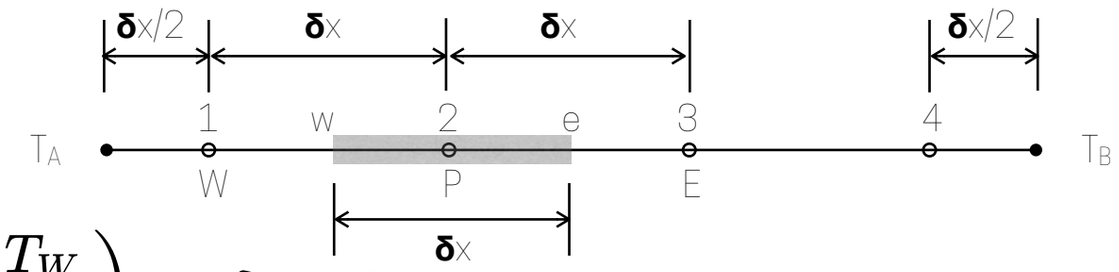
- Node 4:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{2kA}{\delta x}T_P + \frac{kA}{\delta x}T_W + q\delta x = 0 \qquad a_E T_E - a_P T_P + a_W T_W + b = 0$$

- Node 1:

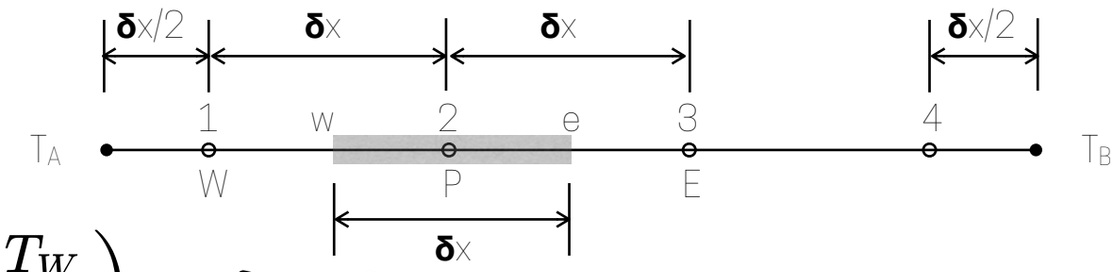
- Node 4:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{2kA}{\delta x}T_P + \frac{kA}{\delta x}T_W + q\delta x = 0 \qquad a_E T_E - a_P T_P + a_W T_W + b = 0$$

- Node 1:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_A}{\frac{\delta x}{2}}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{3kA}{\delta x}T_P + \frac{2kA}{\delta x}T_A + q\delta x = 0$$

$$a_E T_E - a_P T_P + b = 0$$

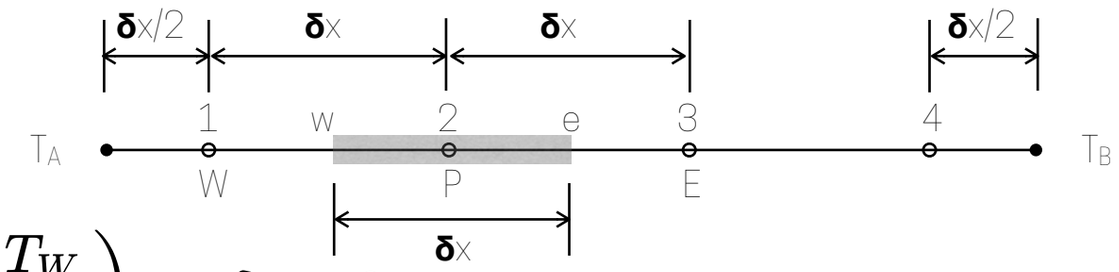
- Node 4:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{2kA}{\delta x}T_P + \frac{kA}{\delta x}T_W + q\delta x = 0 \qquad a_E T_E - a_P T_P + a_W T_W + b = 0$$

- Node 1:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_A}{\frac{\delta x}{2}}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{3kA}{\delta x}T_P + \frac{2kA}{\delta x}T_A + q\delta x = 0$$

$$a_E T_E - a_P T_P + b = 0$$

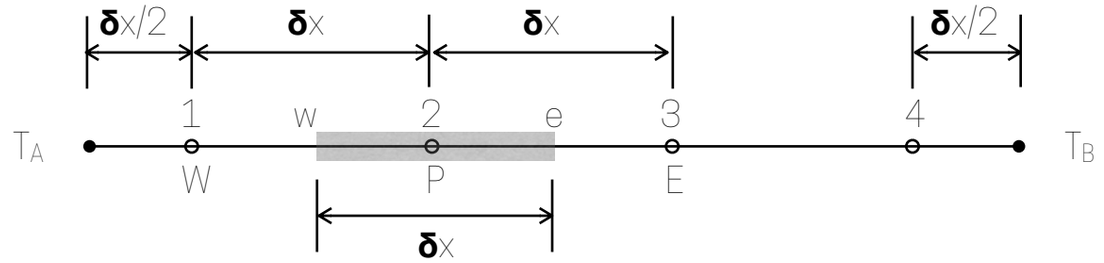
- Node 4:

$$kA\left(\frac{T_B - T_P}{\frac{\delta x}{2}}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

$$\frac{2kA}{\delta x}T_B - \frac{3kA}{\delta x}T_P + \frac{kA}{\delta x}T_w + q\delta x = 0$$

$$a_W T_W - a_P T_P + b = 0$$

Algebraic form of equations



$$\begin{pmatrix} \frac{3kA}{\delta x} & -\frac{kA}{\delta x} & 0 & 0 \\ -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} & 0 \\ 0 & -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} \\ 0 & 0 & -\frac{kA}{\delta x} & \frac{3kA}{\delta x} \end{pmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{pmatrix} \frac{2kA}{\delta x} T_A + q\delta x \\ q\delta x \\ q\delta x \\ \frac{2kA}{\delta x} T_B + q\delta x \end{pmatrix}$$

In this example we can invert that matrix very easy….however, in general for CFD, equations are not linear and matrices are extremely large…so inverting them is not the most efficient solution

Iterative Methods!

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. **Iterative methods**

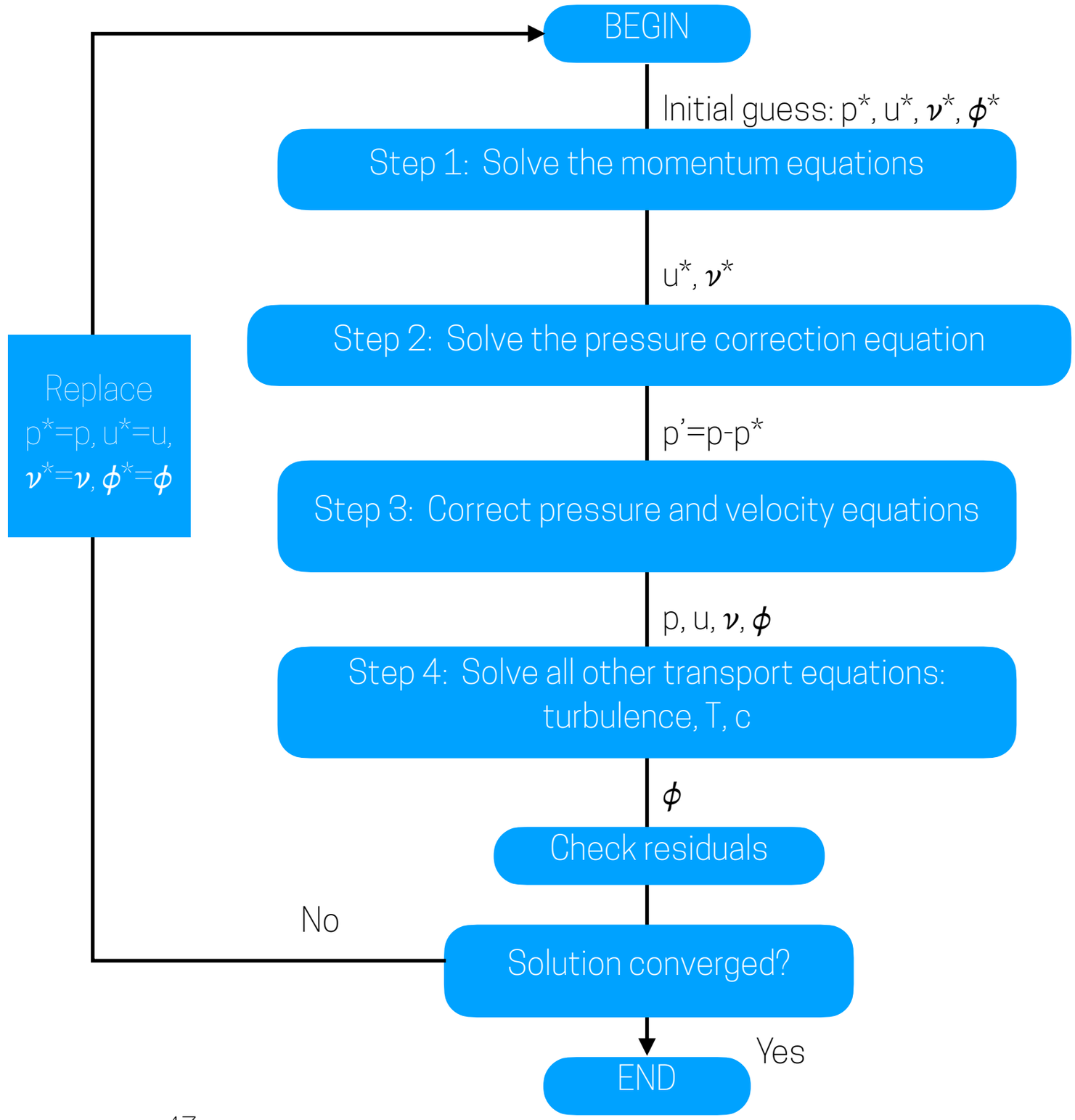
8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Iterative methods: SIMPLE

SIMPLE: the semi-implicit method for pressure-linkage equations



Iterative methods: convergence

Iterative methods: convergence

- Since solving the equations is an iterative process, we need to set a criteria to determine when we consider the problem as solved

- This will affect accuracy and efficiency

- There are two types of convergence:

  1. Convergence associated with the convergence of the numerical solutions towards a grid-independent solution, closely linked to discretization error;
  2. Refers to error reduction in iterative solution methods. For example when solving a steady problem, a traditional convergence criteria is when the residuals (differences between the time present solution and the time - 1 solution) are lower than a determined threshold, for example 10^{-6}.

Iterative methods

Steady vs Unsteady

Iterative methods

Steady vs Unsteady

The method used for the solution depend on the problem:

Iterative methods

Steady vs Unsteady

The method used for the solution depend on the problem:

- **Unsteady flow:** are based on the use of initial value problems for ordinary differential equations (marching in time), at each time step the problem is solved, and each time step the boundary conditions vary.
  - You could think about the wind in a city, if you look at it instantaneously it varies, and therefore it is an unsteady problem.

- **Steady flow:** are usually solved by pseudo-time marching or iteration schemes, since the equations are non-linear, each iteration is used to solve them.
  - As an example, you could think about wind over a year, imagine you need to project the determined amount of wind energy that can be extracted over a hill for a potential wind farm, you could used a steady flow solution based on a yearly averaged data.

Questions so far?

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

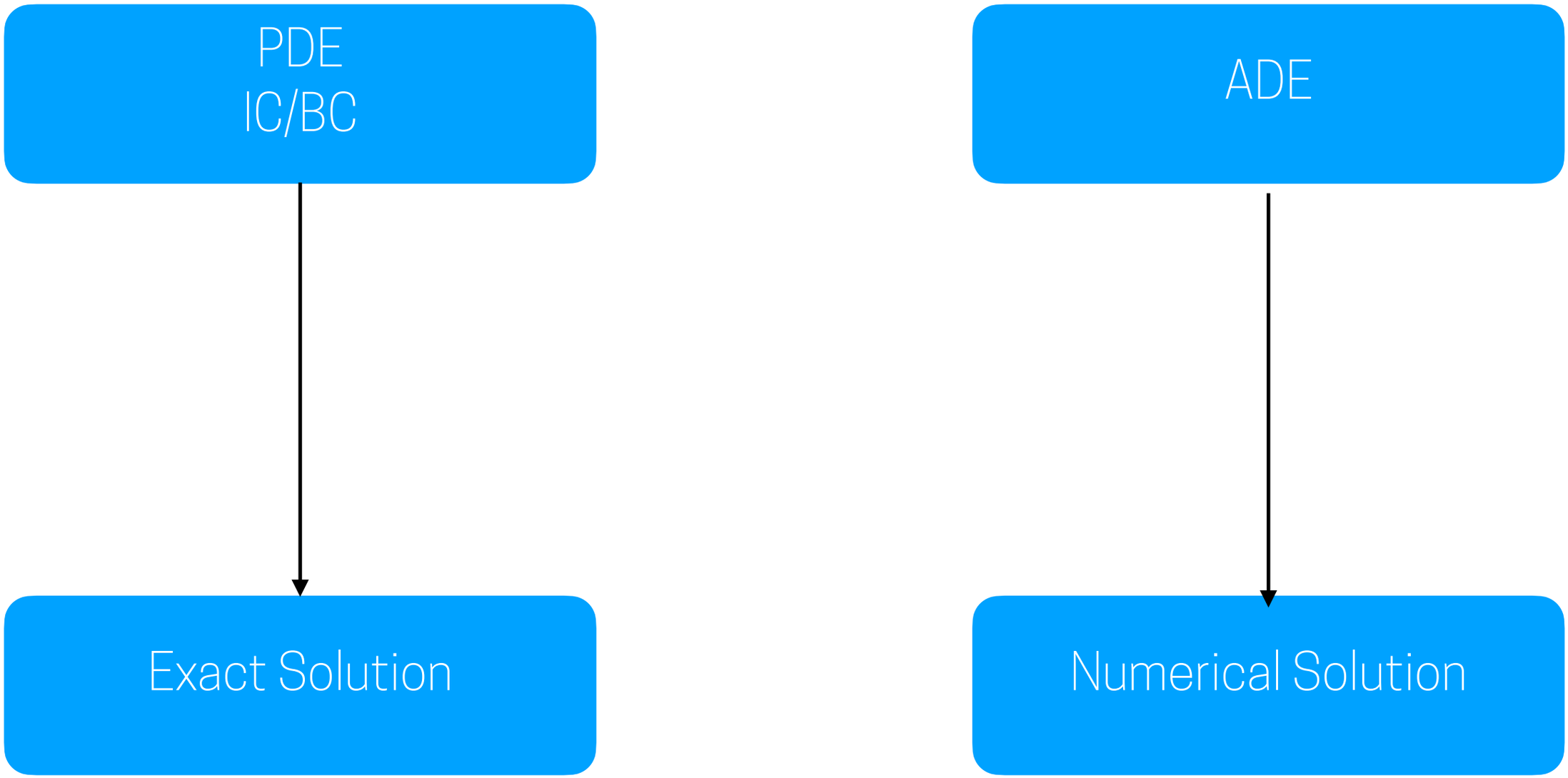
6. Iterative methods

8. **Properties of Numerical Solution Methods**

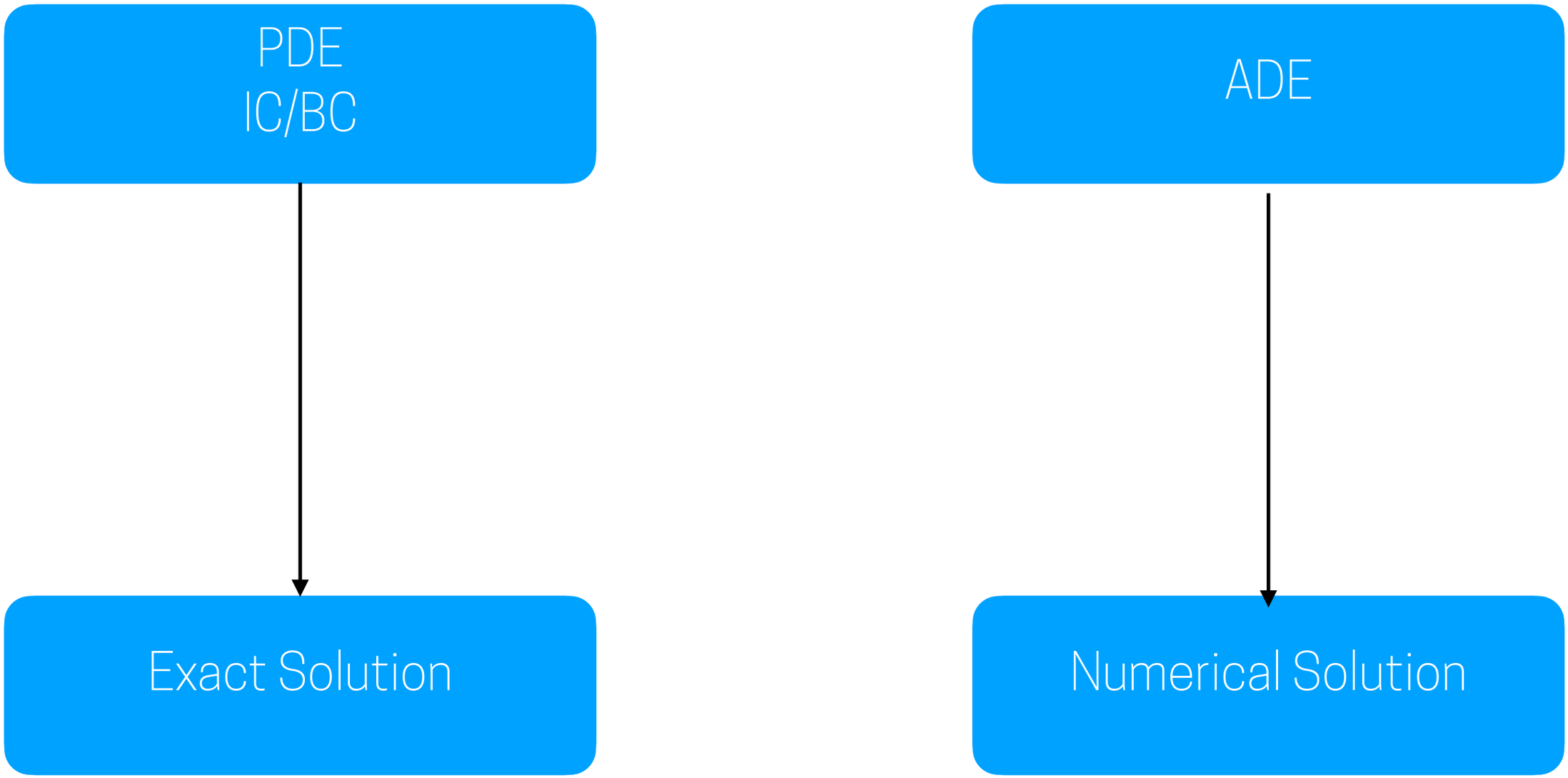
9. Simulating Urban Neutral ABL flows

10. Wrap-up

Properties of Numerical Solution Methods

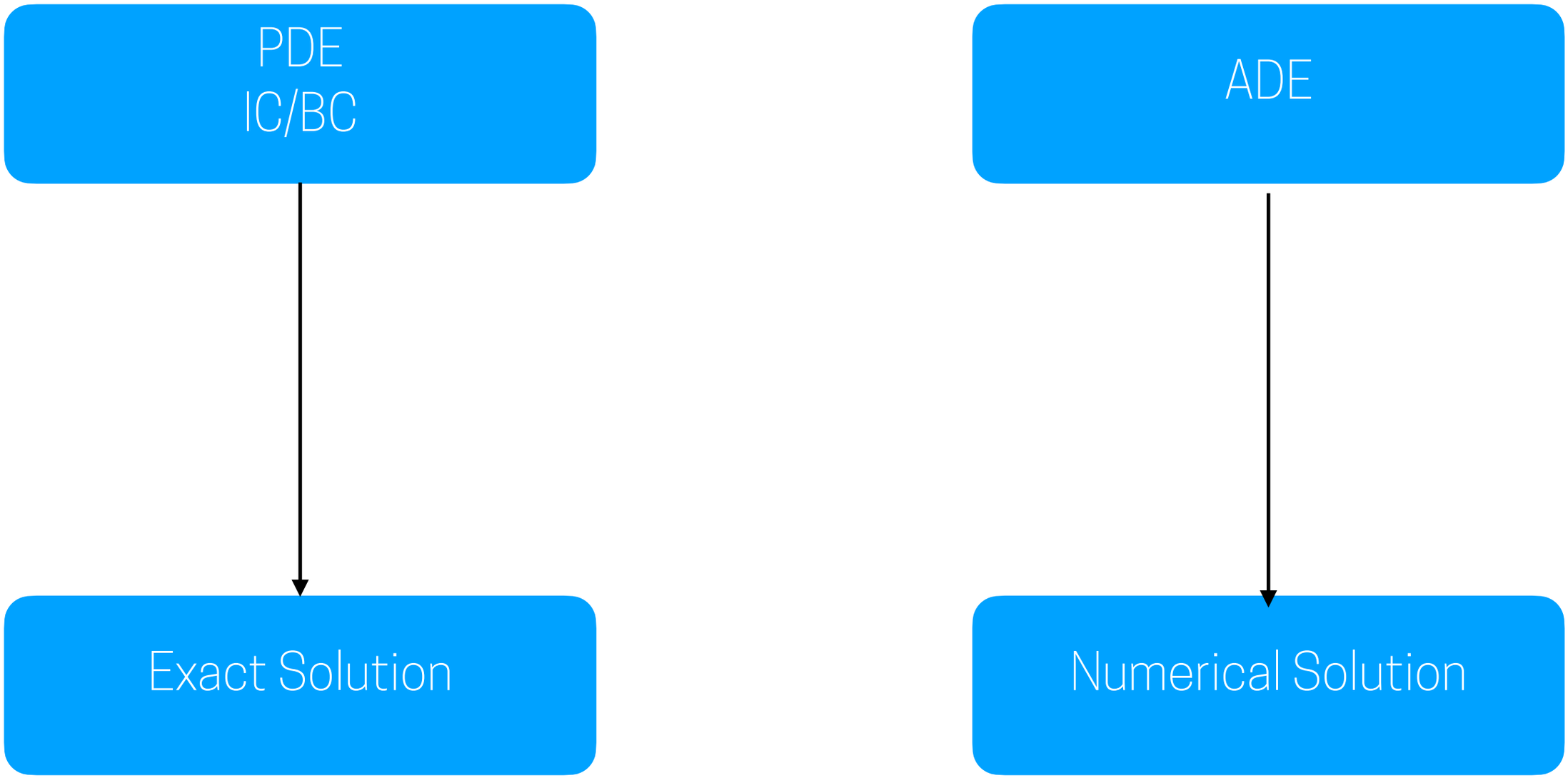


Properties of Numerical Solution Methods

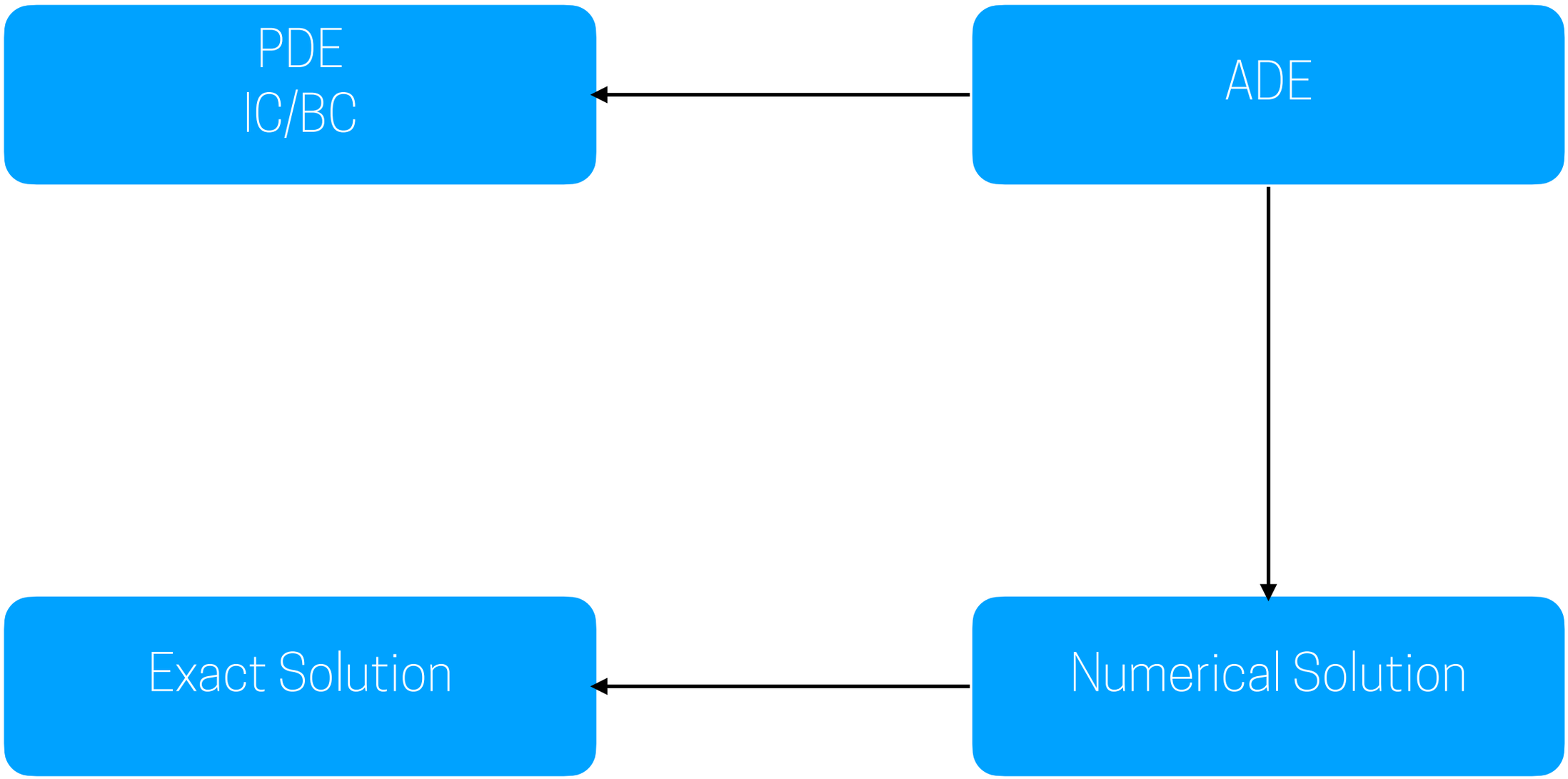


What we want…

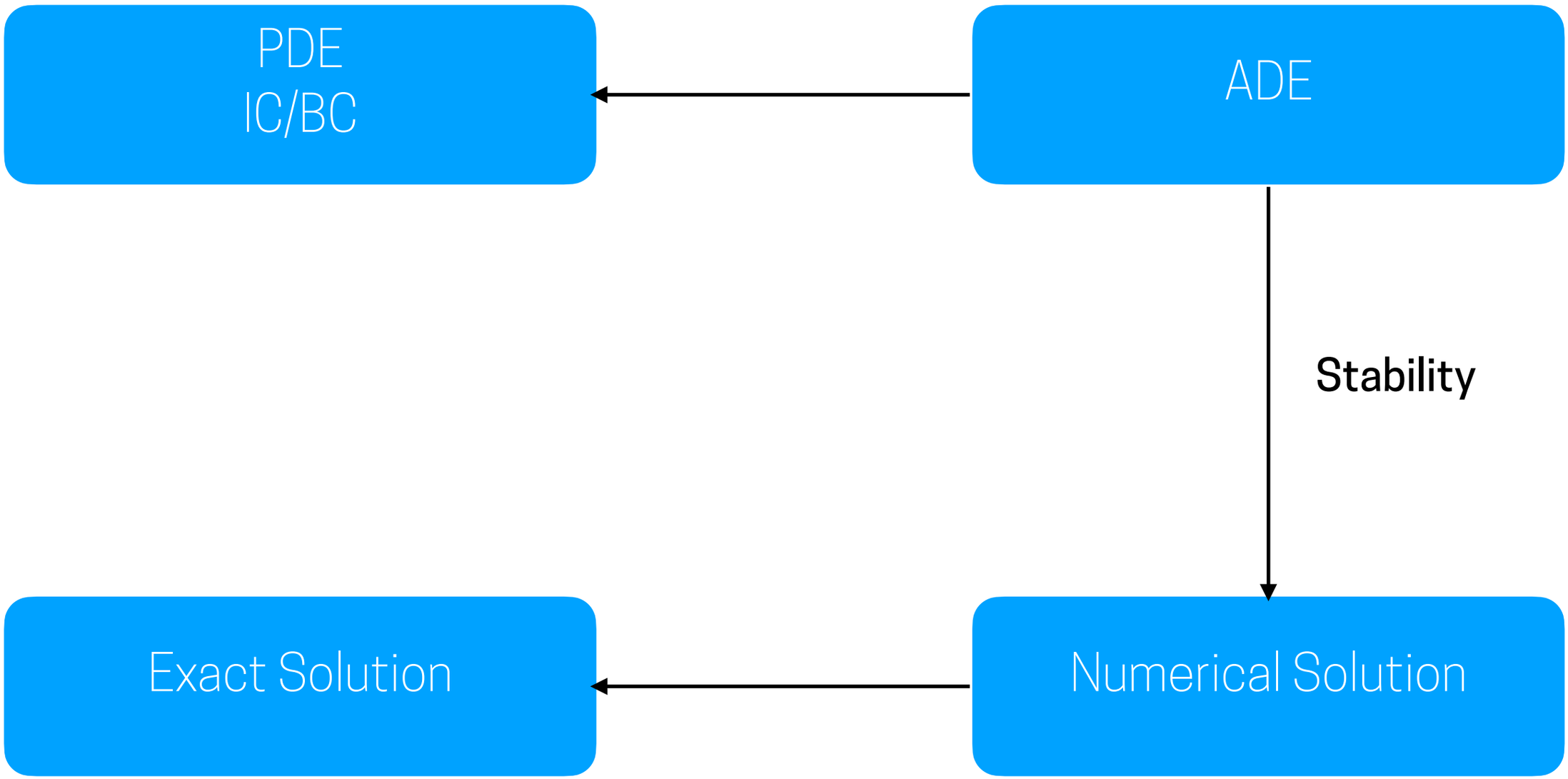
Properties of Numerical Solution Methods



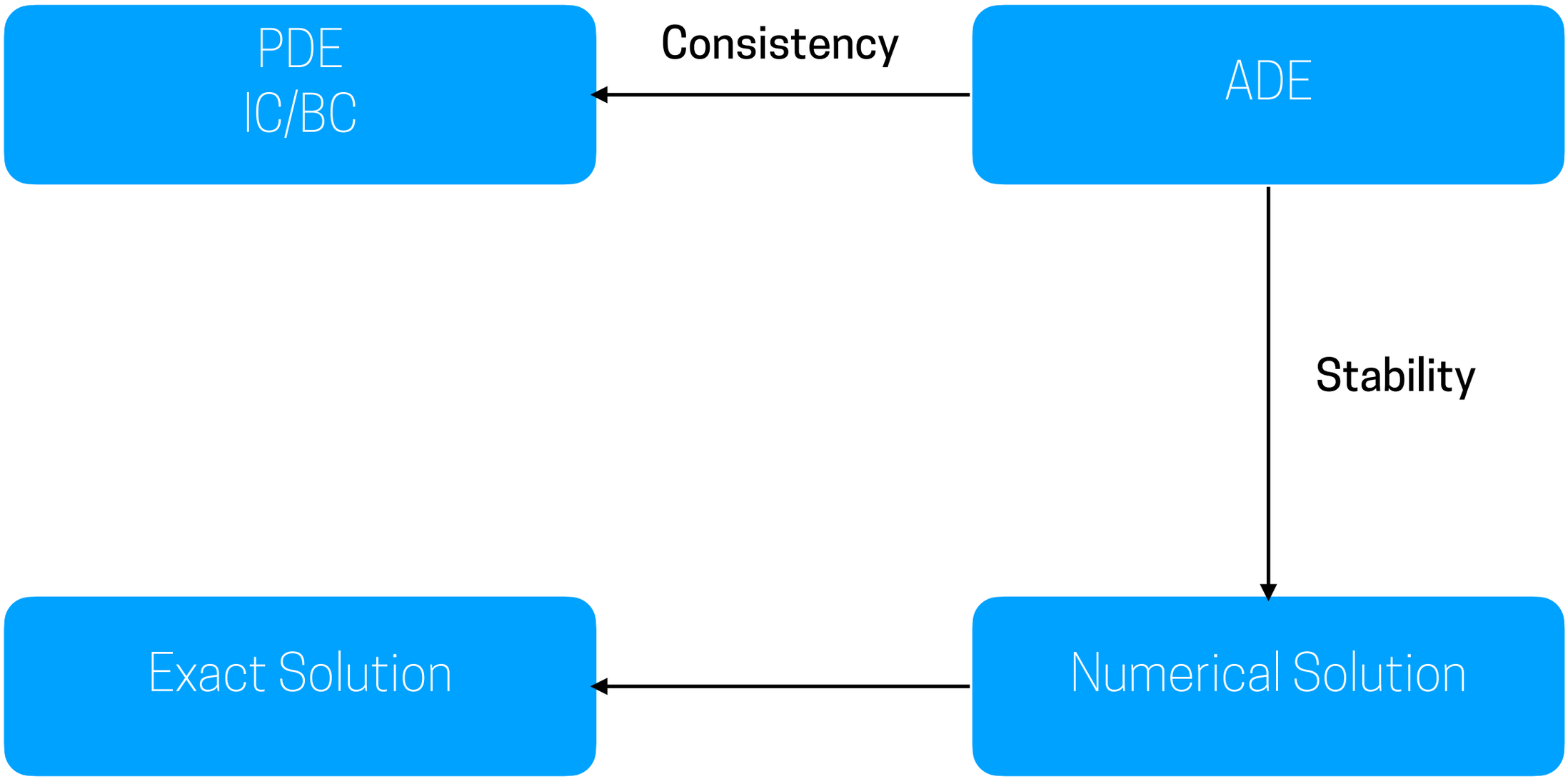
Properties of Numerical Solution Methods



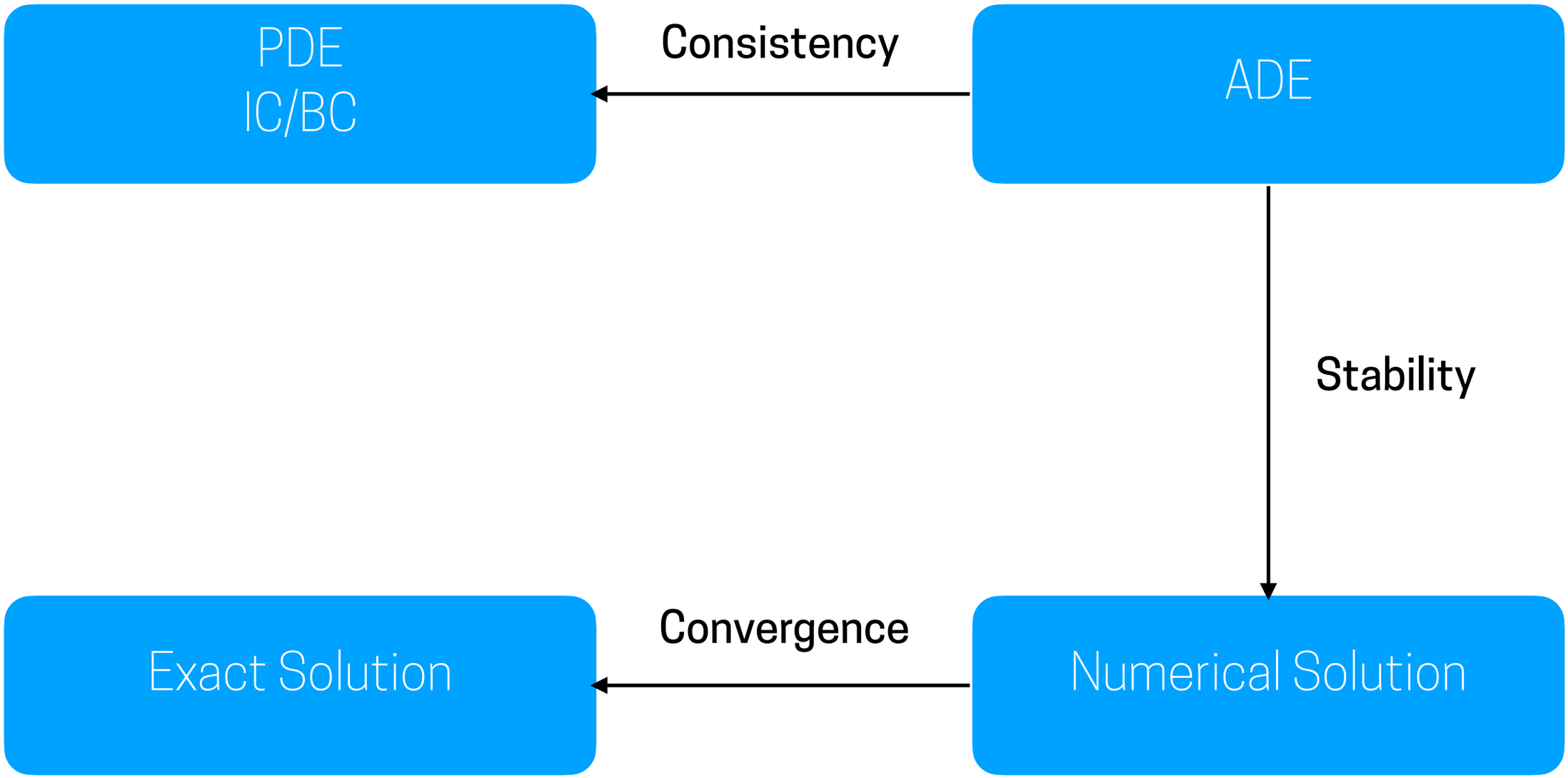
Properties of Numerical Solution Methods



Properties of Numerical Solution Methods

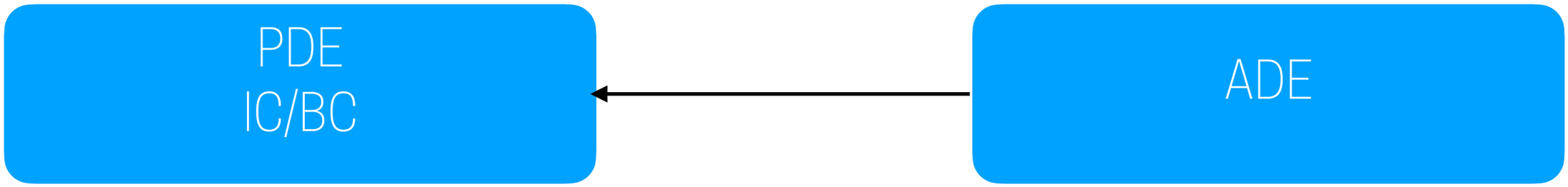


Properties of Numerical Solution Methods



Properties of Numerical Solution Methods

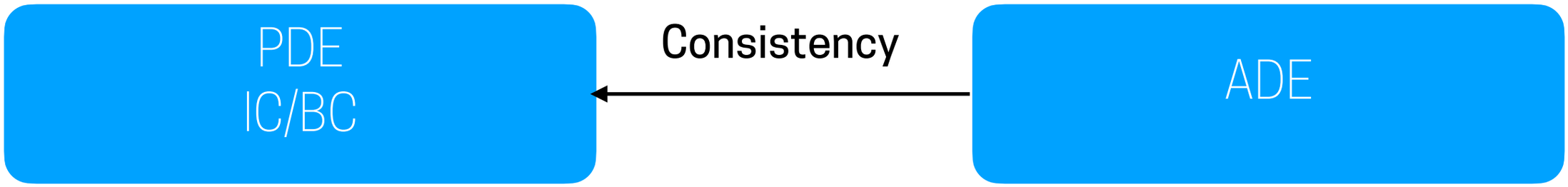
Consistency



- Discretization should become exact as the grid and time spacing tends to zero.
- The difference between the discretized equation and the exact one is called truncation error.
- For a method to be consistent, the truncation error must become zero when the mesh spacing $\boldsymbol{\Delta}t \rightarrow 0$ and/or $\boldsymbol{\Delta}x_i \rightarrow 0$.
- The truncation error is usually proportional to a power of the grid spacing $((\boldsymbol{\Delta}x_i)^n)$ and/or the time step $((\boldsymbol{\Delta}t)^n)$, where n is the nth-order approximation.

Properties of Numerical Solution Methods

Consistency

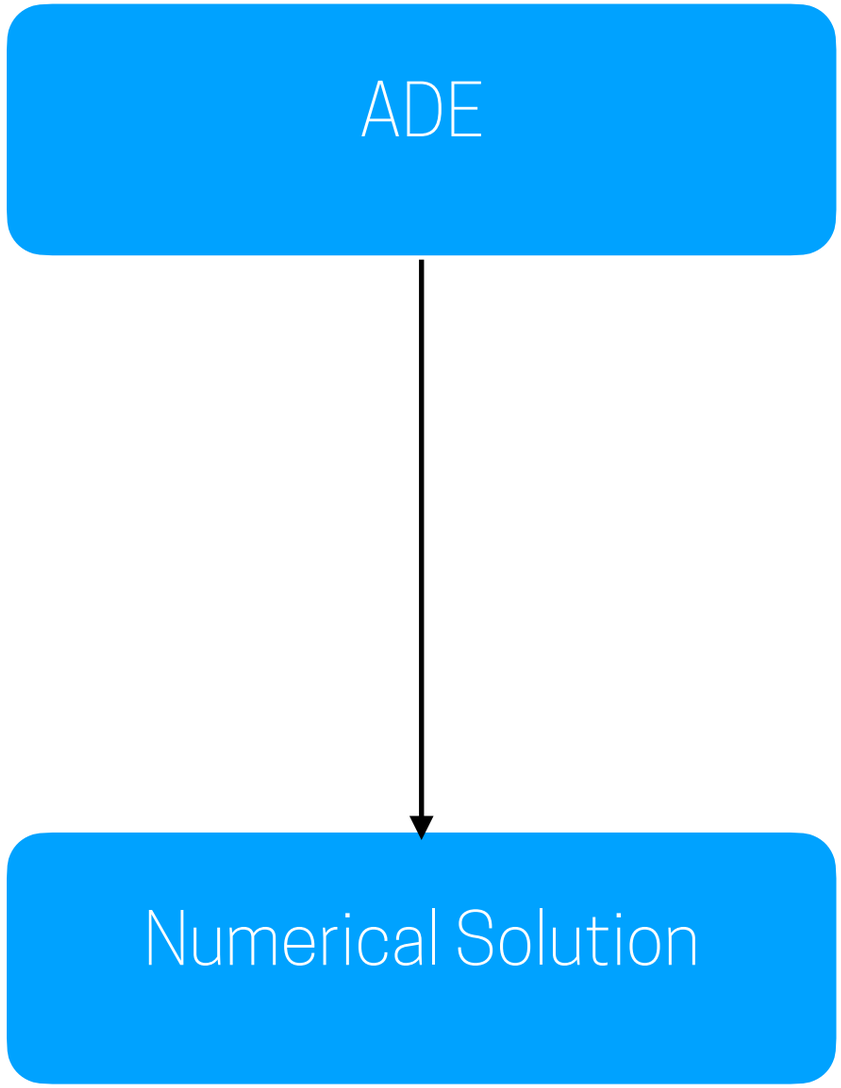


- Discretization should become exact as the grid and time spacing tends to zero.
- The difference between the discretized equation and the exact one is called truncation error.
- For a method to be consistent, the truncation error must become zero when the mesh spacing $\boldsymbol{\Delta}t \rightarrow 0$ and/or $\boldsymbol{\Delta}x_i \rightarrow 0$.
- The truncation error is usually proportional to a power of the grid spacing $((\boldsymbol{\Delta}x_i)^n)$ and/or the time step $((\boldsymbol{\Delta}t)^n)$, where n is the nth-order approximation.

Properties of Numerical Solution Methods

Stability

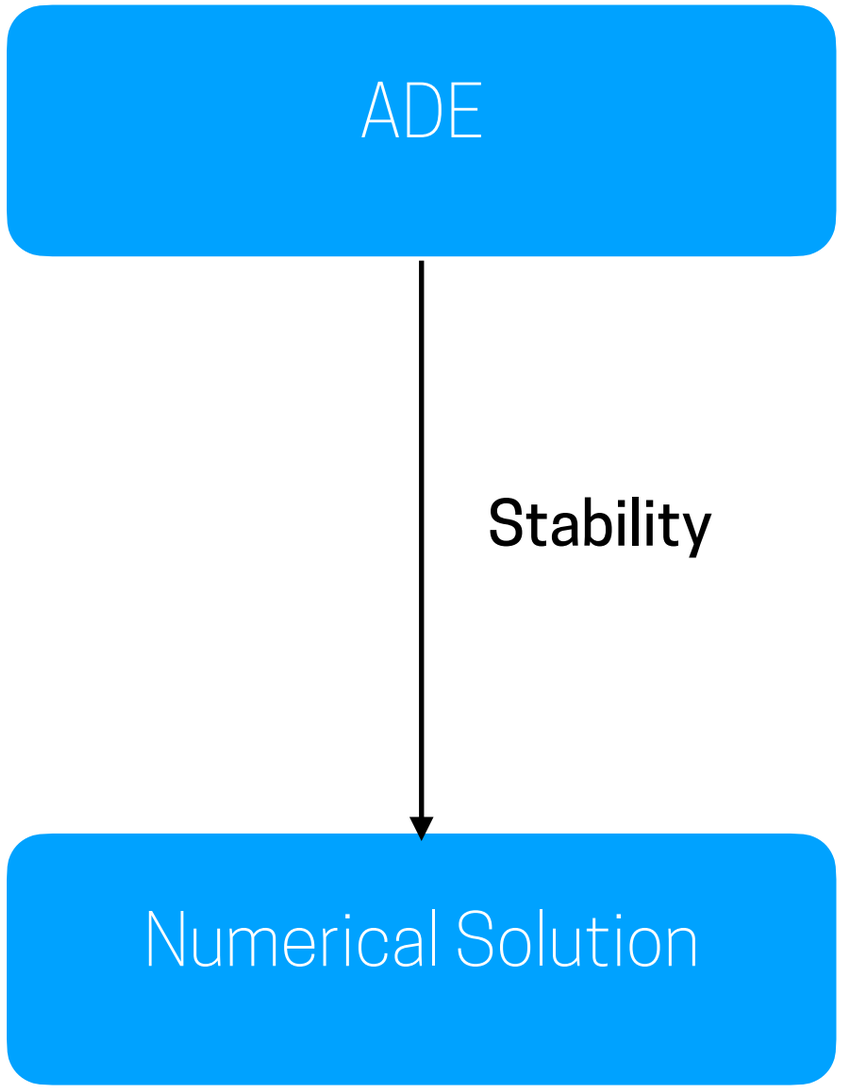
- Stability guarantees that the method produces a bounded solution whenever the solution of the exact equation is bounded.
- In iterative methods, a stable method is one that does not diverge, which basically means numerical values of the quantities become so large and nonphysical that a computer can not handle it



Properties of Numerical Solution Methods

Stability

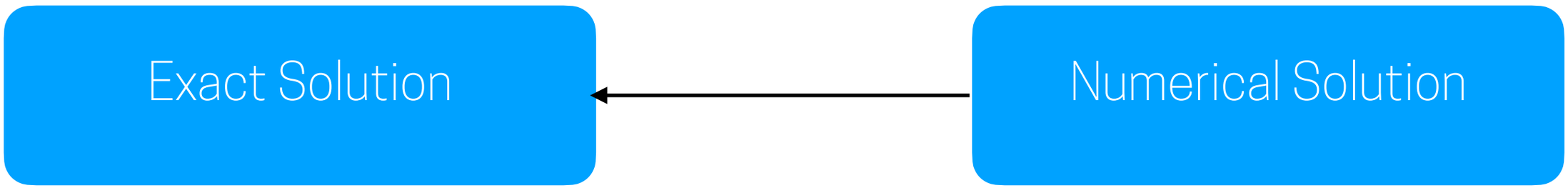
- Stability guarantees that the method produces a bounded solution whenever the solution of the exact equation is bounded.
- In iterative methods, a stable method is one that does not diverge, which basically means numerical values of the quantities become so large and nonphysical that a computer can not handle it



Properties of Numerical Solution Methods

Convergence

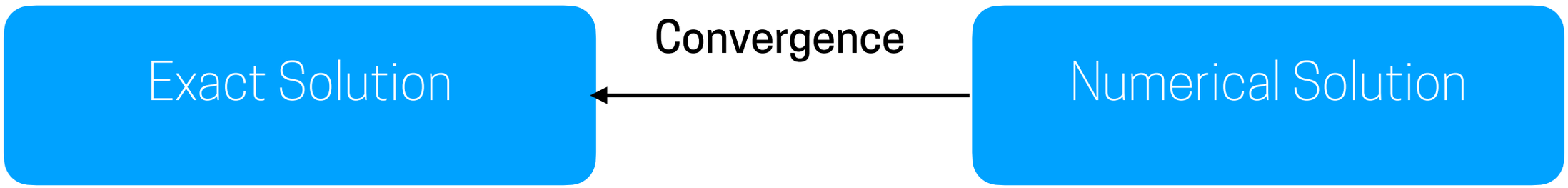
- The solution of the discretized equations tends to the exact solution of the differential equation as the grid spacing tends to zero.
- For linear initial value problems, the “Lax equivalence theorem (Richtmyer and Morton, 1967)” states given a properly posed linear initial value problem an a finite difference approximation to it that satisfies the consistency condition, stability is the necessary and sufficient condition for convergence.
- Unfortunately, for non-linear problems which are strongly affected by the boundary conditions, stability and convergence of a method are not easy to prove.
- Generally convergence is checked repeating a calculation on a series of successively refined grids, and if the method is stable, and all the approximations used in the discretization process are consistent, we usually find that the solution does converge to grid-independent solution.



Properties of Numerical Solution Methods

Convergence

- The solution of the discretized equations tends to the exact solution of the differential equation as the grid spacing tends to zero.
- For linear initial value problems, the “Lax equivalence theorem (Richtmyer and Morton, 1967)” states given a properly posed linear initial value problem an a finite difference approximation to it that satisfies the consistency condition, stability is the necessary and sufficient condition for convergence.
- Unfortunately, for non-linear problems which are strongly affected by the boundary conditions, stability and convergence of a method are not easy to prove.
- Generally convergence is checked repeating a calculation on a series of successively refined grids, and if the method is stable, and all the approximations used in the discretization process are consistent, we usually find that the solution does converge to grid-independent solution.



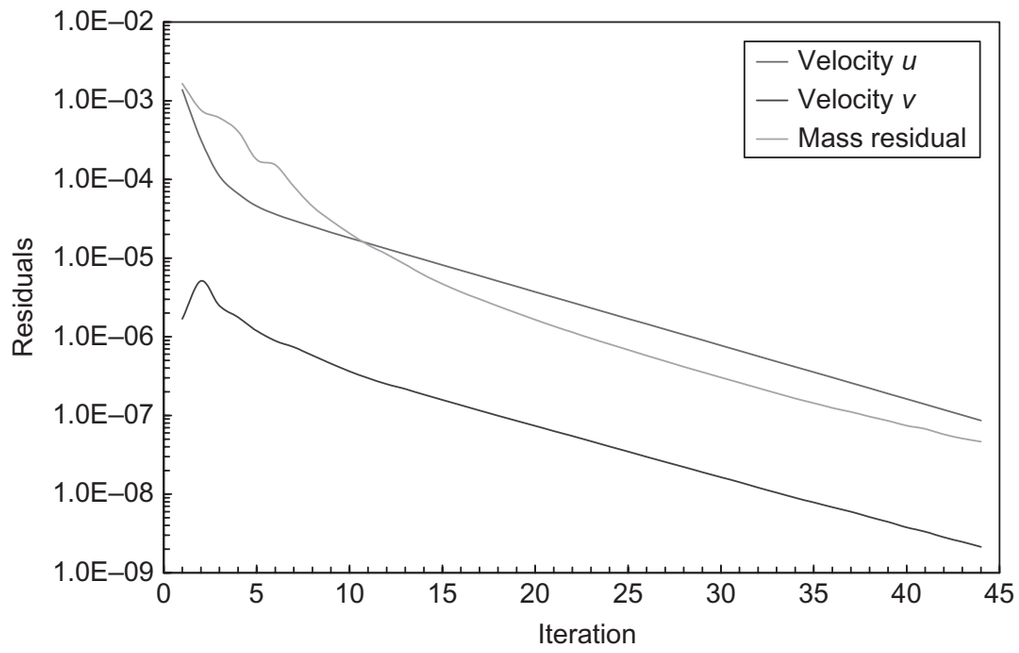
Properties of Numerical Solution Methods

Convergence

- In practice to check convergence on a case we can do two things:

  1. Plot the residuals and observe the decay and the number at the end of the simulation. If their values decayed several orders of magnitude and stabilised at around $<10^{-5}$, we can assume the simulation is converged

  2. Plotting the field values and checking that further iterations will not change their values



Exact Solution ← **Convergence** ← Numerical Solution

Properties of Numerical Solution Methods

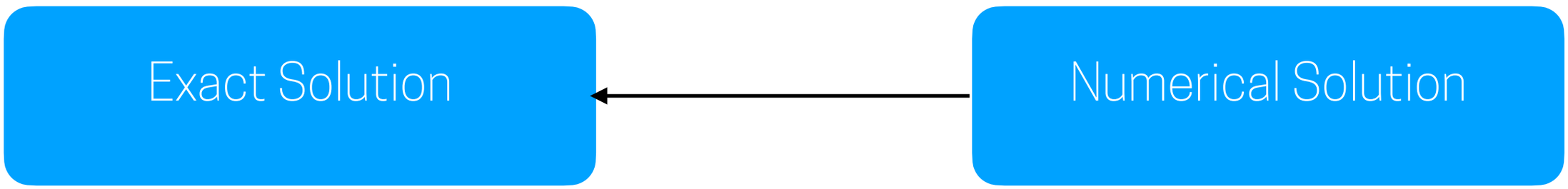
Accuracy

Numerical solutions of a fluid flow and heat transfer problems are only approximate solutions. In addition to the errors introduced in the course of the development of the solution algorithm, in programming or setting up the boundary conditions, numerical solutions always include three kinds of systematic errors:

- Modeling errors: these are the difference between the actual flow and the exact solution of the mathematical model.

- Discretization errors: difference between the exact solution of the conservation equations and the exact solution of the algebraic system of equations obtained by discretizaing these equations.

- Iteration errors: defined as the difference between the iterative and exact solutions of the algebraic equations systems.



Properties of Numerical Solution Methods

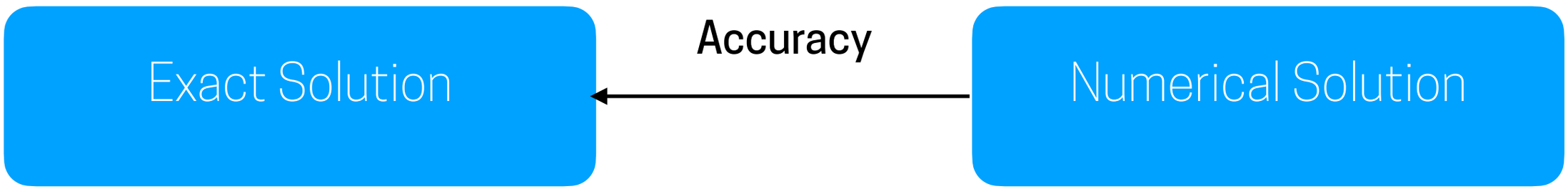
Accuracy

Numerical solutions of a fluid flow and heat transfer problems are only approximate solutions. In addition to the errors introduced in the course of the development of the solution algorithm, in programming or setting up the boundary conditions, numerical solutions always include three kinds of systematic errors:

- Modeling errors: these are the difference between the actual flow and the exact solution of the mathematical model.

- Discretization errors: difference between the exact solution of the conservation equations and the exact solution of the algebraic system of equations obtained by discretizaing these equations.

- Iteration errors: defined as the difference between the iterative and exact solutions of the algebraic equations systems.



Questions so far?

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Simulating urban neutral ABL flows

Computational domain

Simulating urban neutral ABL flows

Computational domain

The size of your domain should not affect the flow, remember we are confining an open flow within a box, so you need to make sure the box is big enough

Simulating urban neutral ABL flows

Computational domain

The size of your domain should not affect the flow, remember we are confining an open flow within a box, so you need to make sure the box is big enough

There are some guidelines that you should always follow in terms of:

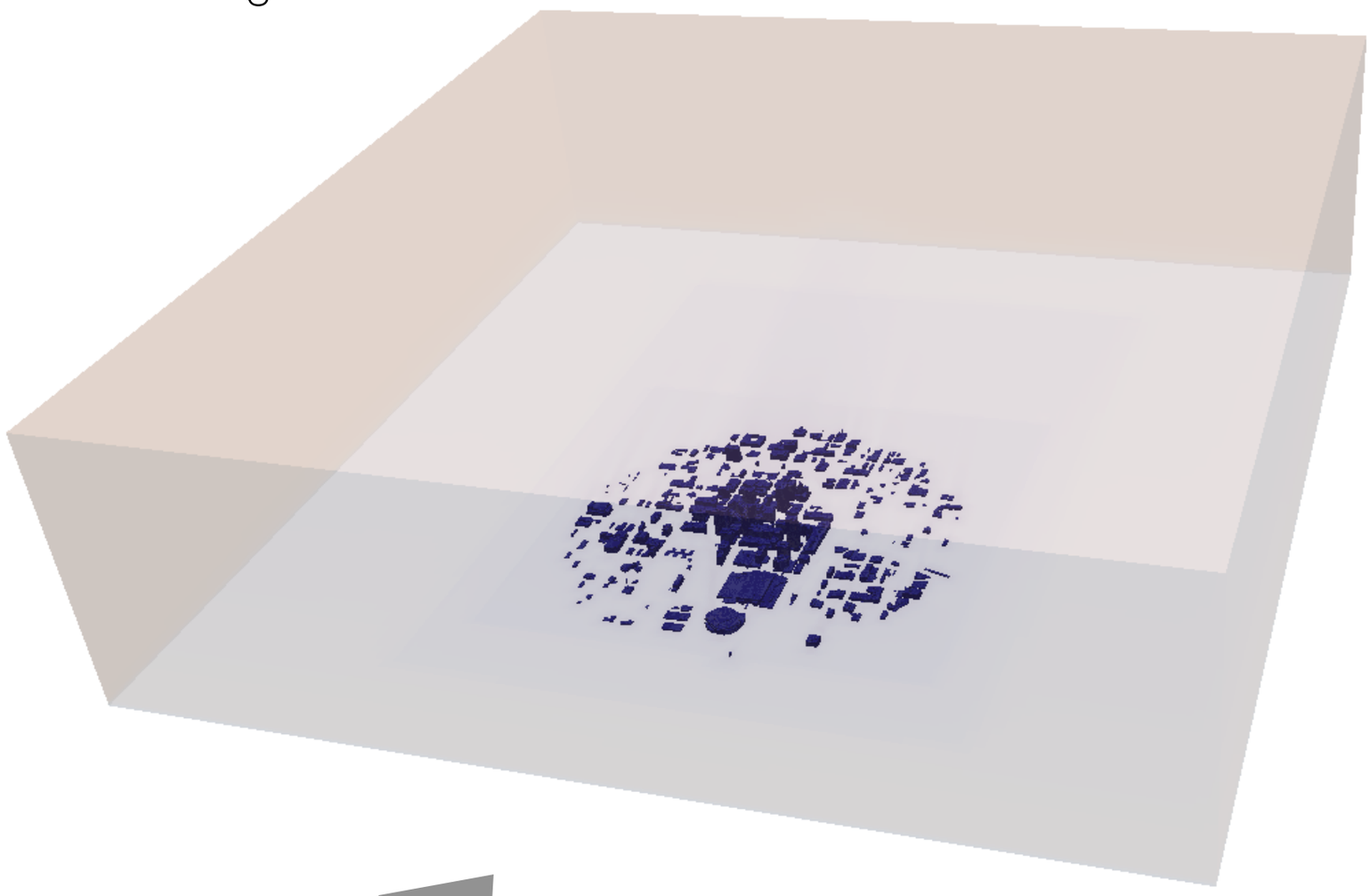
1. Minimum distance from the region of interest to the domain boundaries
2. Blockage

Simulating urban neutral ABL flows

Computational domain

1. Minimum distance from the region of interest to the domain boundaries

H_{max}= tallest building



Wind

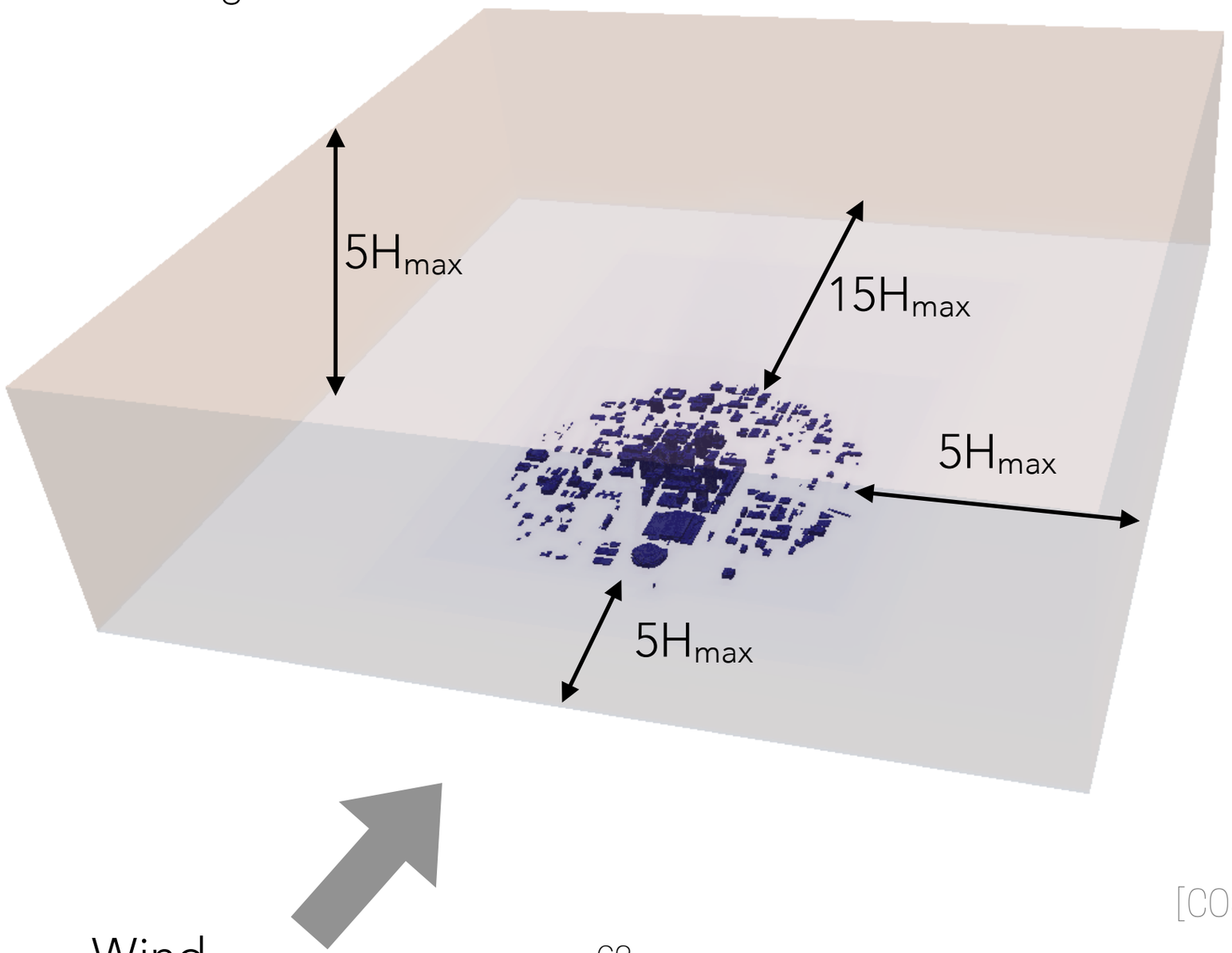
[COST action 732]

Simulating urban neutral ABL flows

Computational domain

1. Minimum distance from the region of interest to the domain boundaries

H_{max}= tallest building



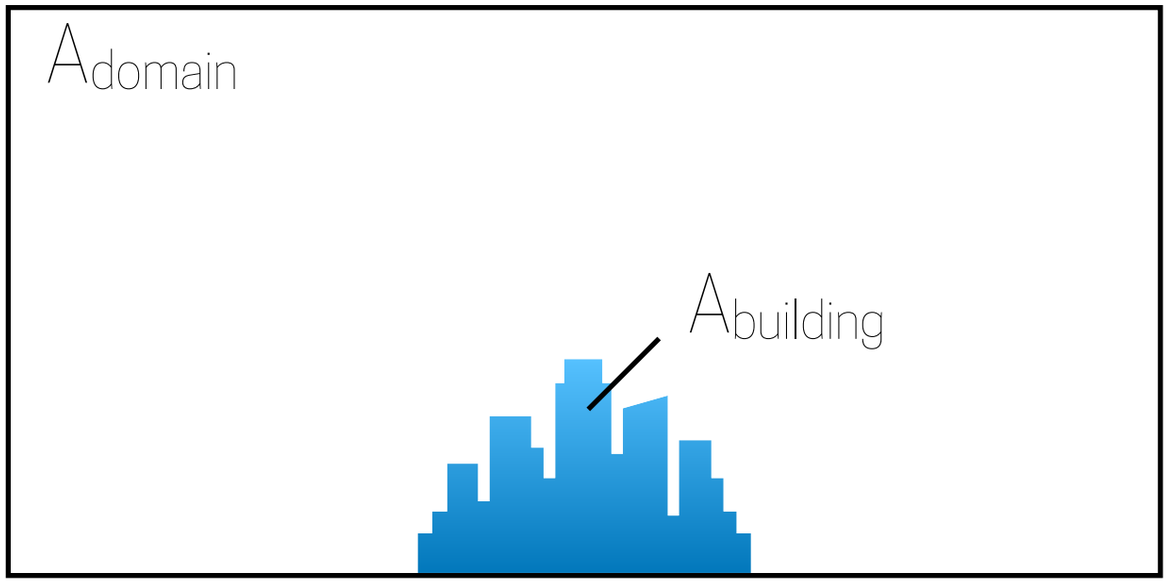
[COST action 732]

Simulating urban neutral ABL flows

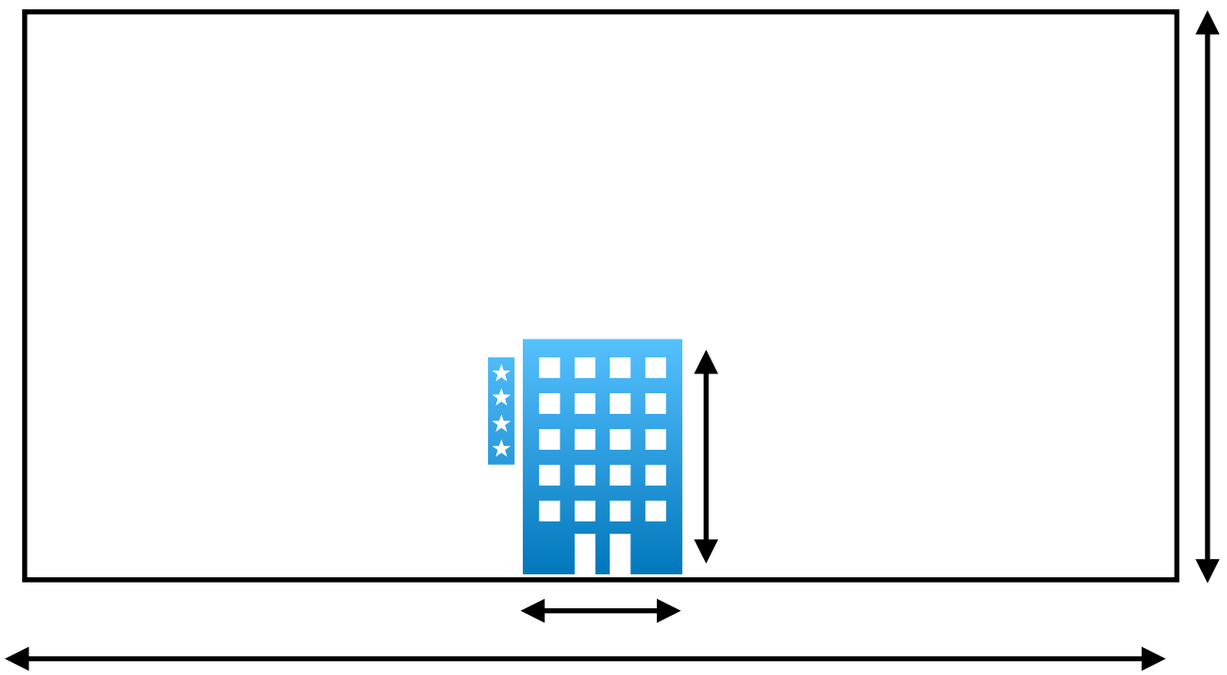
Computational domain

2. Maximum Blockage

$$BR = \frac{A_{building}}{A_{domain}} < 3 - 5\%$$



$$BR_L = \frac{L_{building}}{L_{domain}} \quad \&\& \quad BR_H = \frac{H_{building}}{H_{domain}} < 17 - 22\%$$



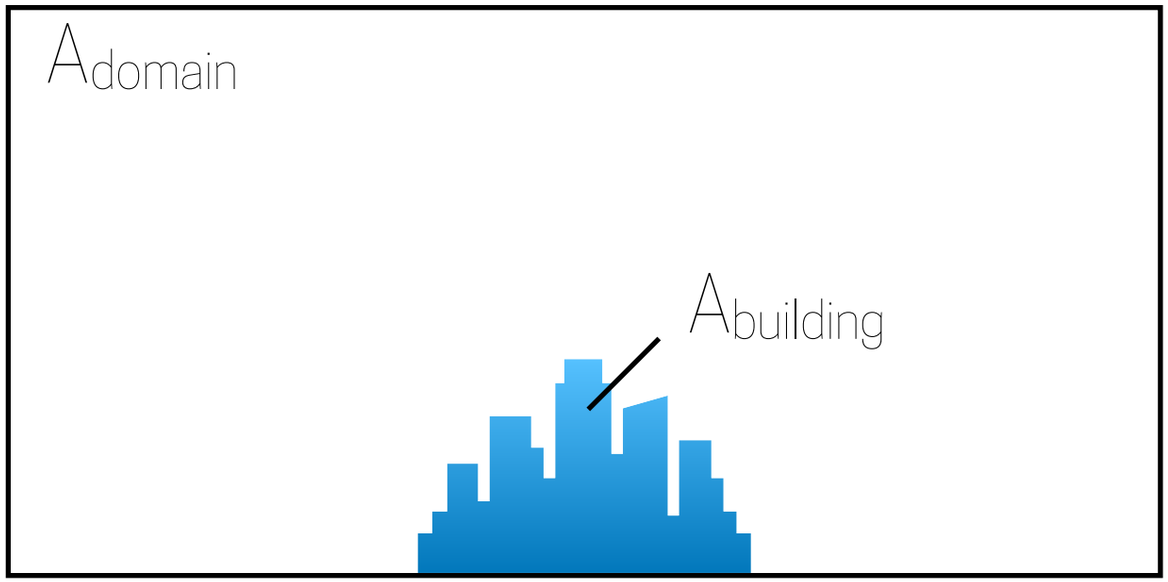
[Baetke, 1997, Blocken and Carmeliet, 2004]

Simulating urban neutral ABL flows

Computational domain

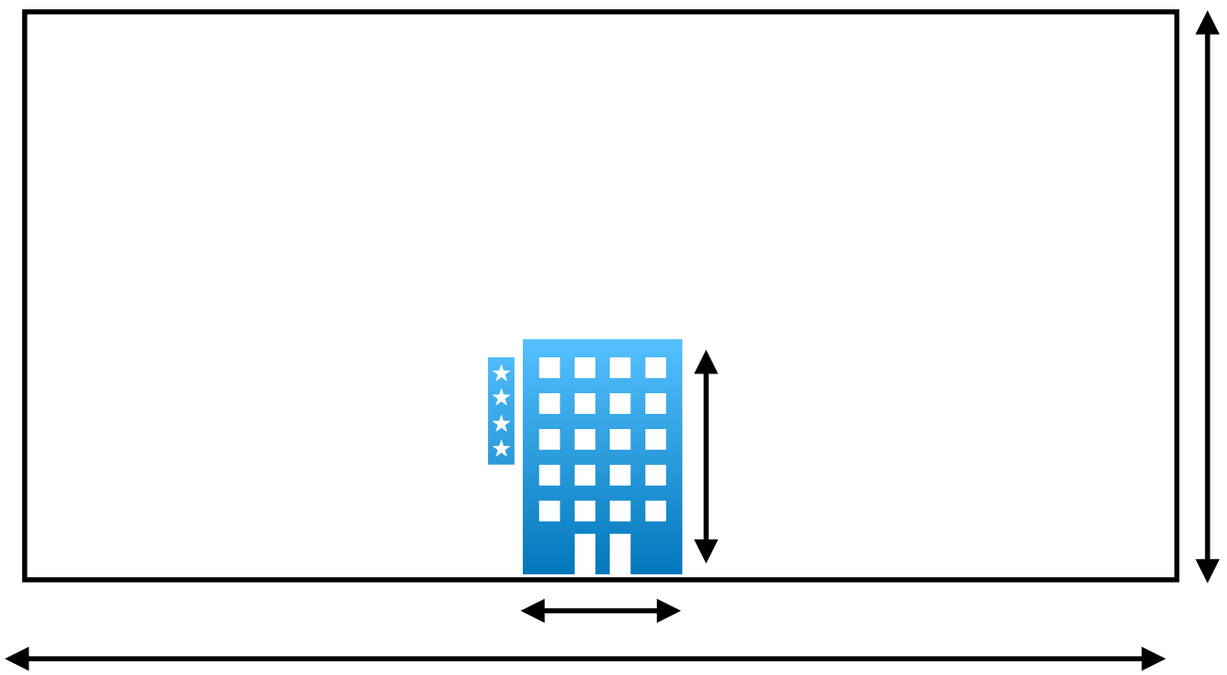
2. Maximum Blockage

$$BR = \frac{A_{building}}{A_{domain}} < 3 - 5\%$$



Directional Blockage

$$BR_L = \frac{L_{building}}{L_{domain}} \quad \&\& \quad BR_H = \frac{H_{building}}{H_{domain}} < 17 - 22\%$$



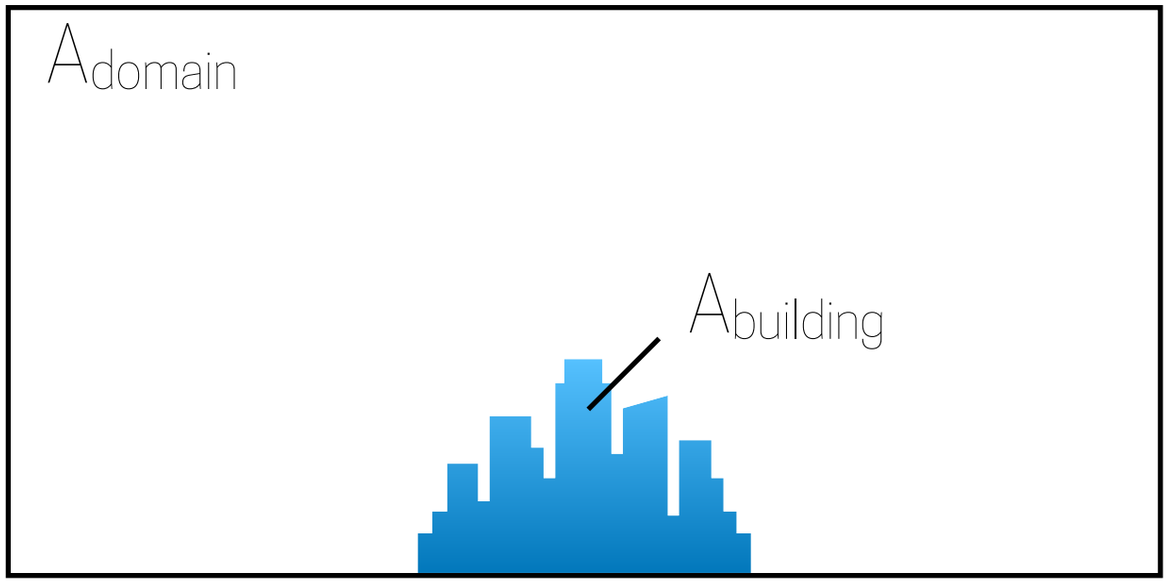
[Baetke, 1997, Blocken and Carmeliet, 2004]

Simulating urban neutral ABL flows

Computational domain

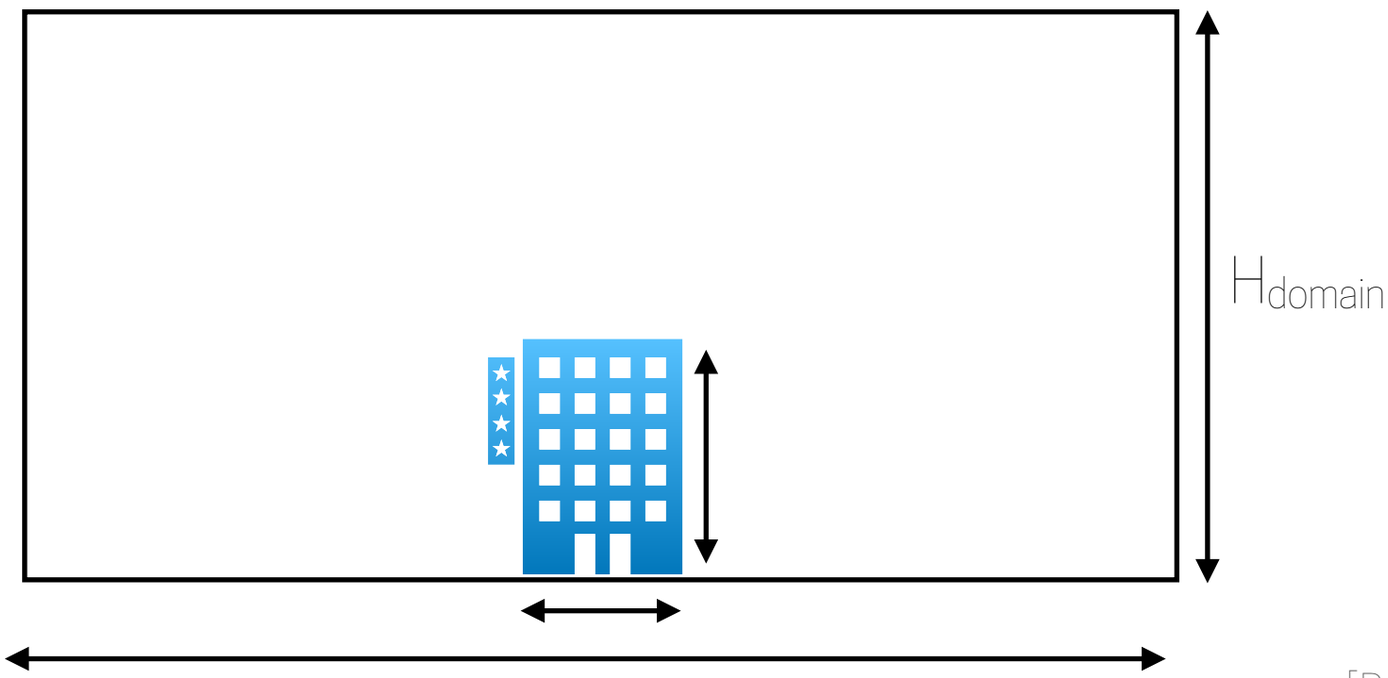
2. Maximum Blockage

$$BR = \frac{A_{building}}{A_{domain}} < 3 - 5\%$$



Directional Blockage

$$BR_L = \frac{L_{building}}{L_{domain}} \quad \&\& \quad BR_H = \frac{H_{building}}{H_{domain}} < 17 - 22\%$$



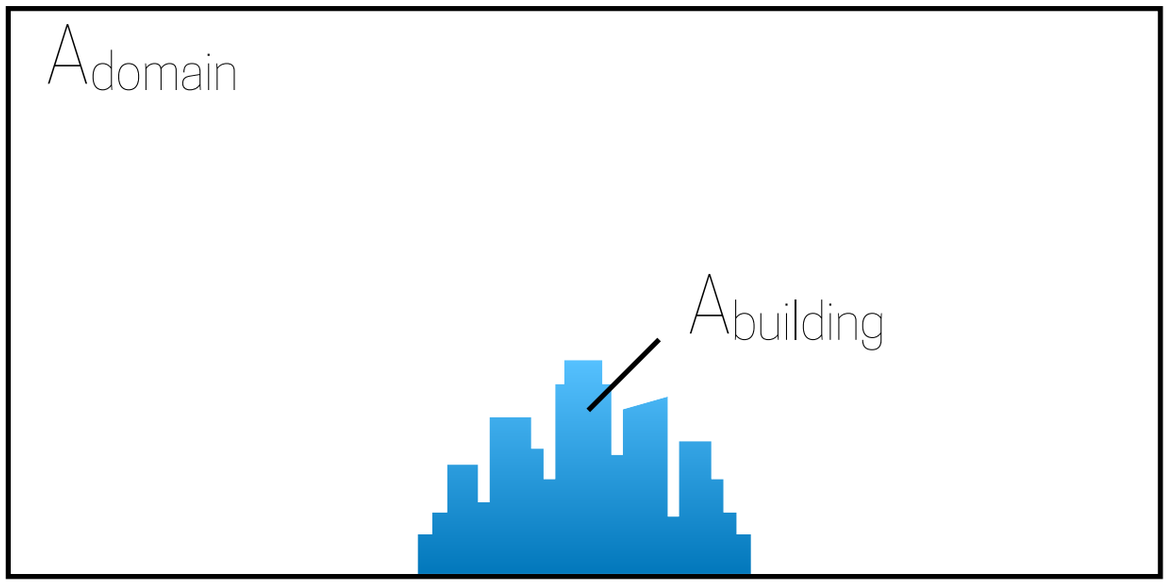
[Baetke, 1997, Blocken and Carmeliet, 2004]

Simulating urban neutral ABL flows

Computational domain

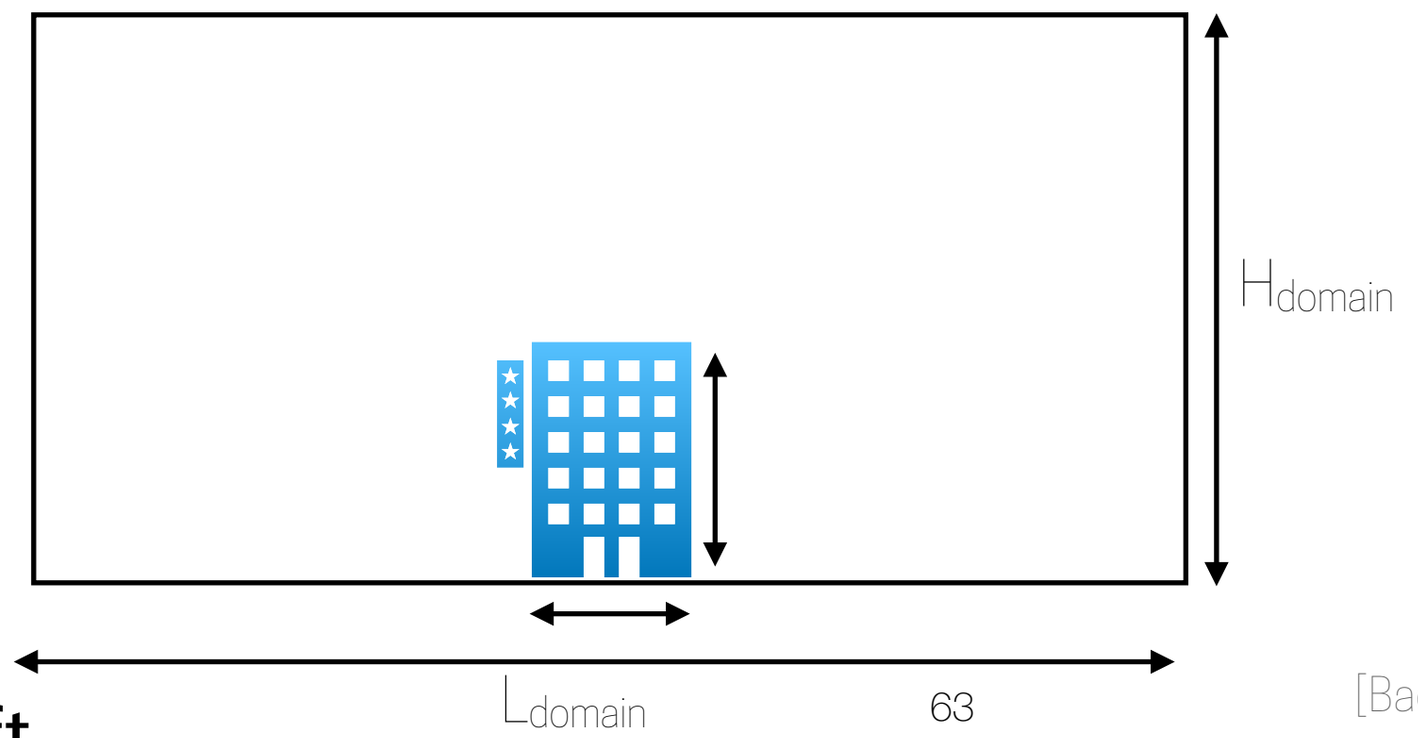
2. Maximum Blockage

$$BR = \frac{A_{building}}{A_{domain}} < 3 - 5\%$$



Directional Blockage

$$BR_L = \frac{L_{building}}{L_{domain}} \quad \&\& \quad BR_H = \frac{H_{building}}{H_{domain}} < 17 - 22\%$$



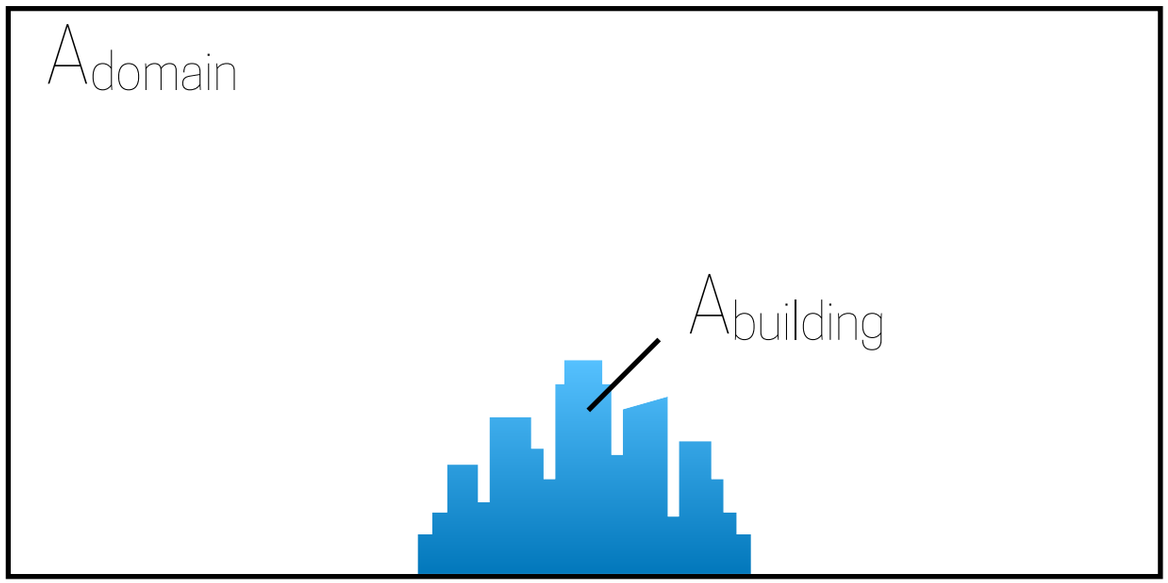
[Baetke, 1997, Blocken and Carmeliet, 2004]

Simulating urban neutral ABL flows

Computational domain

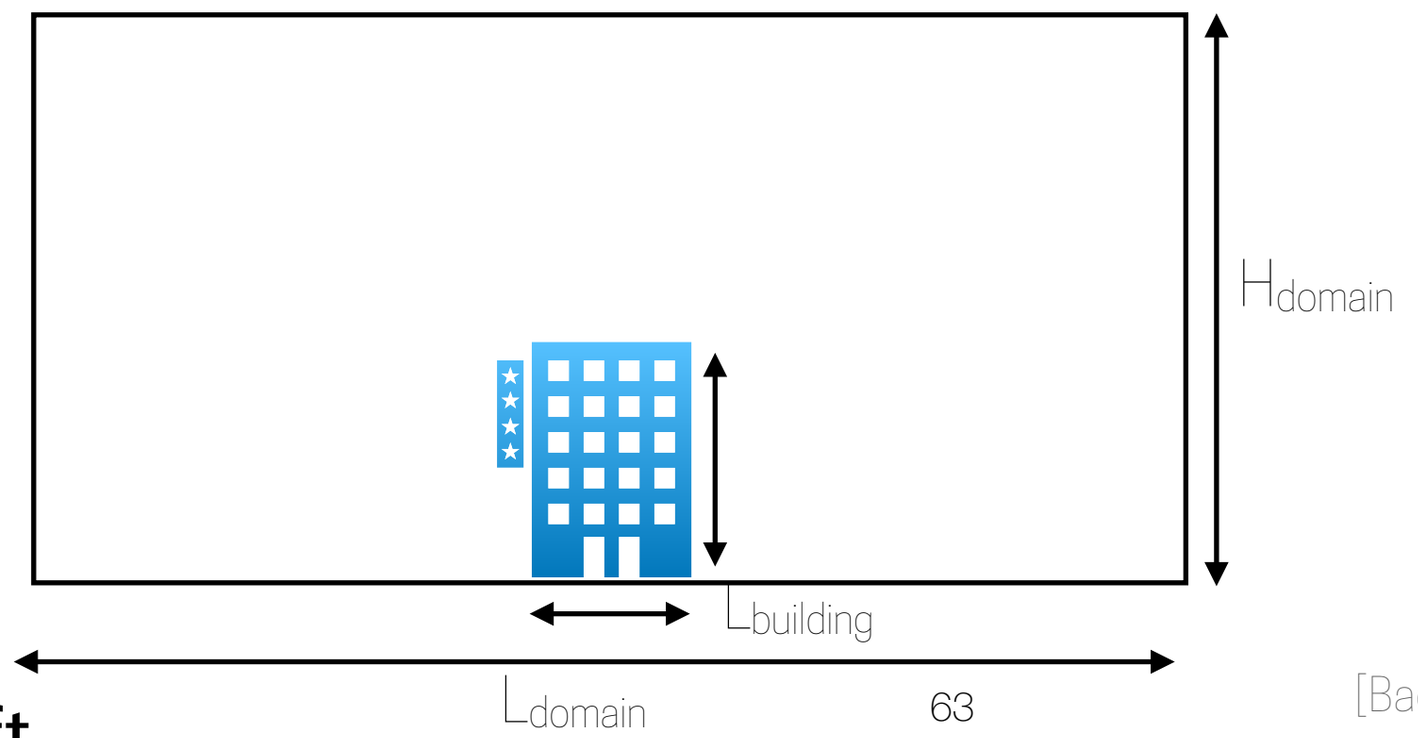
2. Maximum Blockage

$$BR = \frac{A_{building}}{A_{domain}} < 3 - 5\%$$



Directional Blockage

$$BR_L = \frac{L_{building}}{L_{domain}} \quad \&\& \quad BR_H = \frac{H_{building}}{H_{domain}} < 17 - 22\%$$



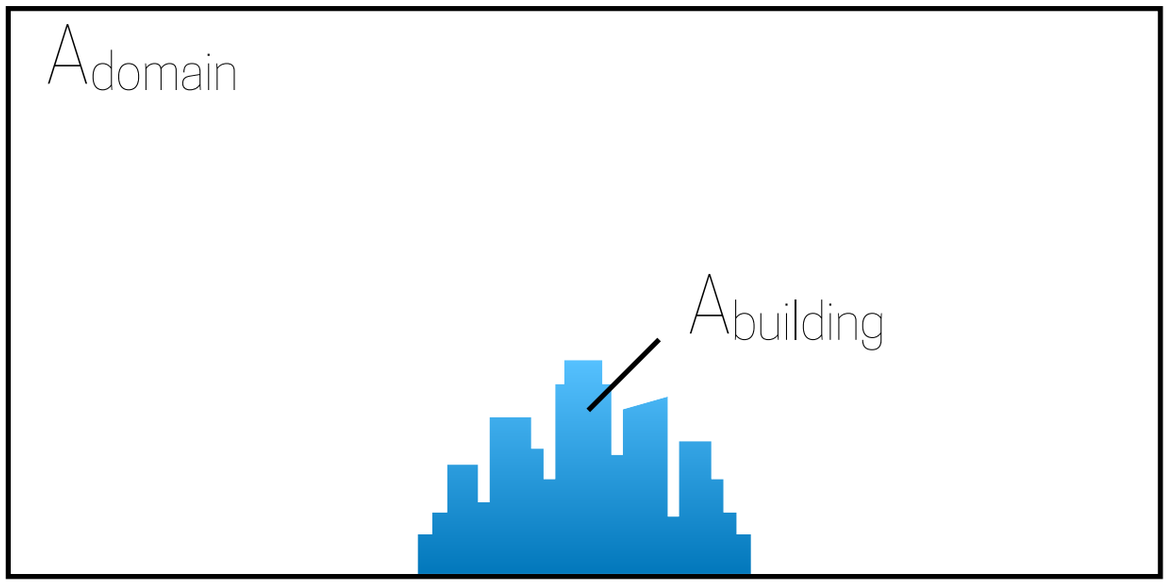
[Baetke, 1997, Blocken and Carmeliet, 2004]

Simulating urban neutral ABL flows

Computational domain

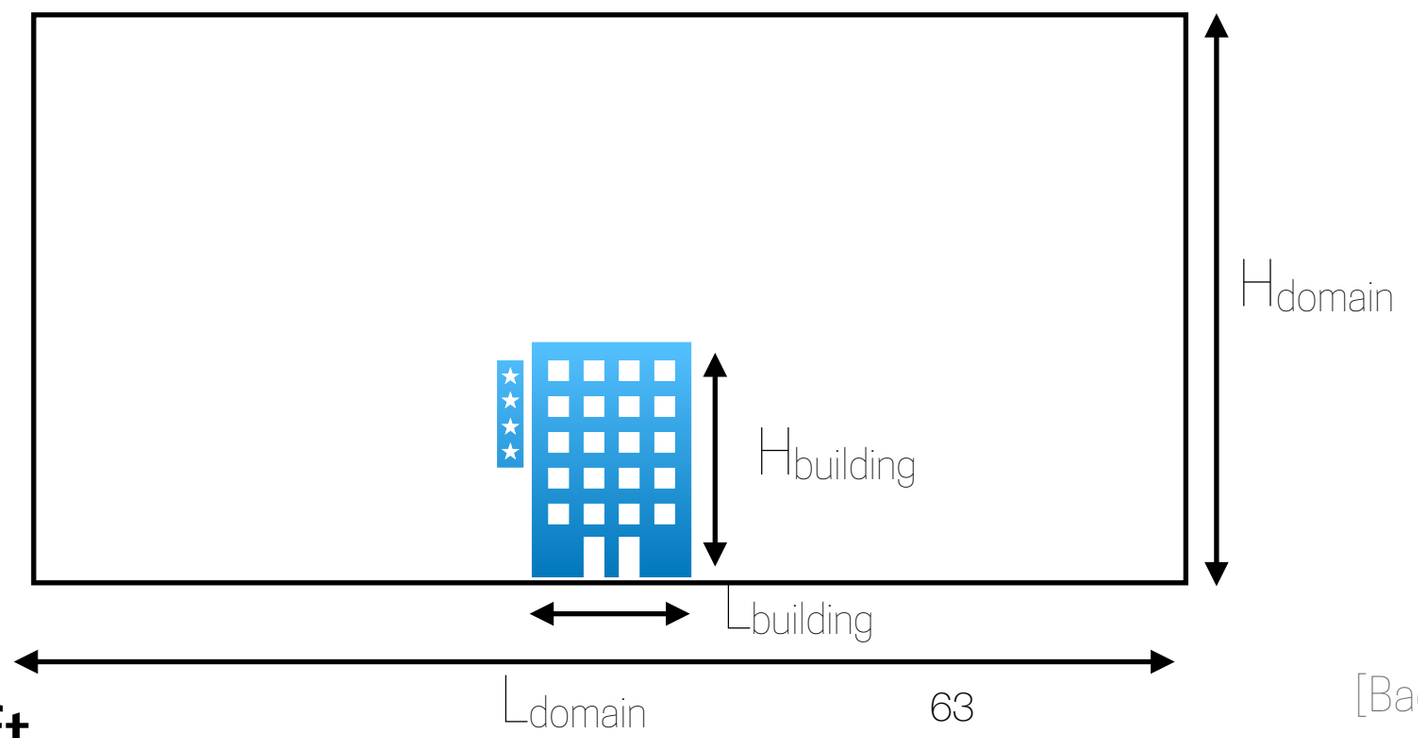
2. Maximum Blockage

$$BR = \frac{A_{building}}{A_{domain}} < 3 - 5\%$$



Directional Blockage

$$BR_L = \frac{L_{building}}{L_{domain}} \quad \&\& \quad BR_H = \frac{H_{building}}{H_{domain}} < 17 - 22\%$$

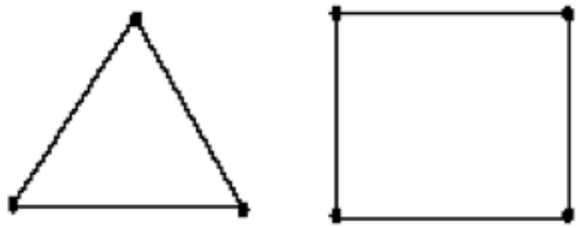


[Baetke, 1997, Blocken and Carmeliet, 2004]

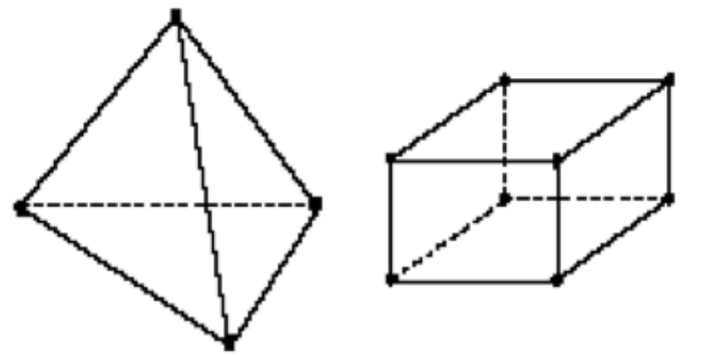
Simulating urban neutral ABL flows

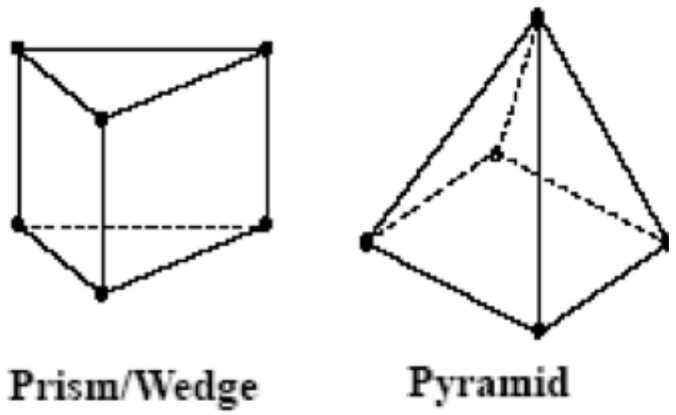
Mesh design

2D cell types



3D cell types

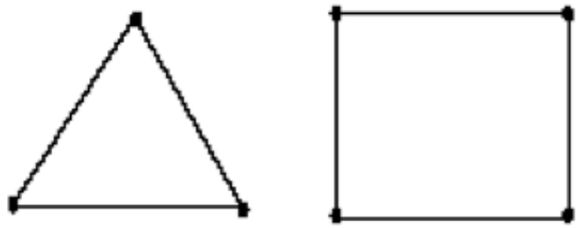




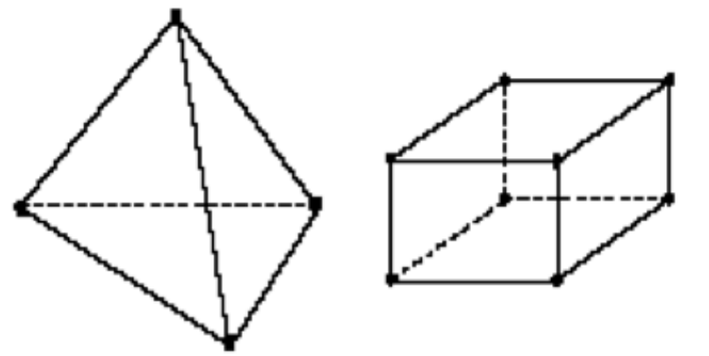
Simulating urban neutral ABL flows

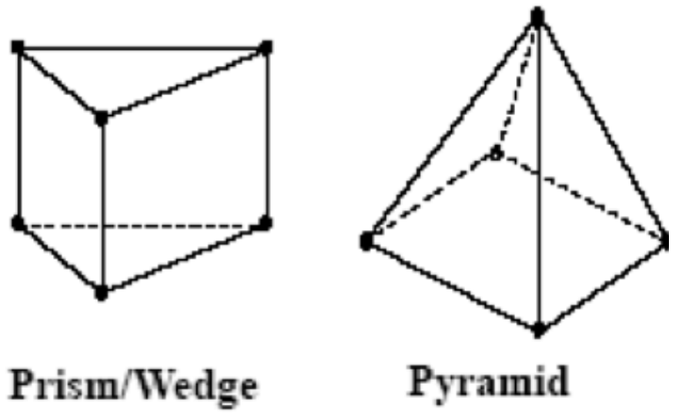
Mesh design

2D cell types



3D cell types



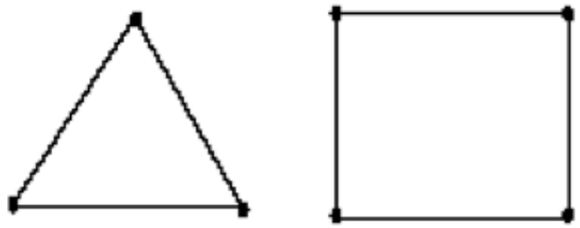


Mesh quality is very important, because it will affect convergency and the results

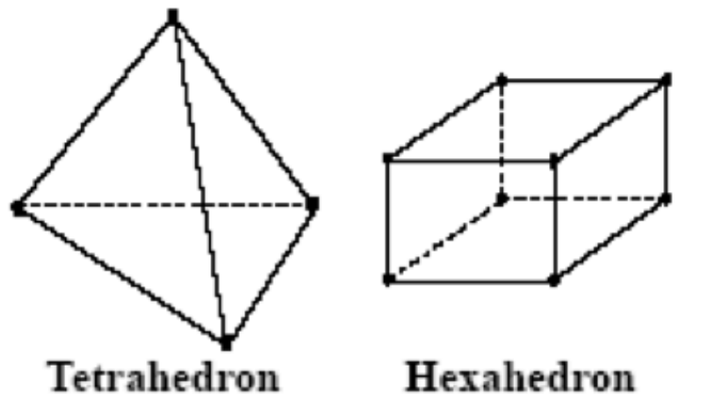
Simulating urban neutral ABL flows

Mesh design

2D cell types



3D cell types



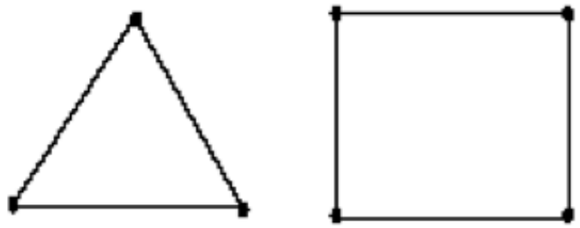
Mesh quality is very important, because it will affect convergency and the results

<u>Cell quality: aspect ratio & skewness are very important</u>

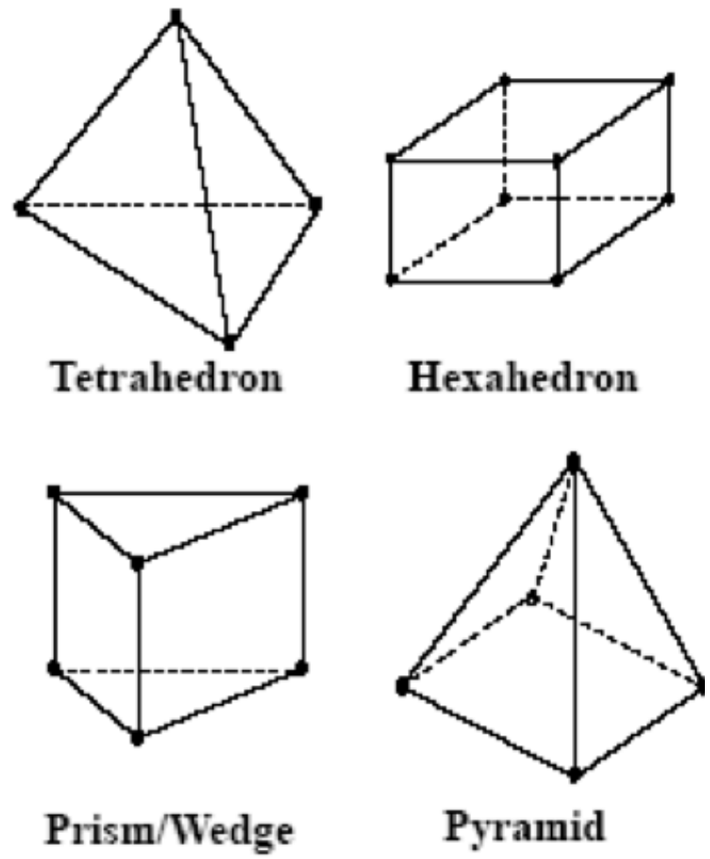
Simulating urban neutral ABL flows

Mesh design

2D cell types

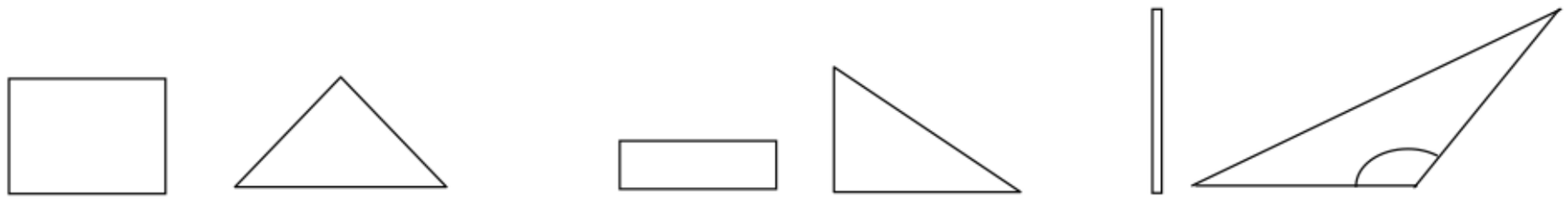


3D cell types



Mesh quality is very important, because it will affect convergency and the results

<u>Cell quality: aspect ratio & skewness are very important</u>



Simulating urban neutral ABL flows

Mesh design

Simulating urban neutral ABL flows

Mesh design

Grid resolution is essential:

Simulating urban neutral ABL flows

Mesh design

Grid resolution is essential:

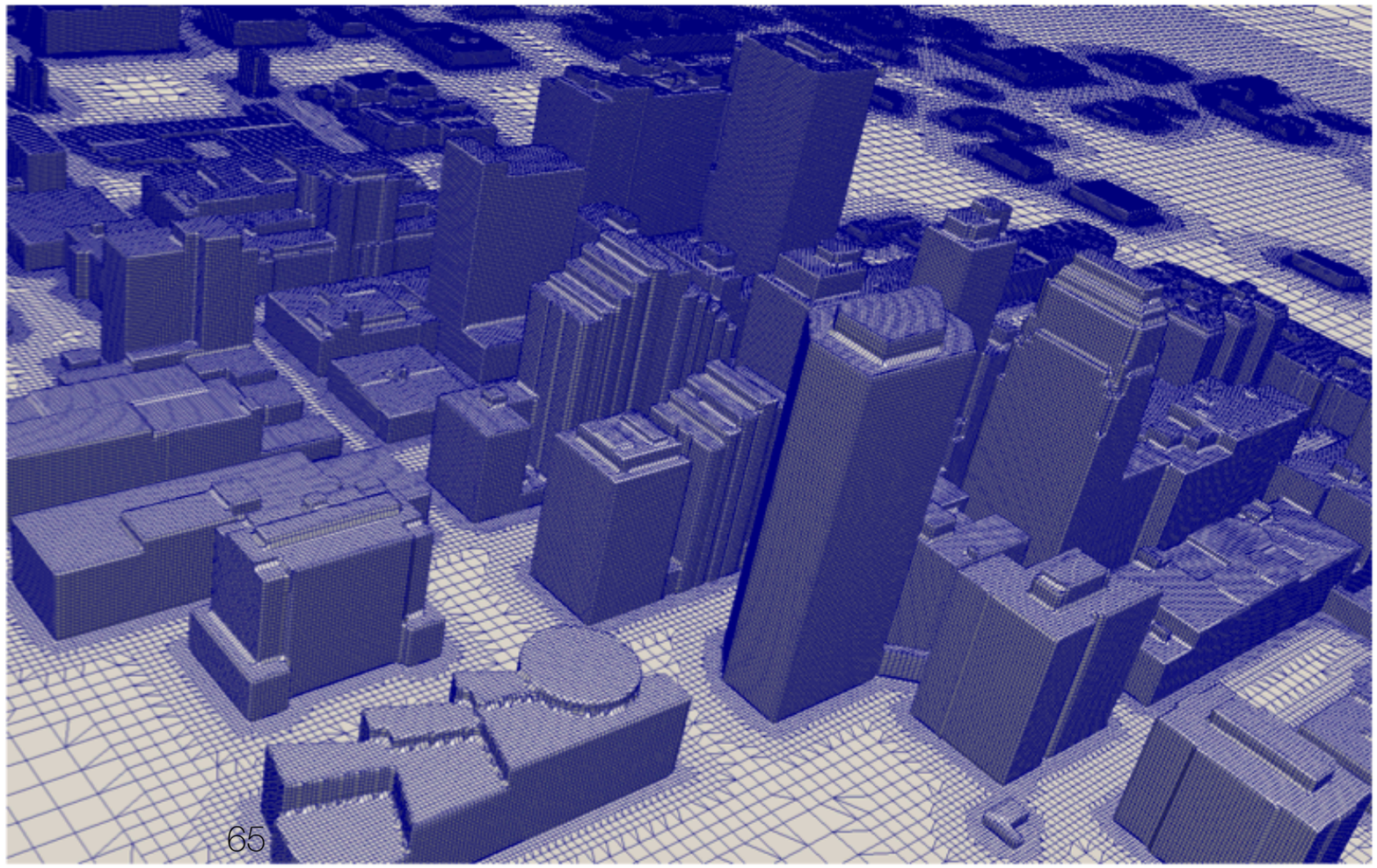
- Higher where large flow gradients are expected
- Higher in regions of interest
- Each building: at least 10 cells along length, width, and height
- Passages between buildings in regions of interest: at least 10 cells

Simulating urban neutral ABL flows

Mesh design

Grid resolution is essential:

- Higher where large flow gradients are expected
- Higher in regions of interest
- Each building: at least 10 cells along length, width, and height
- Passages between buildings in regions of interest: at least 10 cells



TU Delft

Simulating urban ABL flows

Mesh design

Simulating urban ABL flows

Mesh design

Grid convergence is fundamental when running CFD:

Simulating urban ABL flows

Mesh design

Grid convergence is fundamental when running CFD:

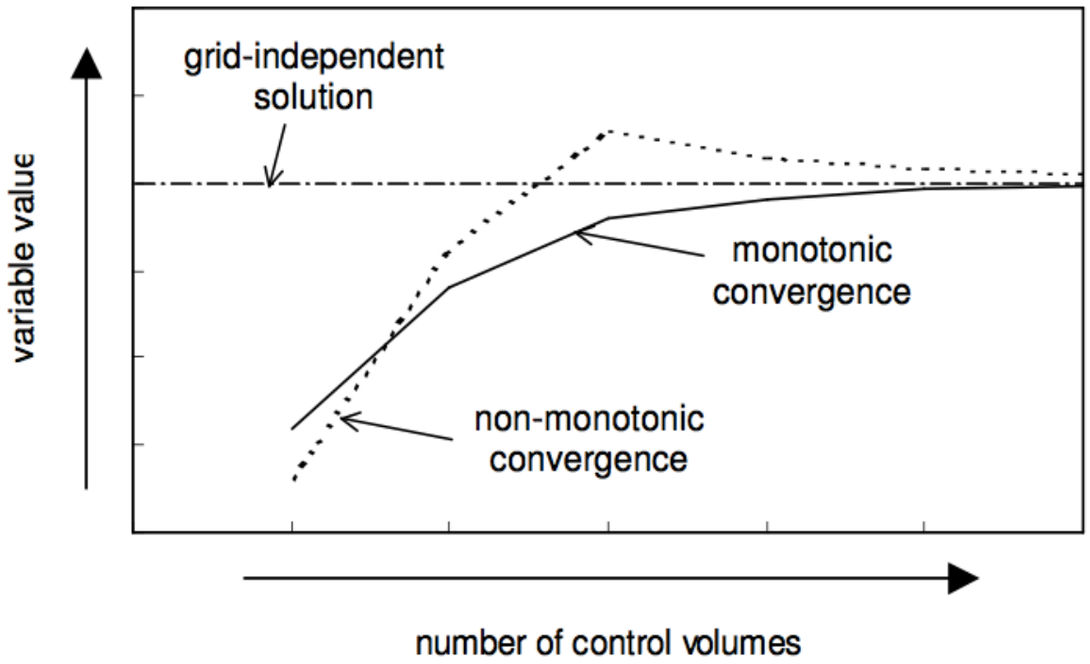
- Systematically refine or coarsen the grid by a constant factor (typically 2 or sqrt(2)), at least in the region of interest
- Try to obtain a grid-independent solution

Simulating urban ABL flows

Mesh design

Grid convergence is fundamental when running CFD:

- Systematically refine or coarsen the grid by a constant factor (typically 2 or sqrt(2)), at least in the region of interest
- Try to obtain a grid-independent solution

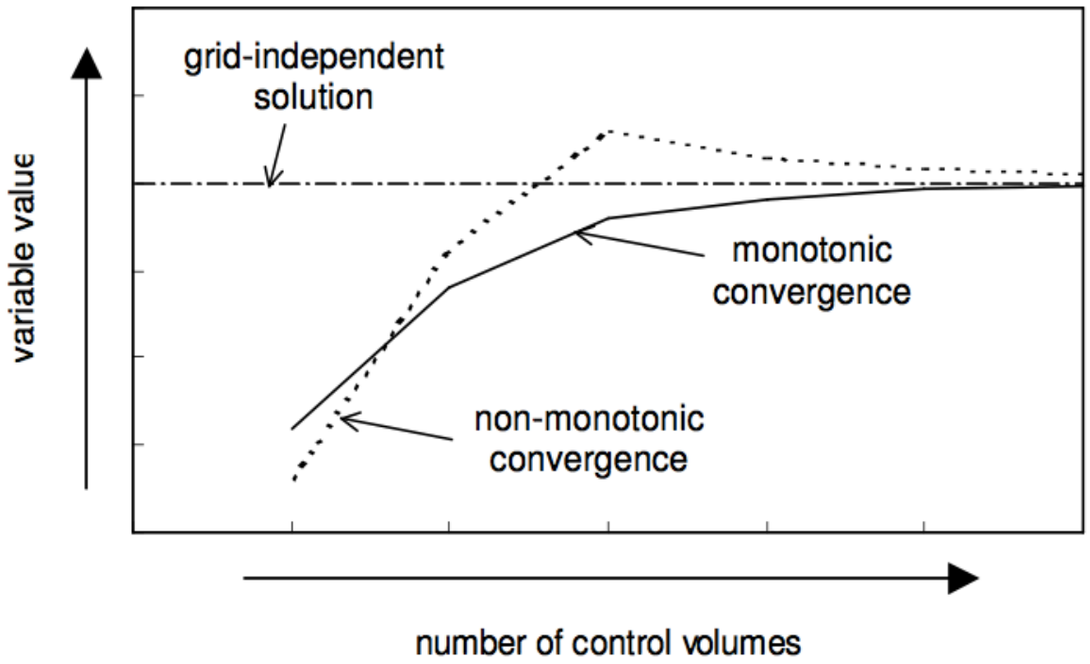


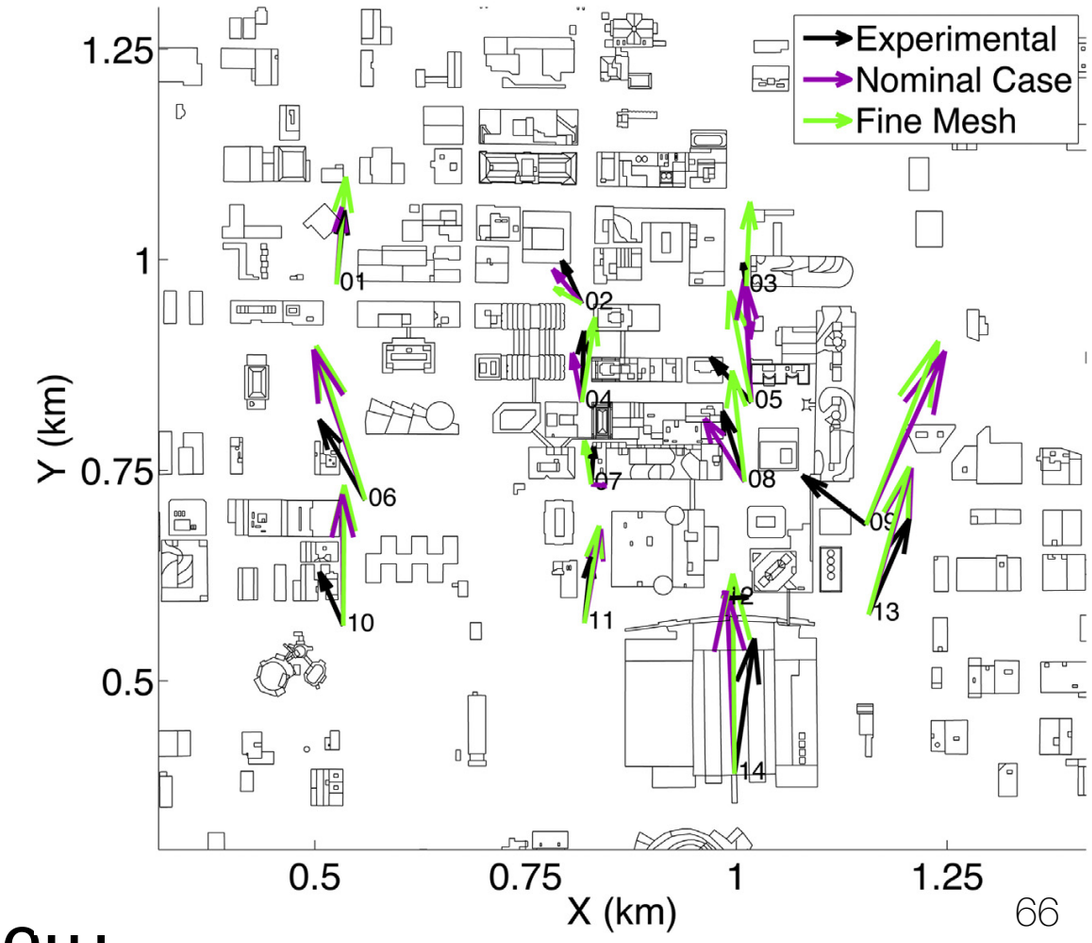
Simulating urban ABL flows

Mesh design

Grid convergence is fundamental when running CFD:

- Systematically refine or coarsen the grid by a constant factor (typically 2 or sqrt(2)), at least in the region of interest
- Try to obtain a grid-independent solution





Simulating urban neutral ABL flows

Physical models - equations

Assumptions

- Vertical velocity is zero
- Pressure is constant
- Shear stress is constant

- k and **ε** satisfy their conservation equations:

Simulating urban neutral ABL flows

Physical models - equations

Assumptions

- Vertical velocity is zero
- Pressure is constant
- Shear stress is constant

$$(\mu_l + \mu_t)\frac{dU}{dz} = \tau_0 = \rho u_*^2 \qquad \mu_t = \rho C_\mu \frac{k^2}{\epsilon}$$

- k and **ε** satisfy their conservation equations:

Simulating urban neutral ABL flows

Physical models - equations

Assumptions

- Vertical velocity is zero
- Pressure is constant
- Shear stress is constant

$$(\mu_l + \mu_t)\frac{dU}{dz} = \tau_0 = \rho u_*^2 \qquad \mu_t = \rho C_\mu \frac{k^2}{\epsilon}$$

- k and **ε** satisfy their conservation equations:

$$\frac{\partial}{\partial z}\left(\frac{\mu_t}{\sigma_k}\frac{\partial k}{\partial z}\right) + \mu_t\left(\frac{\partial U}{\partial z}\right)^2 - \rho\epsilon = 0$$

$$\frac{\partial}{\partial z}\left(\frac{\mu_t}{\sigma_\epsilon}\frac{\partial \epsilon}{\partial z}\right) + C_1\mu_t\left(\frac{\partial U}{\partial z}\right)^2\frac{\epsilon}{k} - C_2\rho\frac{\epsilon^2}{k} = 0$$

Simulating urban neutral ABL flows

Boundary and Initial Conditions - Inflow

Using the previous assumptions, the following initial and inflow profiles are derived:

Simulating urban neutral ABL flows

Boundary and Initial Conditions - Inflow

Using the previous assumptions, the following initial and inflow profiles are derived:

$$U(z) = \frac{u_*}{\kappa} log\left(\frac{z + z_0}{z_0}\right)$$

Simulating urban neutral ABL flows

Boundary and Initial Conditions - Inflow

Using the previous assumptions, the following initial and inflow profiles are derived:

$$U(z) = \frac{u_*}{\kappa} log\left(\frac{z + z_0}{z_0}\right)$$

$$k(z) = \frac{u_*^2}{\sqrt{C_\mu}}$$

Simulating urban neutral ABL flows

Boundary and Initial Conditions - Inflow

Using the previous assumptions, the following initial and inflow profiles are derived:

$$U(z) = \frac{u_*}{\kappa} log\left(\frac{z + z_0}{z_0}\right)$$

$$k(z) = \frac{u_*^2}{\sqrt{C_\mu}}$$

$$\varepsilon(z) = \frac{u_*^3}{\kappa\left(z + z_0\right)}$$

Simulating urban neutral ABL flows

Boundary and Initial Conditions - Top & sides

Simulating urban neutral ABL flows

Boundary and Initial Conditions - Top & sides

The top boundary condition is very important for sustaining equilibrium boundary layer profiles. Two options:

1. Therefore the prescription of a constant shear stress at the top, corresponding to the inflow profiles, is recommended to prevent a horizontal change from the inflow profiles.
2. Another option is to prescribe the values for the velocities and the turbulence quantities of the inflow profile at the height of the top boundary over the entire top boundary (Blocken et al., 2006).

Simulating urban neutral ABL flows

Boundary and Initial Conditions - Top & sides

The top boundary condition is very important for sustaining equilibrium boundary layer profiles. Two options:

1. Therefore the prescription of a constant shear stress at the top, corresponding to the inflow profiles, is recommended to prevent a horizontal change from the inflow profiles.
2. Another option is to prescribe the values for the velocities and the turbulence quantities of the inflow profile at the height of the top boundary over the entire top boundary (Blocken et al., 2006).

At lateral boundaries symmetry boundary conditions are frequently used when the approach flow direction is parallel to them. As symmetry boundary conditions enforce a parallel flow by requiring a vanishing normal velocity component at the boundary, the boundary should be positioned far enough away from the built area of interest to not lead to an artificial acceleration of the flow in the region of interest

In case different wind directions are to be simulated with the same computational domain, then the lateral boundaries become inflow and outflow boundaries with corresponding boundary conditions

Simulating urban neutral ABL flows

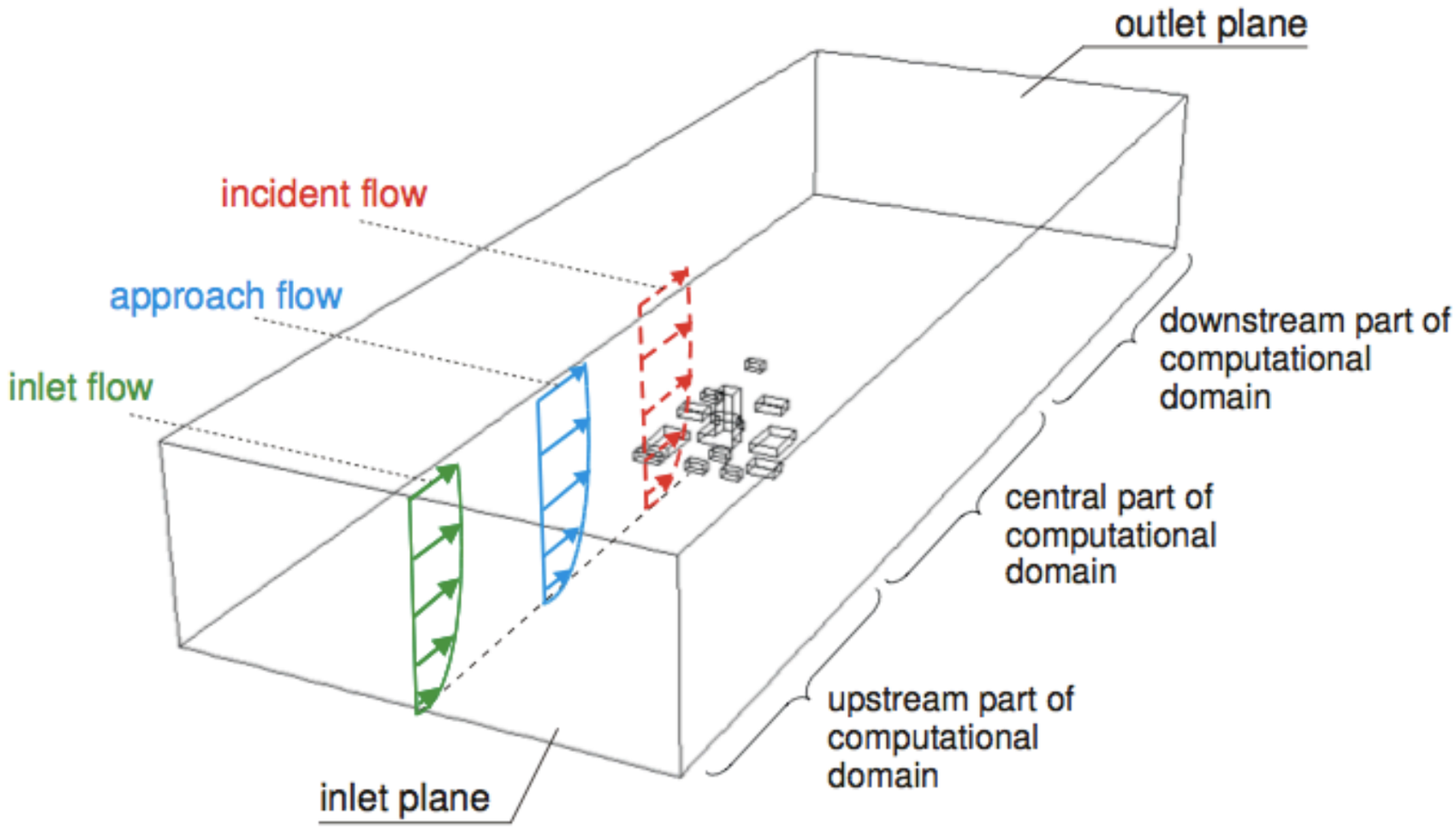
Boundary and Initial Conditions - Outflow

- At the boundary behind the obstacles (where all or most of the fluid leaves the computational domain), open boundary conditions are used in commercial CFD and microscale obstacle-accommodating meteorology models.
- The open boundary conditions in commercial CFD codes are either outflow or constant static pressure boundary conditions.
- With an outflow boundary condition the derivatives of all flow variables are forced to vanish, corresponding to a fully developed flow. Therefore this boundary should be ideally far enough away from the built area to not have any fluid entering into the computational domain through this boundary.
- This also holds for the use of a constant static pressure at the outflow boundary, where then only the derivatives of all other flow variables are forced to vanish.

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall - Horizontal Inhomogeneity

Important problem for CFD studies of the ABL in wind engineering (Blocken et al. 2007)

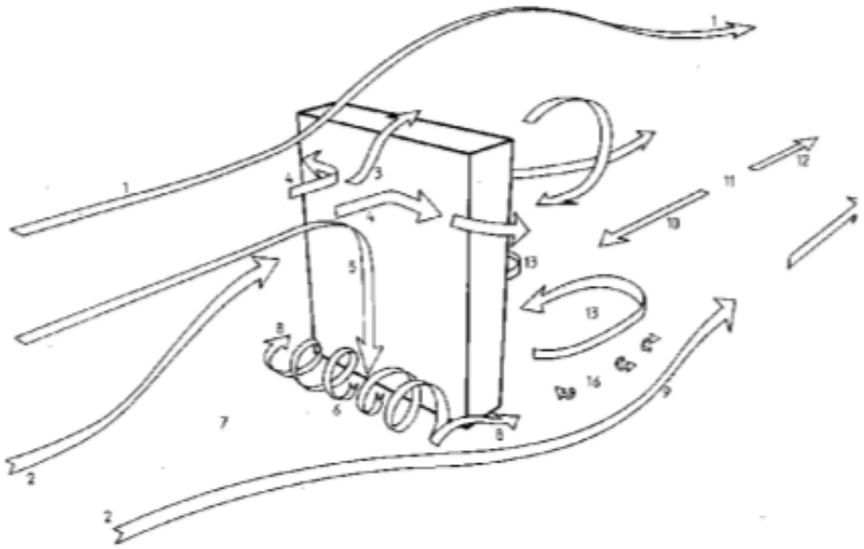


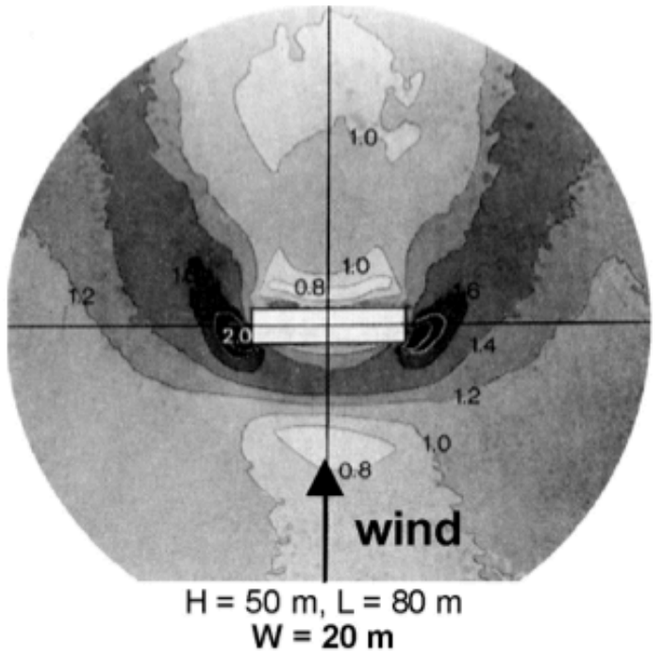
Blocken B, Stathopoulos T, Carmeliet J. 2007. CFD simulation of the Atmospheric Boundary Layer – wall function problems. ***Atmospheric Environment*** 41(2): 238-252.

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall - Horizontal Inhomogeneity

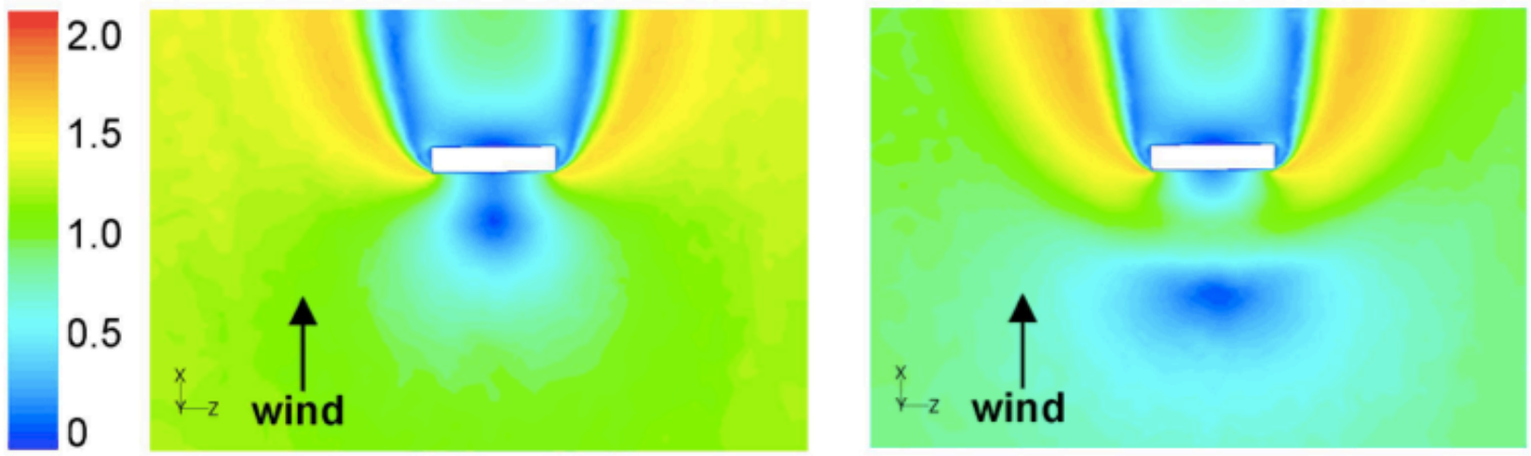
Can lead to unexpected results:





without correction

with correction



Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

Horizontal inhomogeneity can be fixed through the use of consistent wall functions.

[http://brennen.caltech.edu/fluidbook/basicfluiddynamics/turbulence/lawofthewall.pdf]

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

Horizontal inhomogeneity can be fixed through the use of consistent wall functions.

With commercial codes two options exist:

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)
2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

[http://brennen.caltech.edu/fluidbook/basicfluiddynamics/turbulence/lawofthewall.pdf]

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

Horizontal inhomogeneity can be fixed through the use of consistent wall functions.

With commercial codes two options exist:

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)
2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

What is a wall function?

[http://brennen.caltech.edu/fluidbook/basicfluiddynamics/turbulence/lawofthewall.pdf]

Simulating urban neutral ABL flows

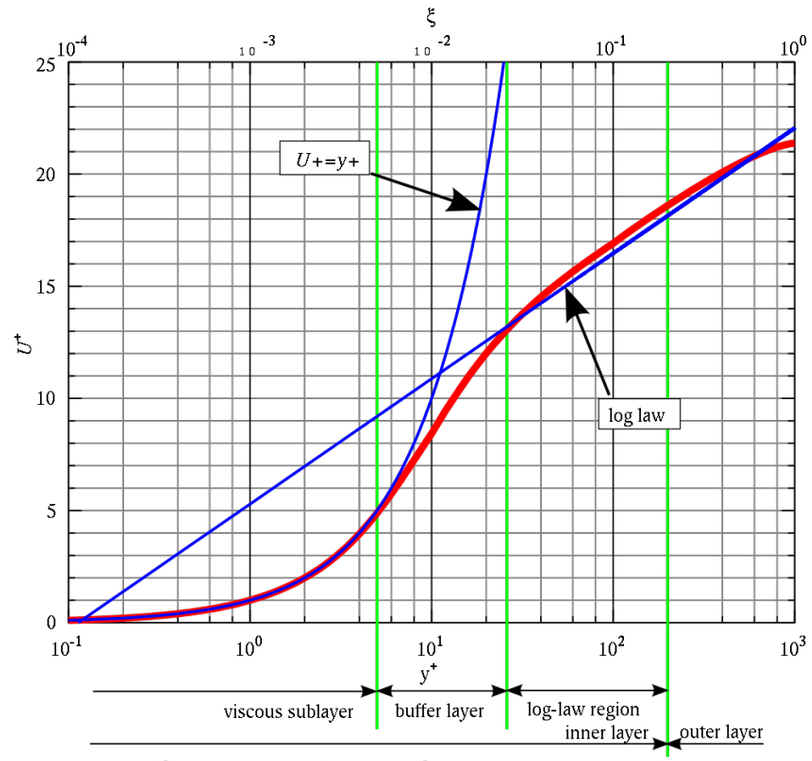
Boundary and Initial Conditions - wall functions

Horizontal inhomogeneity can be fixed through the use of consistent wall functions.

With commercial codes two options exist:

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)
2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

What is a wall function?



[http://brennen.caltech.edu/fluidbook/basicfluiddynamics/turbulence/lawofthewall.pdf]

Simulating urban neutral ABL flows

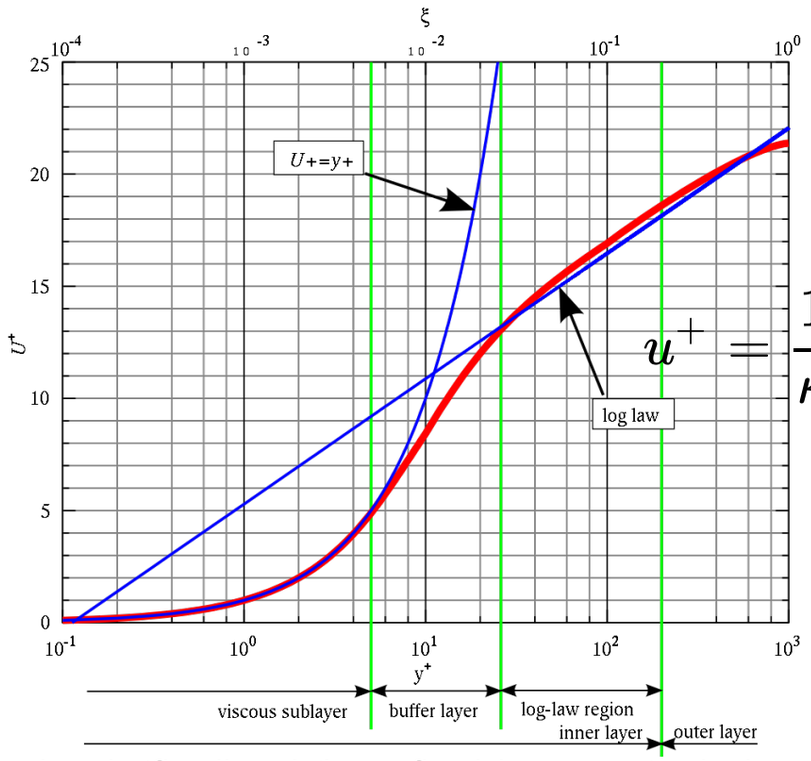
Boundary and Initial Conditions - wall functions

Horizontal inhomogeneity can be fixed through the use of consistent wall functions.

With commercial codes two options exist:

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)
2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

What is a wall function?



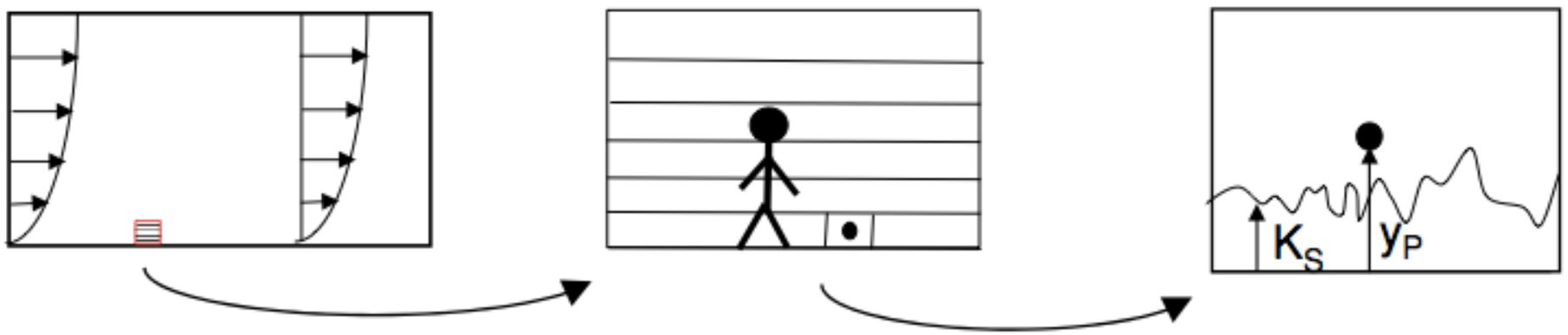
$$u^+ = \frac{1}{\kappa} ln y^+ + C^+$$

[http://brennen.caltech.edu/fluidbook/basicfluiddynamics/turbulence/lawofthewall.pdf]

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

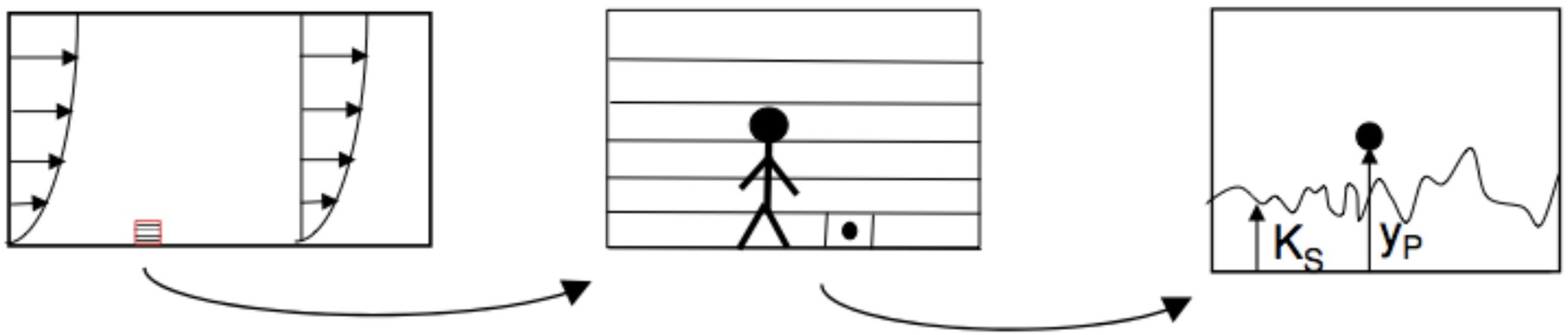


Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

Usually impossible to satisfy all requirements for modelling ABL flow:



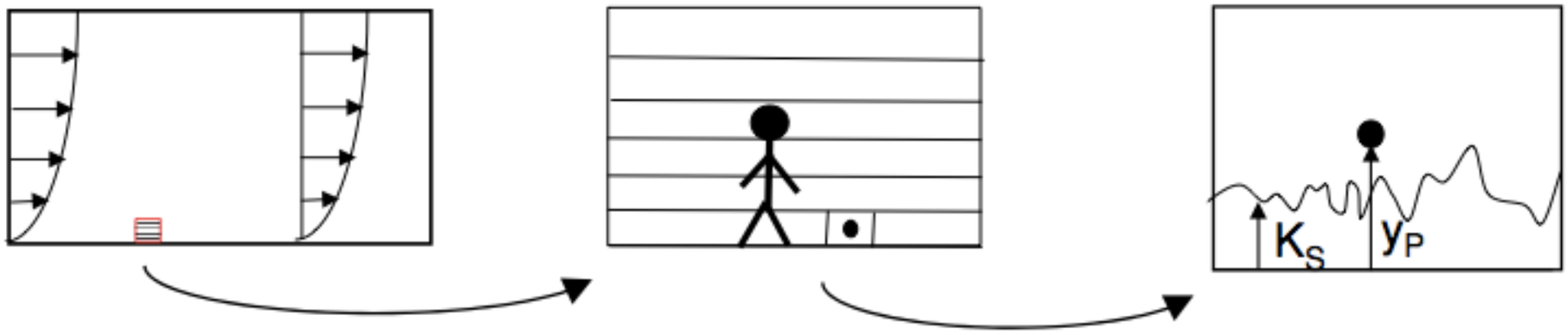
Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

Usually impossible to satisfy all requirements for modelling ABL flow:

- High mesh resolution in the vertical direction close to the ground: 3-4 cells below 1.5m (COST732)
- Distance from the center of the wall-adjacent cell to the wall larger than the physical roughness height or "equivalent sand-grain roughness" ($y_p > k_s$)
- Avoid unintended streamwise gradients by using the correct relationship between k_s and y_p



Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

Solution:

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

Solution:

1. Determine y_0
2. Select y_p (as large as possible, but with sufficient resolution)
3. Determine $k_{s,ABL}$ with the above equation for C_s=0.5
4. If $k_{s,ABL} < y_p$, set $k_{s,ABL} = y_p$
5. Determine updated C_s from the above equation

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

Solution:

$$k_{S,ABL} = \frac{9.793\, y_0}{C_s}$$

1. Determine y_0
2. Select y_p (as large as possible, but with sufficient resolution)
3. Determine $k_{s,ABL}$ with the above equation for C_s=0.5
4. If $k_{s,ABL} < y_p$, set $k_{s,ABL} = y_p$
5. Determine updated C_s from the above equation

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

Davenport/Wieringa aerodynamic roughness length (y_0,z_0), 1992

	y_0 (m)	Landscape description
1	0.0002 Sea	Open sea or lake (irrespective of the wave size), tidal flat, snow-covered flat plain, featureless desert, tarmac, concrete, with a free fetch of several kilometres.
2	0.005 Smooth	Featureless land surface without any noticeable obstacles and with negligible vegetation; e.g. beaches, pack ice without large ridges, morass, and snow-covered or fallow open country.
3	0.03 Open	Level country with low vegetation (e.g. grass) and isolated obstacles with separations of at least 50 obstacle heights; e.g. grazing land without windbreaks, heather, moor and tundra, runway area of airports.
4	0.10 Roughly open	Cultivated area with regular cover of low crops, or moderately open country with occasional obstacles (e.g. low hedges, single rows of trees, isolated farms) at relative horizontal distances of at least 20 obstacle heights.
5	0.25 Rough	Recently-developed "young" landscape with high crops or crops of varying height, and scattered obstacles (e.g. dense shelterbelts, vineyards) at relative distances of about 15 obstacle heights.
6	0.50 Very rough	"Old" cultivated landscape with many rather large obstacle groups (large farms, clumps of forest) separated by open spaces of about 10 obstacle heights. Also low large vegetation with small interspaces such as bush land, orchards, young densely-planted forest.
7	1.0 Closed	Landscape totally and quite regularly covered with similar-size large obstacles, with open spaces comparable to the obstacle heights; e.g. mature regular forests, homogeneous cities or villages.
8	≥ 2.0 Chaotic	Centres of large towns with mixture of low-rise and high-rise buildings. Also irregular large forests with many clearings.

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

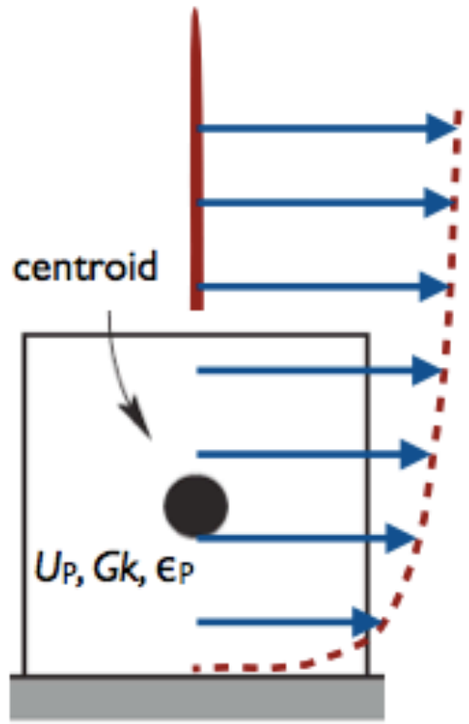
2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

Richards and Hoxey (1993)

$$U_p = \frac{u_*}{\kappa} ln\left(\frac{z + z_0}{z_0}\right)$$

$$\epsilon_p = \frac{C_\mu^{0.75} k^{1.5}}{\kappa (z + z_0)}$$

$$G_k = \frac{\tau_w^2}{\rho \kappa C_\mu^{0.25} k^{0.5} (z + z_0)}$$



Implementation
(Fluent, OpenFOAM, Fine Open)

$$\frac{U_p}{u_*} = \frac{1}{\kappa} ln\left(\widetilde{E}\widetilde{y}^+\right)$$

Smooth wall

$$\widetilde{y}^+ = y^+ \qquad \widetilde{E} = E$$

Rough wall

$$\widetilde{y}^+ = \frac{u_* (z + z_0)}{\nu} \qquad \widetilde{E} = \frac{\nu}{z_0 u_*}$$

Switch based on local surface properties

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

$$U = \frac{u_*}{\kappa} ln\left(\frac{z+z_0}{z_0}\right) \qquad k = \frac{u_*^2}{\sqrt{C_\mu}} \qquad \epsilon = \frac{u_*^3}{\kappa\left(z+z_0\right)}$$

$$\sigma_\epsilon = \frac{\kappa^2}{\left(C_{\epsilon 2} - C_{\epsilon 1}\right)\sqrt{C_\mu}} \quad \text{or} \quad S_\epsilon\left(z\right) = \frac{\rho u_*^4}{\left(z+z_0\right)^2}\left(\frac{\left(C_{\epsilon 2} - C_{\epsilon 1}\right)\sqrt{C_\mu}}{\kappa^2} - \frac{1}{\sigma_\epsilon}\right)$$

Simulating urban neutral ABL flows

Boundary and Initial Conditions - wall functions

2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

$$U = \frac{u_*}{\kappa} ln\left(\frac{z+z_0}{z_0}\right) \qquad k = \frac{u_*^2}{\sqrt{C_\mu}} \qquad \epsilon = \frac{u_*^3}{\kappa\left(z+z_0\right)}$$

$$\sigma_\epsilon = \frac{\kappa^2}{\left(C_{\epsilon 2} - C_{\epsilon 1}\right)\sqrt{C_\mu}} \quad \text{or} \quad S_\epsilon\left(z\right) = \frac{\rho u_*^4}{\left(z+z_0\right)^2}\left(\frac{\left(C_{\epsilon 2} - C_{\epsilon 1}\right)\sqrt{C_\mu}}{\kappa^2} - \frac{1}{\sigma_\epsilon}\right)$$

Some issues with this approach:

1. Default value of C_{μ}=0.09 often results in too low TKE compared to experimental data —> we can modify it's value to adjust it to k; adjust $\boldsymbol{\sigma_\varepsilon}$ or introduce source term S_ε accordingly
2. k profile can vary with height —> modify k and **ε** solutions

Simulating urban neutral ABL flows

Discretization and solution methods

Discretization

Second order schemes for all terms →

- Momentum equations
- Pressure interpolation
- Transport equation of k and **ε**

Simulating urban neutral ABL flows

Discretization and solution methods

Discretization

Second order schemes for all terms →

- Momentum equations
- Pressure interpolation
- Transport equation of k and **ε**

Solution

- Residuals need to be monitored to measure convergence
- Residuals level should drop 1-3 orders of magnitude
- Check if the quantity of interest (QoI), in example velocity, is not changing anymore after each iteration

Questions so far?

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

8. Properties of Numerical Solution Methods

9. Simulating Urban Neutral ABL flows

10. Wrap-up

Wrap up:

1. Understand the fundamentals of theory behind CFD.

2. Understand the best practice guidelines in urban flows.

Next class:

Learn about the flow governing equations

Outline of the class:

1. Introduction

2. Mathematical Model

3. Mesh design and generation

4. Discretization

5. Algebraic form of equations

6. Iterative methods

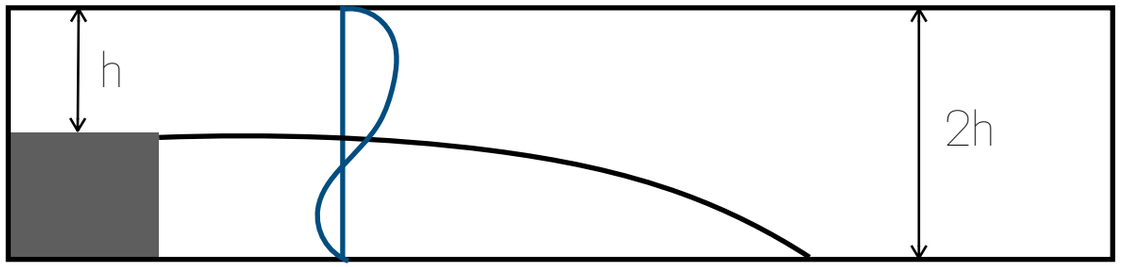
8. Properties of Numerical Solution Methods

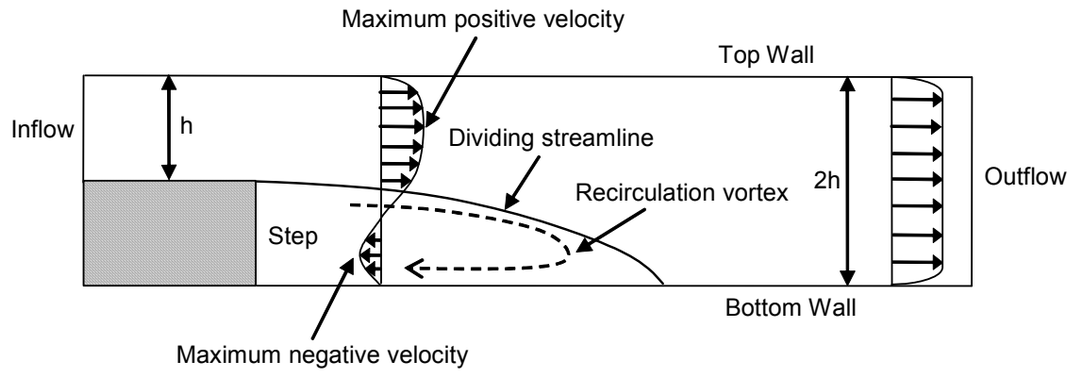
9. Simulating Urban Neutral ABL flows

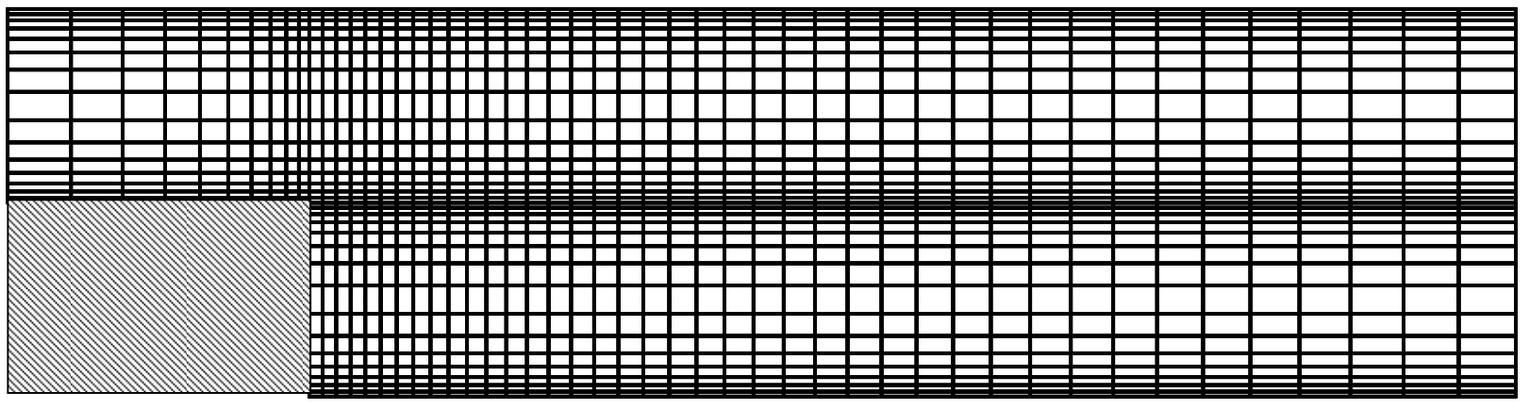
10. Wrap-up

Mesh design and generation

Importance of mesh spacing



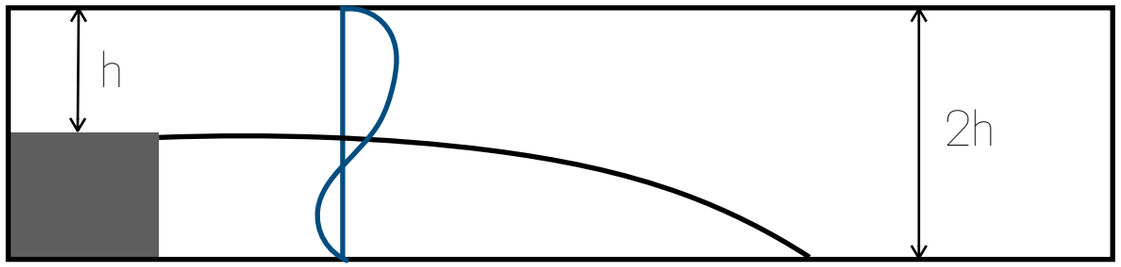


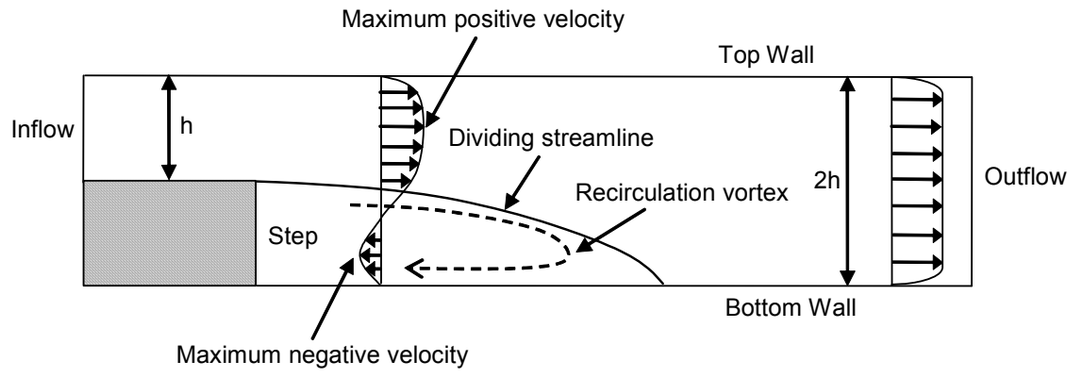


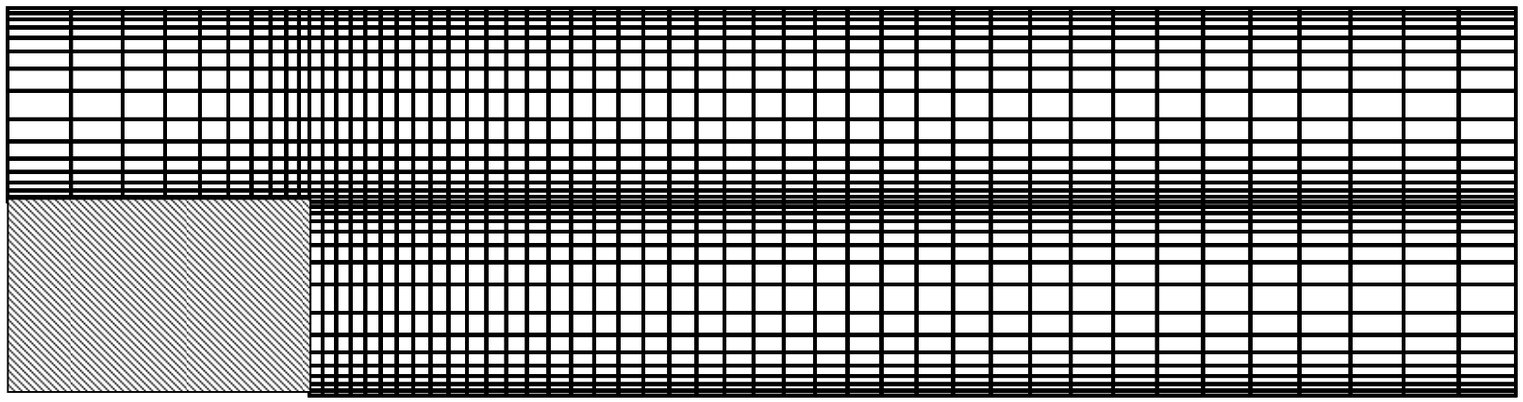
Mesh design and generation

Importance of mesh spacing

Mesh spacing is fundamental to capture flow features:

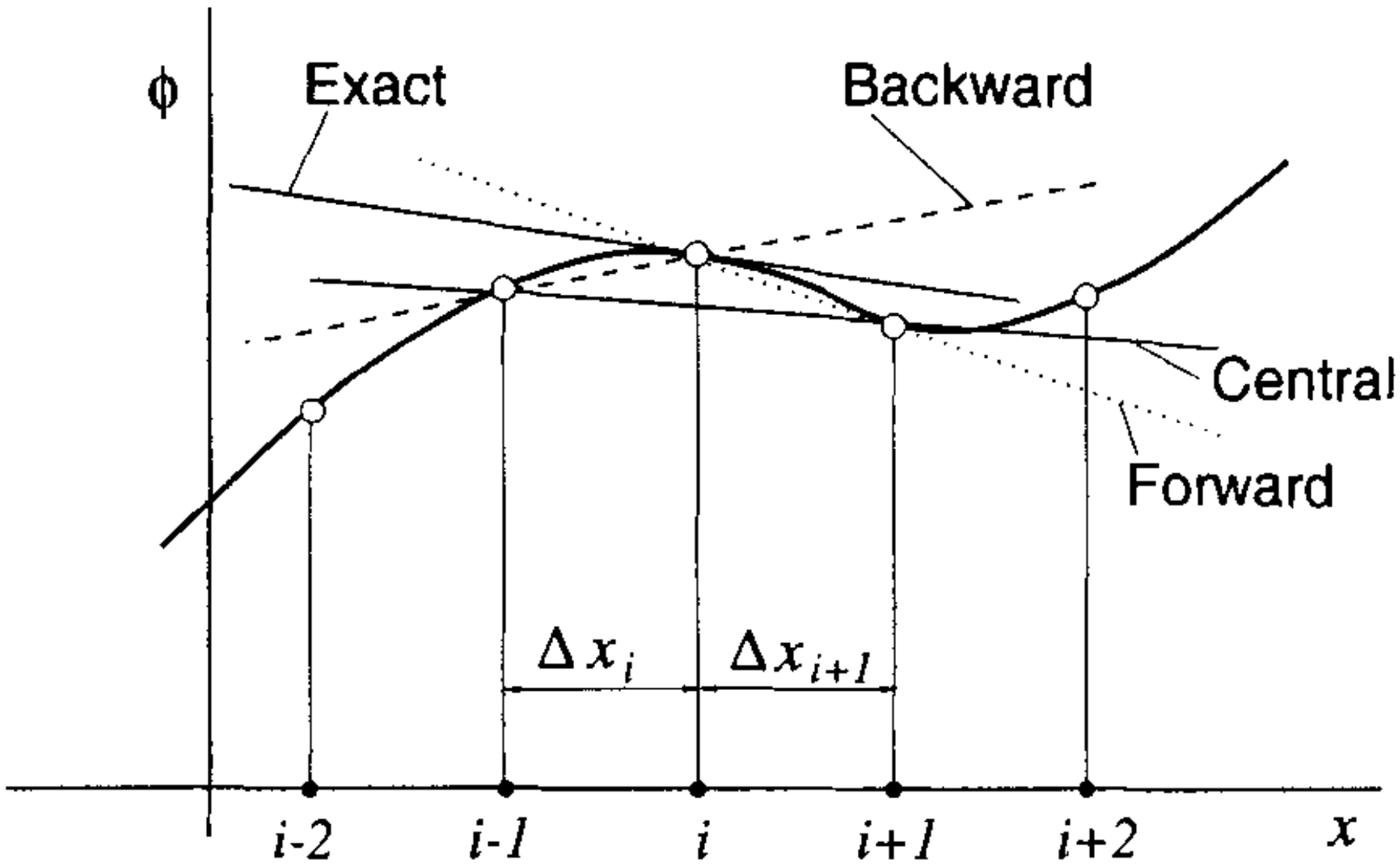






Components of a Numerical Solution Method:

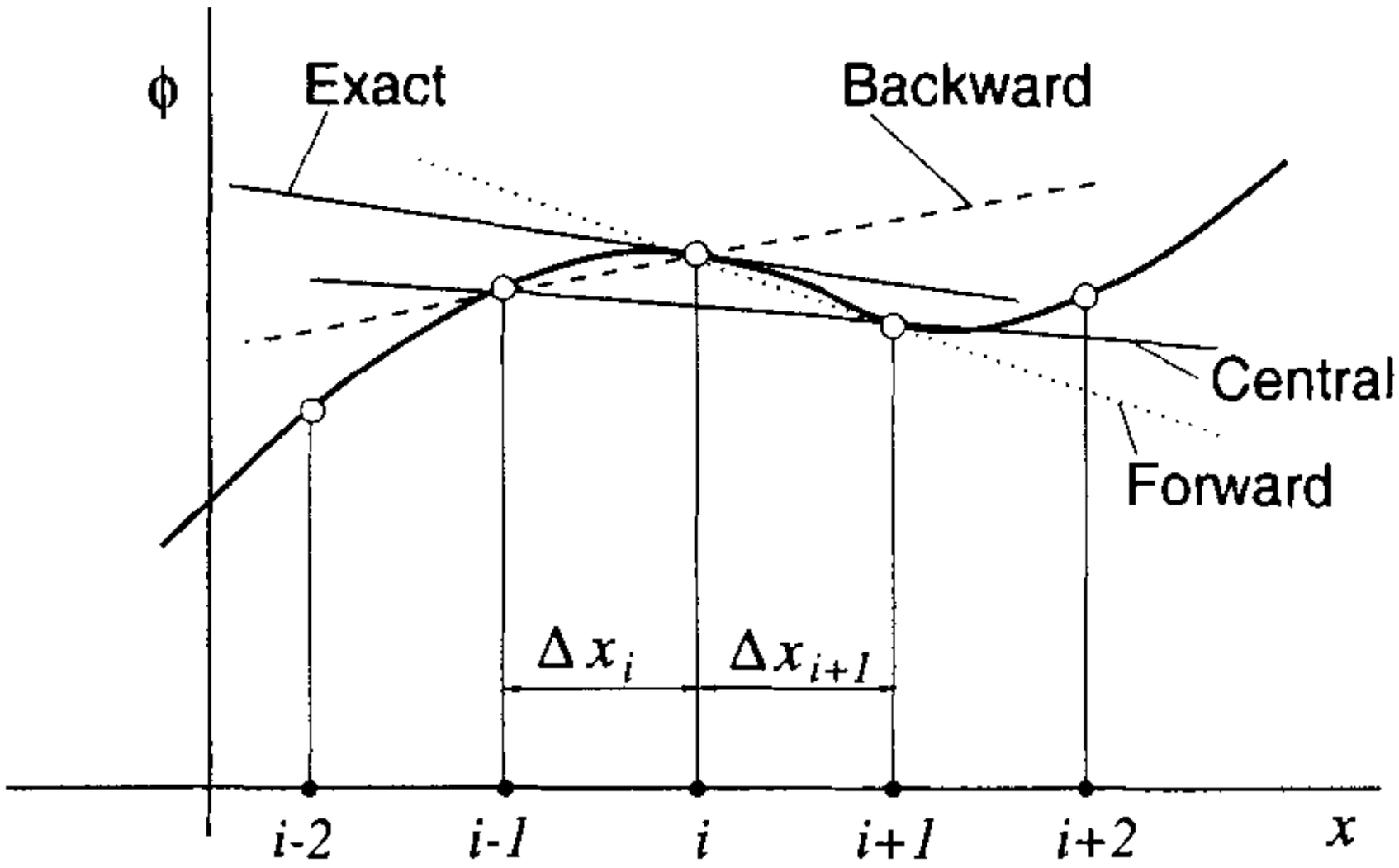
Finite approximations



The choice influences the accuracy of the approximation

Components of a Numerical Solution Method:

Finite approximations



$$\left(\frac{\partial \phi}{\partial x}\right)_i \approx \frac{\phi_{i+1} - \phi_i}{x_{i+1} - x_i}$$

$$\left(\frac{\partial \phi}{\partial x}\right)_i \approx \frac{\phi_i - \phi_{i-1}}{x_i - x_{i-1}}$$

$$\left(\frac{\partial \phi}{\partial x}\right)_i \approx \frac{\phi_{i+1} - \phi_{i-1}}{x_{i+1} - x_{i-1}}$$

The choice influences the accuracy of the approximation

Discretization

The choice influences the accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i)\frac{\partial \phi}{\partial x}|_i + \frac{(x - x_i)^2}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + \ldots + \frac{(x - x_i)^n}{n!}\frac{\partial^n \phi}{\partial x^n}|_i$$

$$\frac{\partial \phi}{\partial x}_i = \frac{\phi_{i+1} - \phi_{i-1}}{2\Delta x} - \frac{\Delta x^2}{3!}\frac{\partial^3 \phi}{\partial x^3}|_i + h.o.t.$$

Discretization

The choice influences the accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i)\frac{\partial \phi}{\partial x}|_i + \frac{(x - x_i)^2}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + \ldots + \frac{(x - x_i)^n}{n!}\frac{\partial^n \phi}{\partial x^n}|_i$$

Using a **forward** discretisation:

$$\frac{\partial \phi}{\partial x}|_i = \frac{\phi_{i+1} - \phi_i}{\Delta x} - \frac{\Delta x}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + h.o.t$$

$$\frac{\partial \phi}{\partial x}_i = \frac{\phi_{i+1} - \phi_{i-1}}{2\Delta x} - \frac{\Delta x^2}{3!}\frac{\partial^3 \phi}{\partial x^3}|_i + h.o.t.$$

Discretization

The choice influences the accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i)\frac{\partial \phi}{\partial x}|_i + \frac{(x - x_i)^2}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + \ldots + \frac{(x - x_i)^n}{n!}\frac{\partial^n \phi}{\partial x^n}|_i$$

Using a **forward** discretisation:

$$\frac{\partial \phi}{\partial x}|_i = \frac{\phi_{i+1} - \phi_i}{\Delta x} - \frac{\Delta x}{2!}\frac{\partial^2 \phi}{\partial x^2}|_i + h.o.t$$

Using a **central** discretisation:

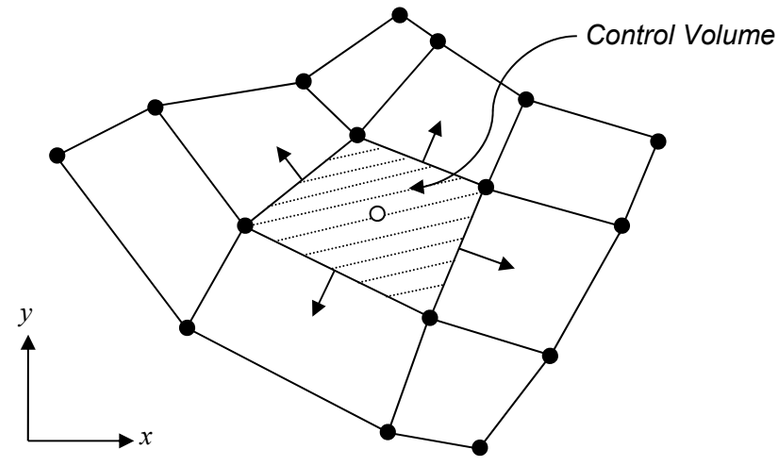
$$\frac{\partial \phi}{\partial x}_i = \frac{\phi_{i+1} - \phi_{i-1}}{2\Delta x} - \frac{\Delta x^2}{3!}\frac{\partial^3 \phi}{\partial x^3}|_i + h.o.t.$$

Discretization

Finite Volume

By using Gauss's divergence theorem in the x direction:

$$\int\int\int_V (\nabla F) dV = \int\int_A (F\vec{n}) dA$$



$$\frac{\partial \phi}{\partial x} = \frac{1}{\Delta V}\int_V \frac{\partial \phi}{\partial x} dV = \frac{1}{\Delta V}\int_A \phi dA^x \approx \frac{1}{\Delta V}\sum_{i=1}^{N} \phi A_i^x$$

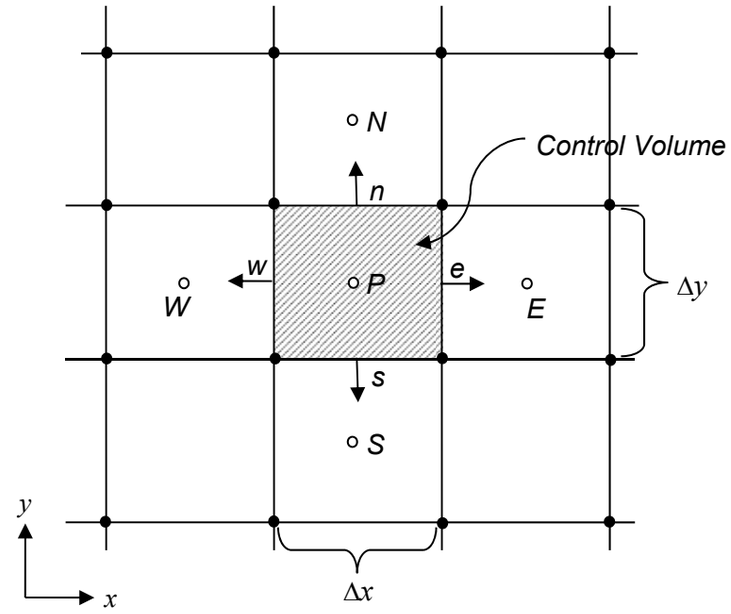
$$\frac{\partial \phi}{\partial y} = \frac{1}{\Delta V}\int_V \frac{\partial \phi}{\partial y} dV = \frac{1}{\Delta V}\int_A \phi dA^y \approx \frac{1}{\Delta V}\sum_{i=1}^{N} \phi A_i^y$$

Discretization

Finite Volume

Determine the discretised form of the 2D continuity equation using the FV method in a structured uniform grid arrangement:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$



$$\frac{\partial u}{\partial x} = \frac{1}{\Delta V}\int_A u dA^x \approx \frac{1}{\Delta V}\sum_{i=1}^{N} u_i A_i^x = \frac{1}{\Delta V}(u_e A_e^x - u_w A_w^x + \cancelto{0}{v_n A_n^x} - \cancelto{0}{v_s A_s^x}) = \frac{1}{\Delta V}(u_e A_e^x - u_w A_w^x)$$

$$\frac{\partial v}{\partial y} = \frac{1}{\Delta V}\int_A v dA^y \approx \frac{1}{\Delta V}\sum_{i=1}^{N} v_i A_i^y = \frac{1}{\Delta V}(\cancelto{0}{v_e A_e^y} - \cancelto{0}{v_w A_w^y} + v_n A_n^y - v_s A_s^y) = \frac{1}{\Delta V}(v_n A_n^y - v_s A_s^y)$$

Algebraic form of equations

$$a_P T_P = a_E T_E + a_W T_W + b$$

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

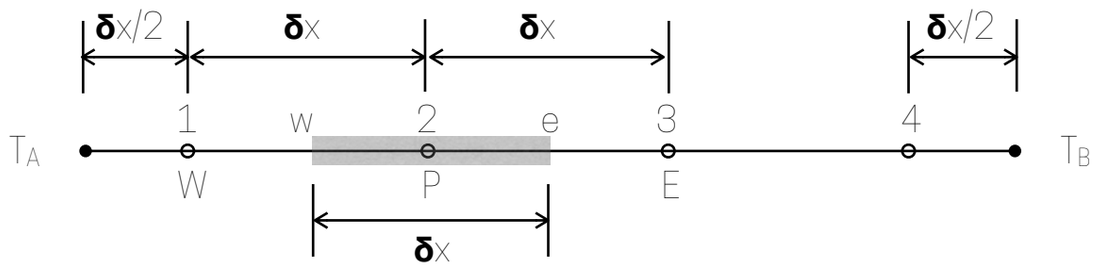
Specifically for each node:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:

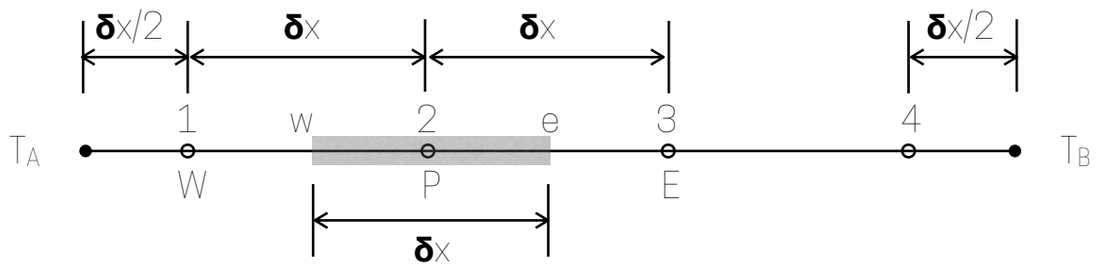


Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3:

- Node 1:

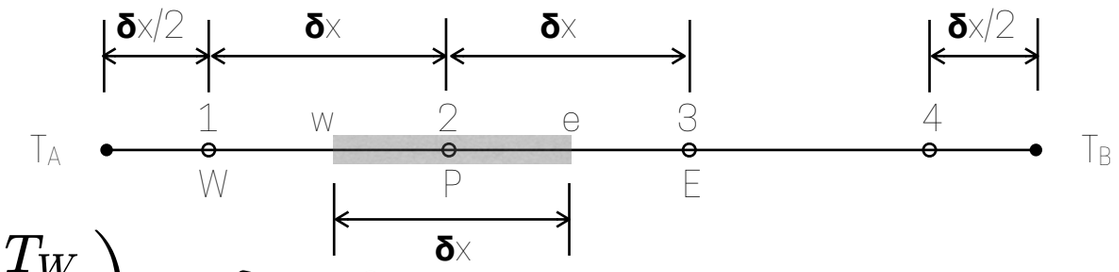
- Node 4:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3: $kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$

$$\frac{kA}{\delta x}T_E - \frac{2kA}{\delta x}T_P + \frac{kA}{\delta x}T_W + q\delta x = 0 \qquad a_E T_E - a_P T_P + a_W T_W + b = 0$$

- Node 1:

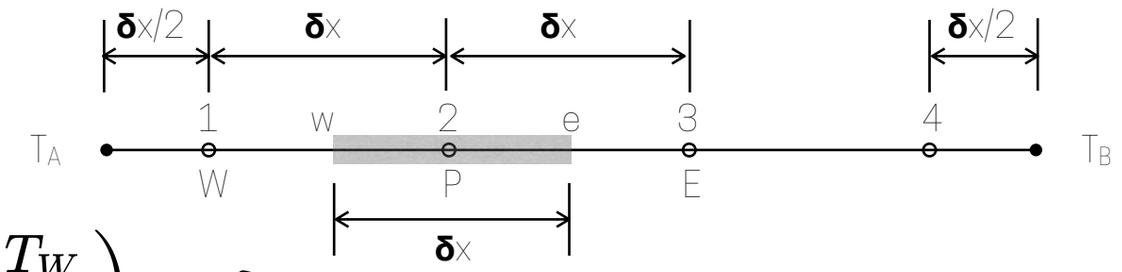
- Node 4:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{2kA}{\delta x}T_P + \frac{kA}{\delta x}T_W + q\delta x = 0 \qquad a_E T_E - a_P T_P + a_W T_W + b = 0$$

- Node 1:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_A}{\frac{\delta x}{2}}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{3kA}{\delta x}T_P + \frac{2kA}{\delta x}T_A + q\delta x = 0$$

$$a_E T_E - a_P T_P + b = 0$$

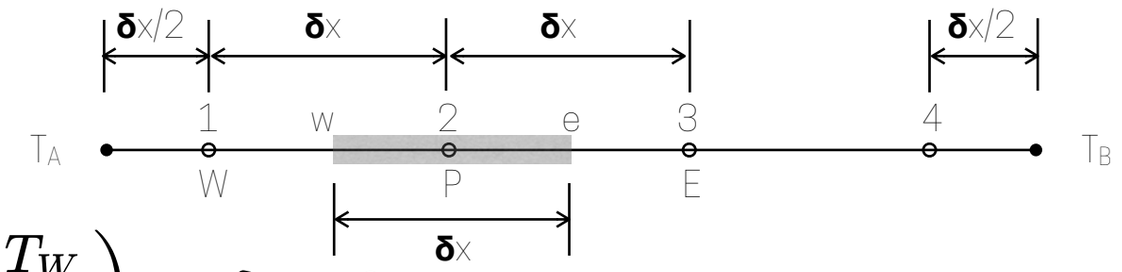
- Node 4:

Algebraic form of equations

Rearranged into algebraic form:

$$a_P T_P = a_E T_E + a_W T_W + b$$

Specifically for each node:



- Node 2 and 3:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{2kA}{\delta x}T_P + \frac{kA}{\delta x}T_W + q\delta x = 0 \qquad a_E T_E - a_P T_P + a_W T_W + b = 0$$

- Node 1:

$$kA\left(\frac{T_E - T_P}{\delta x}\right) - kA\left(\frac{T_P - T_A}{\frac{\delta x}{2}}\right) + q\delta x = 0$$

$$\frac{kA}{\delta x}T_E - \frac{3kA}{\delta x}T_P + \frac{2kA}{\delta x}T_A + q\delta x = 0 \qquad a_E T_E - a_P T_P + b = 0$$

- Node 4:

$$kA\left(\frac{T_B - T_P}{\frac{\delta x}{2}}\right) - kA\left(\frac{T_P - T_W}{\delta x}\right) + q\delta x = 0$$

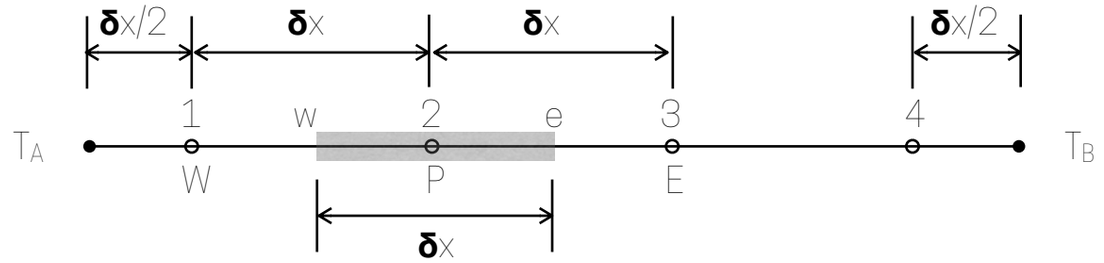
$$\frac{2kA}{\delta x}T_B - \frac{3kA}{\delta x}T_P + \frac{kA}{\delta x}T_w + q\delta x = 0 \qquad a_W T_W - a_P T_P + b = 0$$

Algebraic form of equations

$$\begin{pmatrix} \frac{3kA}{\delta x} & -\frac{kA}{\delta x} & 0 & 0 \\ -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} & 0 \\ 0 & -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} \\ 0 & 0 & -\frac{kA}{\delta x} & \frac{3kA}{\delta x} \end{pmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{pmatrix} \frac{2kA}{\delta x} T_A + q\delta x \\ q\delta x \\ q\delta x \\ \frac{2kA}{\delta x} T_B + q\delta x \end{pmatrix}$$

Iterative Methods!

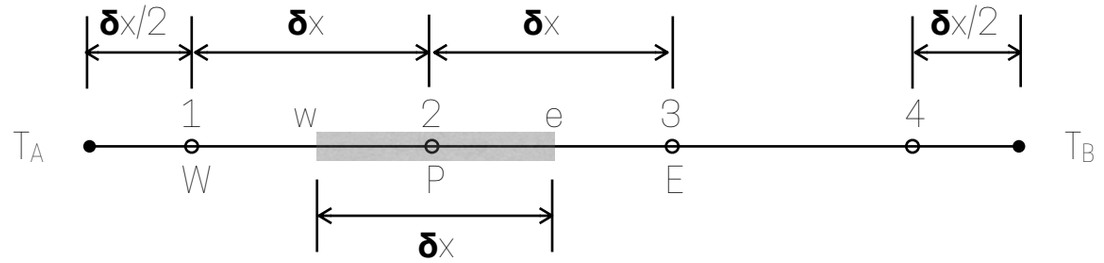
Algebraic form of equations



$$\begin{pmatrix} \frac{3kA}{\delta x} & -\frac{kA}{\delta x} & 0 & 0 \\ -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} & 0 \\ 0 & -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} \\ 0 & 0 & -\frac{kA}{\delta x} & \frac{3kA}{\delta x} \end{pmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{pmatrix} \frac{2kA}{\delta x} T_A + q\delta x \\ q\delta x \\ q\delta x \\ \frac{2kA}{\delta x} T_B + q\delta x \end{pmatrix}$$

Iterative Methods!

Algebraic form of equations



$$\begin{pmatrix} \frac{3kA}{\delta x} & -\frac{kA}{\delta x} & 0 & 0 \\ -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} & 0 \\ 0 & -\frac{kA}{\delta x} & \frac{2kA}{\delta x} & -\frac{kA}{\delta x} \\ 0 & 0 & -\frac{kA}{\delta x} & \frac{3kA}{\delta x} \end{pmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{pmatrix} \frac{2kA}{\delta x} T_A + q\delta x \\ q\delta x \\ q\delta x \\ \frac{2kA}{\delta x} T_B + q\delta x \end{pmatrix}$$

In this example we can invert that matrix very easy….however, in general for CFD, equations are not linear and matrices are extremely large…so inverting them is not the most efficient solution

→ Iterative Methods!